Contents

1 Introduction to MatPL **1**

2 Installation Guide **3**
2.1 Offline and Online Installation . . . 3
2.1.1 Lonxun Supercomputing Cloud (Mcloud) Users . . . 3
2.1.2 Lonxun Integrated Appliance (GPU) Users . . . 3
2.1.3 Lonxun Integrated Appliance (CPU) Users . . . 4
2.2 Offline Installation . . . 4
2.2.1 Download the Offline Package . . . 5
2.2.2 Extract the Package . . . 5
2.2.3 Check Compiler Versions . . . 5
2.2.4 Run the Installation Command . . . 6
2.2.5 Load the Environment . . . 7
2.2.6 LAMMPS Packages Included in the Offline Installation . . . 8
2.3 Online Installation . . . 8
2.3.1 Build and Install MatPL . . . 9
2.3.2 Build and Install the Open-Source MatPL LAMMPS 2026.3 Interface . . . 13
2.3.3 Build and Install the Closed-Source MatPL LAMMPS 2026.3 Interface . . . 17
2.3.4 Run LAMMPS . . . 21
2.4 Common Installation Errors . . . 24
2.4.1 Build Environment . . . 24
2.4.2 Issue: `cuda_runtime.h` Header Not Found . . . 25
2.4.3 Issue: NeighConst.so Compilation Error . . . 25
2.4.4 Issue: `Unknown CUDA Architecture Name 9.0a` When Building the Offline Package . . . 26
2.4.5 Issue: NVCC Reports `fatal error: math.h: No such file or directory` . . . 28
2.5 Common Runtime Errors . . . 29
2.5.1 Check Environment Variables . . . 29
2.5.2 Issue: CUDA driver version is insufficient for CUDA runtime version . . . 29
2.5.3 Shared-Library Loading Error: MKL . . . 31
2.5.4 Common LAMMPS Interface Runtime Errors . . . 31

3 MatPL Commands **34**
3.1 train . . . 34
3.2 test . . . 35
3.3 ASE Interface . . . 37
3.4 Other Utility Commands . . . 37
3.4.1 extract_ff . . . 37

3.4.2 infer . . . 37
3.4.3 totxt . . . 38
3.4.4 compress . . . 38
3.4.5 script . . . 38
3.5 Using Force Fields in LAMMPS . . . 39

4 MatPL Parameters 41
4.1 Basic Parameters . . . 41
4.1.1 model_type . . . 41
4.1.2 atom_type . . . 41
4.1.3 train_data . . . 41
4.1.4 valid_data . . . 41
4.1.5 test_data . . . 41
4.1.6 format . . . 42
4.1.7 model_load_file . . . 42
4.1.8 nep_txt_file . . . 42
4.1.9 recover_train . . . 42
4.1.10 reserve_work_dir . . . 42
4.1.11 save_step . . . 42
4.1.12 max_save_num . . . 42
4.2 NEP Model Hyperparameters . . . 43
4.2.1 cutoff . . . 43
4.2.2 n_max . . . 43
4.2.3 basis_size . . . 43
4.2.4 l_max . . . 43
4.2.5 network_size . . . 44
4.2.6 zbl . . . 44
4.2.7 use_typewise_cutoff_zbl . . . 44
4.2.8 fix_cij . . . 44
4.2.9 fix_hiddenlayer . . . 44
4.2.10 fix_outlayer . . . 44
4.2.11 Interpreting NEP.txt . . . 44
4.3 DP Model Hyperparameters . . . 45
4.3.1 type_embedding . . . 46
4.3.2 Rmax . . . 46
4.3.3 Rmin . . . 46
4.3.4 M2 . . . 46
4.3.5 network_size . . . 47
4.4 NN Model Hyperparameters . . . 47
4.4.1 Rmax . . . 47
4.4.2 Rmin . . . 47

4.4.3 feature_type . . . 47
4.4.4 network_size . . . 47
4.5 Linear Model Hyperparameters . . . 48
4.5.1 Rmax . . . 48
4.5.2 Rmin . . . 48
4.5.3 feature_type . . . 48
4.6 ADAM Optimizer Hyperparameters . . . 48
4.6.1 optimizer . . . 49
4.6.2 reset_epoch . . . 49
4.6.3 epochs . . . 49
4.6.4 batch_size . . . 49
4.6.5 print_freq . . . 49
4.6.6 train_energy . . . 50
4.6.7 train_force . . . 50
4.6.8 train_virial . . . 50
4.6.9 lambda_2 . . . 50
4.6.10 learning_rate . . . 50
4.6.11 stop_lr . . . 50
4.6.12 stop_step . . . 50
4.6.13 decay_step . . . 51
4.6.14 Gradient Clipping . . . 51
4.6.15 Cosine Annealing with Warm Restarts: t_0 and t_mult . . . 52
4.6.16 Statistics of the first five restart epochs for each group . . . 52
4.6.17 Detailed calculation process . . . 53
4.6.18 Learning-Rate Scaling (scale_lr / scaling_method) . . . 53
4.6.19 Energy and Force Prefactors in the Loss . . . 54
4.7 KF Optimizer Hyperparameters . . . 55
4.7.1 block_size . . . 55
4.7.2 p0_weight . . . 56
4.7.3 kalman_lambda . . . 56
4.7.4 kalman_nue . . . 56
4.7.5 pre_fac_etot . . . 56
4.7.6 pre_fac_force . . . 56
4.7.7 pre_fac_virial . . . 56

5 MatPL Tutorials 56
5.1 NEP Model . . . 57
5.1.1 Model Overview . . . 57
5.1.2 NEP Training Benchmarks . . . 59
5.1.3 LAMMPS Interface Benchmarks . . . 59

5.2 NEP Tutorial . . . 60
5.2.1 train . . . 61
5.2.2 Multi-Node, Multi-GPU Training . . . 63
5.2.3 test . . . 63
5.2.4 infer a Single Structure . . . 64
5.2.5 totxt: Convert a Trained Checkpoint to nep5.txt . . . 64
5.2.6 lammps MD . . . 65
5.2.7 ASE Interface . . . 67
5.3 DP Model . . . 68
5.3.1 DP Model Overview . . . 68
5.3.2 type embedding . . . 69
5.3.3 Performance Benchmark: Accuracy . . . 70
5.3.4 Performance Benchmark: Training Time . . . 71
5.3.5 Polynomial Model Compression . . . 71
5.3.6 Polynomial Model Compression Procedure . . . 73
5.4 DP Tutorial . . . 76
5.4.1 train . . . 77
5.4.2 test . . . 78
5.4.3 infer a Single Structure . . . 78
5.4.4 compress a Model . . . 78
5.4.5 script: Convert to an MD Force Field . . . 79
5.4.6 lammps MD . . . 79
5.4.7 ASE Interface . . . 80
5.5 NN Model . . . 80
5.5.1 Neural Network (NN) Model Overview . . . 80
5.5.2 2-b and 3-b features with piecewise cosine functions (feature 1 & 2) . . . 81
5.5.3 2-b and 3-b Gaussian feature (feature 3 & 4) . . . 83
5.5.4 Moment Tensor Potential (feature 5) . . . 84
5.5.5 Spectral Neighbor Analysis Potential (feature 6) . . . 85
5.5.6 DP-Chebyshev (feature 7) . . . 86
5.5.7 DP-Gaussian (feature 8) . . . 87
5.6 NN Tutorial . . . 88
5.6.1 train . . . 88
5.6.2 test . . . 89
5.6.3 extract_ff . . . 89
5.6.4 lammps MD . . . 90
5.7 LINEAR Model . . . 90
5.7.1 Model Overview . . . 90
5.8 LINEAR Tutorial . . . 90
5.8.1 train . . . 91

5.8.2 test . . . 92
5.8.3 lammps MD . . . 92

6 PWact Active Learning 93
6.1 PWact Platform Overview . . . 93
6.1.1 Pretraining-Data Preparation . . . 94
6.1.2 Active Learning . . . 95
6.1.3 Dependencies . . . 96
6.1.4 Installation . . . 96
6.1.5 Command List . . . 97
6.1.6 Input Files . . . 99
6.1.7 Active-Learning Examples . . . 99
6.2 init_bulk param.json Configuration . . . 99
6.2.1 data_format . . . 99
6.2.2 reserve_work . . . 99
6.2.3 interval . . . 100
6.2.4 sys_config_prefix . . . 100
6.2.5 sys_configs . . . 100
6.2.6 sys_config Example . . . 101
6.2.7 dft_style . . . 102
6.2.8 pseudo . . . 102
6.2.9 gaussian_param . . . 102
6.2.10 relax_input . . . 103
6.2.11 aimd_input . . . 103
6.2.12 bigmodel_input . . . 103
6.2.13 direct_input . . . 104
6.2.14 Complete Example . . . 104
6.3 run param.json Configuration . . . 107
6.3.1 reserve_work . . . 108
6.3.2 reserve_md_traj . . . 108
6.3.3 reserve_scf_files . . . 108
6.3.4 data_format . . . 108
6.3.5 init_data . . . 108
6.3.6 valid_data . . . 108
6.3.7 init_model_list . . . 108
6.3.8 use_pre_model . . . 109
6.3.9 train . . . 109
6.3.10 strategy . . . 110
6.3.11 explore . . . 112
6.3.12 run param.json Example . . . 122

6.4 resource Configuration . . . 124
6.4.1 resource.json File . . . 124
6.4.2 Parameter Details . . . 127
6.4.3 group_size . . . 128
6.4.4 number_node . . . 128
6.4.5 gpu_per_node . . . 129
6.4.6 cpu_per_node . . . 129
6.4.7 queue_name . . . 129
6.4.8 custom_flags . . . 129
6.4.9 source_list . . . 130
6.4.10 module_list . . . 130
6.4.11 env_list . . . 130
6.4.12 Configuration Example: train Module . . . 131
6.4.13 Configuration Example: explore Module . . . 132
6.4.14 Configuration Example: DFT Module . . . 133
6.5 Active-Learning Example for Silicon . . . 134
6.5.1 init_bulk Example . . . 134
6.5.2 init_bulk Directory Structure . . . 135
6.5.3 init_bulk Directory Structure: collection . . . 135
6.5.4 init_bulk Directory Structure: Tree . . . 135
6.5.5 run Active-Learning Example . . . 136
6.5.6 run Active-Learning Directory . . . 136
6.5.7 run 00.train Directory . . . 138
6.5.8 run 01.explore Directory . . . 138
6.5.9 run 01.explore Directory: md . . . 138
6.5.10 run 01.explore Directory: select . . . 139
6.5.11 run 02.label Directory . . . 140
6.5.12 run 02.label Directory: scf . . . 140
6.5.13 run 02.label Directory: result . . . 140
6.5.14 run Active-Learning Directory Tree . . . 140
6.6 Silicon Example (Foundation-Model Sampling) . . . 142
6.6.1 Prepare Pretraining Data with init_bulk . . . 143
6.6.2 init_bulk Directory Structure . . . 143
6.6.3 Integrate Foundation-Model MD into init_bulk . . . 144
6.6.4 Integrate DIRECT Sampling into init_bulk . . . 144
6.6.5 Active Learning with run . . . 145
6.6.6 Active-Learning Directory . . . 145
6.6.7 train Directory . . . 145
6.6.8 explore Directory . . . 145
6.6.9 label Directory . . . 145

6.6.10 Integrate Foundation-Model Labeling into run . . . 146
6.6.11 result . . . 146
6.7 Au–Ag Alloy Example . . . 146
6.7.1 Prepare Pretraining Data with init_bulk . . . 147
6.7.2 init_bulk Directory Structure . . . 147
6.7.3 Active Learning with run . . . 147
6.7.4 Active-Learning Directory . . . 147

7 pwdata Data Conversion 149
7.1 Overview . . . 149
7.1.1 Supported Data Types . . . 149
7.1.2 Installation . . . 150
7.2 Command-Line Usage . . . 150
7.2.1 pwdata convert_config: Convert a Structure . . . 150
7.2.2 pwdata convert_configs: Convert Training Data . . . 151
7.2.3 pwdata convert_configs Examples . . . 153
7.2.4 Meta Query Examples for convert_configs . . . 153
7.2.5 pwdata scale_cell: Scale a Lattice . . . 155
7.2.6 pwdata super_cell: Build a Supercell . . . 156
7.2.7 pwdata perturb: Perturb a Lattice and Atomic Positions . . . 157
7.2.8 pwdata count: Count Structures . . . 158
7.3 Using pwdata as a Python Library . . . 159
7.3.1 pwdata Config: Read Data . . . 159
7.3.2 pwdata Config.to(): Write Data . . . 161
7.3.3 pwdata build.supercells: Supercell Construction . . . 163
7.3.4 pwdata pertub.perturbation: Structure Perturbation . . . 164
7.3.5 pwdata pertub.scale: Cell Scaling . . . 164
7.3.6 pwdata API Example: Converting MPtraj Files to LMDB Format . . . 165

8 MatPL Literature Examples 168
8.1 Selected Literature . . . 168
8.2 Comparison of NN Descriptors . . . 172
8.3 Silicon Melt-Growth Process . . . 174
8.4 Fe-H System Force Field . . . 178
8.5 Morphological Evolution of Lithium Dendrites . . . 180
8.6 Mg-Cu Alloy Force Field . . . 184
8.7 Calculating the Thermal Conductivity of Amorphous Silicon with a Machine-Learning Force Field . . . 187

1 Introduction to MatPL

Materials Potential Library (MatPL, formerly PWMLFF; current version: MatPL-2026.3) is an open-source software package distributed under the GNU GPL 3.0 license.

MatPL provides a comprehensive collection of software, tools, and data repositories for rapidly developing machine-learning force fields with accuracy comparable to ab initio molecular dynamics (AIMD). The ecosystem includes the MatPL model-training platform, a LAMMPS molecular-dynamics interface, the pwact active-learning data-generation platform, the pwdata format-conversion tool, and repositories for data and models. Use the links below to access their source code and documentation.

Download the MatPL User Manual from Gitee or download it from GitHub

MatPL is also available as a free open-source machine-learning force-field application on the National Supercomputing Internet platform.

1. MatPL Machine-Learning Platform

Source repository

MatPL rapidly trains machine-learning force fields with accuracy comparable to AIMD. NEP models support efficient large-batch training across multiple GPUs and compute nodes.

2. LAMMPS Interface

Source repository

This high-performance molecular-dynamics interface integrates MatPL force-field models with GPU acceleration. The NEP interface uses LAMMPS KOKKOS acceleration, provides excellent multi-node parallel efficiency, and supports simulations with more than one hundred million atoms. A single RTX 4090 can handle systems containing more than six million atoms.

3. PWact Active-Learning Tool

Source repository

PWact is an open-source MatPL-based platform for automated active-learning data generation. It integrates MatPL, the LAMMPS interface, and widely used first-principles packages such as PWmat, VASP, and CP2K. PWact automates job dispatch, monitoring, fault recovery, and result collection, enabling users to generate training datasets that cover broad regions of phase space quickly and cost-effectively.

4. PWdata Structure-Conversion Tool

Source repository

pwdata is MatPL′s data-preprocessing tool for extracting features and labels. It converts structure formats among PWmat, VASP, CP2K, and LAMMPS, and supports supercell construction, lattice scaling, and atomic-position perturbation.

5. MatPL Examples

Test results and application examples for MatPL.

MatPL Citation

https://chemrxiv.org/doi/full/10.26434/chemrxiv.15001665

2 Installation Guide

2.1 Offline and Online Installation

MatPL-2026.3 provides both offline installation and online installation. The offline package includes the complete Python runtime, eliminating the time required to install the Python environment separately.

For users of Lonxun Supercomputing Cloud (`Mcloud`) or a `Lonxun integrated appliance`, MatPL is preinstalled and only needs to be loaded as shown below.

The primary difference between MatPL-2026.3 and MatPL-2025.3 is the extensive optimization of NEP. For single-GPU training, optimized gradient operators provide a `more than threefold speedup` over MatPL-2025.3. Multi-node, multi-GPU large-batch training has also been introduced, together with measures that mitigate the loss of accuracy associated with large batches, substantially improving efficiency on large training datasets.

Because this release does not improve CPU-only training or simulation, no online or offline CPU package is provided for MatPL-2026.3. CPU-only users should use MatPL-2025.3-cpu.

2.1.1 Lonxun Supercomputing Cloud (Mcloud) Users

```
# Load MatPL
source /share/app/MATPL/MatPL-2026.3/env.sh

# Alternatively, load it step by step as follows
# Step 1: Load the Python runtime
# Load Conda
module load conda/3-2020.07
eval "$(conda shell.bash hook)"
# Activate the Python environment
conda activate matpl-2026.3
# Step 2: Load MatPL
module load matpl/2026.3

# Load LAMMPS
module load lammps4matpl/2026.3
# For Linear and NN models, load the CPU LAMMPS interface implemented
 ↪  in Fortran
module load lammps4matpl/fortran
```

2.1.2 Lonxun Integrated Appliance (GPU) Users

```
# Load MatPL and LAMMPS
```

```
module load cuda/11.8 openmpi/4.1.6 # OpenMPI is required to run
 ↪  LAMMPS
# Replace this with the actual installation path
source /share/app/MATPL/MatPL-2026.3/matpl-env.sh

# For Linear and NN models, load the CPU LAMMPS interface implemented
 ↪  in Fortran
module load intel/2020
source /share/app/MATPL/MatPL-2026.3/matpl-fortran-env.sh
```

2.1.3 Lonxun Integrated Appliance (CPU) Users

This release does not improve CPU-only training or simulation, so no online or offline CPU package is provided for MatPL-2026.3. CPU-only users should use MatPL-2025.3.

The `lammps4matpl/2026.3` module on the Mcloud platform and the offline installation package include the following commonly used LAMMPS packages:

CG-SPICA CLASS2 COLLOID COLVARS COMPRESS CORESHELL DIELECTRIC DIFFRACTION DIPOLE DPD-BASIC DPD-MESO DPD-REACT DPD-SMOOTH DRUDE EFF ELECTRODE EXTRA-COMMAND EXTRA-COMPUTE EXTRA-FIX EXTRA-DUMP EXTRA-MOLECULE EXTRA-PAIR FEP GRANULAR INTEL INTERLAYER KSPACE LATBOLTZ LEPTON MANIFOLD MANYBODY MC MEAM MGPT MISC MOFFF MOLECULE MOLFILE OPENMP OPT ORIENT PERI PHONON PLUMED POEMS PTM PYTHON QEQ QMMM QTB REACTION REAXFF REPLICA RIGID SHOCK SMTBQ SPH SPIN SRD TALLY UEF VORONOI YAFF

Run `lmp -h` to list all compiled pair styles.

2.2 Offline Installation

For Lonxun Supercomputing Cloud (`Mcloud`) users, MatPL is preinstalled and ready to load. The offline package provides a GPU version.

- The offline package bundles MatPL-2026.3, the open-source MatPL-2026.3 LAMMPS interface, and the required Python environment. The target machine must provide GCC `8.x or later`, `CUDA 11.8 or later`, `OpenMPI 4.1.4 or later`, and an NVIDIA GPU. `NN and Linear models additionally require the Intel toolchain (ifort, icc, and MKL).`
- Because MatPL-2026.3 does not improve CPU-only training or simulation, no online or offline CPU package is provided. CPU-only users should use MatPL-2025.3.
- The offline package includes PyTorch 2.2. The latest package is `matpl-2026.3-update3`. Compared with update1, it adds type-wise ZBL support to NEP, accelerates

LAMMPS NEP Kokkos on NVIDIA GPUs, and extends the six-component LAMMPS Kokkos virial to nine components to support heat-flux calculations. The bundled LAMMPS version is `stable_22Jul2025_update4`.

2.2.1 Download the Offline Package

Method 1 (recommended): Request the package by email from `matpl@pwmat.com`, `wuxingxing@pwmat.com`, or `support@pwmat.com`. Downloads from the emailed link are significantly faster than from Baidu Netdisk.

Method 2: Download from Baidu Netdisk. If the link expires, contact `matpl@pwmat.com`, `wuxingxing@pwmat.com`, or `support@pwmat.com`: Download the offline package. Access code: `pwmt`.

Method 3: Visit the MatPL-2026.3_update3 GitHub release and download the following files:

```
matpl-2026.3-update3.sh.tar.gz.part_aa
 ↪  matpl-2026.3-update3.sh.tar.gz.part_ab
 ↪  matpl-2026.3-update3.sh.tar.gz.part_ac
 ↪  matpl-2026.3-update3.sh.tar.gz.part_ad
 ↪  matpl-2026.3-update3.sh.tar.gz.part_ae
```

2.2.2 Extract the Package

Because the offline package is large, it is split into several smaller files. Reassemble and extract it as follows:

```
# Combine the parts into one archive
cat matpl-2026.3-update3.sh.tar.gz.part_aa
 ↪  matpl-2026.3-update3.sh.tar.gz.part_ab
 ↪  matpl-2026.3-update3.sh.tar.gz.part_ac
 ↪  matpl-2026.3-update3.sh.tar.gz.part_ad
 ↪  matpl-2026.3-update3.sh.tar.gz.part_ae >
 ↪  matpl-2026.3-update3.sh.tar.gz
# Extract the archive
tar -xzvf matpl-2026.3-update3.sh.tar.gz
```

Extraction produces the following files: `matpl-2026.3-update3.sh`,`check_offenv.sh`

2.2.3 Check Compiler Versions

Most installation failures result from incompatible compiler versions. Use the provided `check_offenv.sh` script to inspect the environment:

```
bash check_offenv.sh
```

The script lists the required compiler versions and those detected in the current environment. A valid result looks like this:

```
========================================
      Environment Check Starting
========================================

=== Checking ifort compiler and MKL library ===
✓ ifort version: 19.1 (>= 19.1)
✓ MKL library is installed

=== Checking GCC version ===
✓ GCC version: 8 (>= 8.0)

=== Checking CUDA version ===
✓ CUDA version: 11.8.89 (>= 11.8)

=== Checking nvcc availability ===
✓ nvcc command exists

=== Checking OpenMPI ===
✓ OpenMPI found (via ompi_info), version: 4.1.6
✓ OpenMPI version meets requirement (>= 4.0)

========================================
        Environment Summary
========================================
✓ Environment check completed. All requirements are satisfied.
========================================
```

2.2.4 Run the Installation Command

After the environment check, install the package with:

```
bash matpl-2026.3-update3.sh [-jN] [-m nn] [-a ARCH] [-d] [-u] [-h]
```

- `-jN`: N is the number of parallel compilation cores. For example, `bash matpl-2026.3-update3.sh -j4` uses four cores. The default is one core.
- `-m nn`: Also compiles the Fortran code for Linear and NN models. This requires Intel ifort, icc, and MKL. Fortran code is not compiled by default.
- `-a ARCH`: Selects the Kokkos CUDA architecture used to build `lammps-2026.3`. The default is AMPERE86. For example:

```
bash matpl-2026.3-update3.sh -a AMPERE80
```

ARCH is a Kokkos CUDA architecture suffix, such as AMPERE80, AMPERE86, ADA89, or HOPPER90. For additional architectures, see https://docs.lammps.org/Build_extras.html#kokkos.

- -d: Enables 64-bit-precision NEP inference when building `lammps-2026.3`:

  ```
  -DPREC_NEPINFER=ON
  ```

  When enabled, `lammps-2026.3` is built in `build-64`. By default, this option is disabled and the build directory is `build`.

- -u: Extracts the package without building it, for environments that require installation in separate stages. The extracted contents include the `lammps-2026.3` source directory, the `MatPL-2026.3` source directory, and the `matpl-2026.3` Python environment.

To verify the installation:

After compilation, the `MatPL-2026.3` directory has the following structure:

```
MatPL-2026.3
    ├── lammps-2026.3/
    ├── matpl-2026.3/
    ├── MatPL-2026.3/
    └── matpl-env.sh
```

- `MatPL-2026.3` is the machine-learning force-field training platform.
- `matpl-2026.3` is the Python environment.
- `lammps-2026.3` is the LAMMPS interface for DP and NEP force fields.
- `matpl-env.sh` defines all environment variables for MatPL-2026.3 and `lammps-2026.3`.

2.2.5 Load the Environment

Before using the GPU version of MatPL, load the same CUDA environment used during compilation. NN or Linear training also requires MKL. Then run the following command to load the Python environment, MatPL-2026.3, and LAMMPS:

```
source /the/path/of/MatPL-2026.3/matpl-env.sh

# For the Fortran LAMMPS interface:
source /the/path/of/MatPL-2026.3/matpl-fortran-env.sh
```

You can also load each component separately:

```
## Step 1. Load the Python environment
source /the/path/MatPL-2026.3/matpl-2026.3/bin/activate
## Step 2. Load MatPL-2026.3
source /the/path/MatPL-2026.3/MatPL-2026.3/env.sh
## Step 3. Load LAMMPS
source /the/path/MatPL-2026.3/lammps-2026.3/env.sh
```

2.2.6 LAMMPS Packages Included in the Offline Installation

The LAMMPS offline package includes the following commonly used packages:

CG-SPICA CLASS2 COLLOID COLVARS COMPRESS CORESHELL DIELECTRIC DIFFRACTION DIPOLE DPD-BASIC DPD-MESO DPD-REACT DPD-SMOOTH DRUDE EFF ELECTRODE EXTRA-COMMAND EXTRA-COMPUTE EXTRA-FIX EXTRA-DUMP EXTRA-MOLECULE EXTRA-PAIR FEP GRANULAR INTEL INTERLAYER KSPACE LATBOLTZ LEPTON MANIFOLD MANYBODY MC MEAM MGPT MISC MOFFF MOLECULE MOLFILE OPENMP OPT ORIENT PERI PHONON PLUMED POEMS PTM PYTHON QEQ QMMM QTB REACTION REAXFF REPLICA RIGID SHOCK SMTBQ SPH SPIN SRD TALLY UEF VORONOI YAFF

2.3 Online Installation

For Lonxun Supercomputing Cloud (Mcloud) users, MatPL is preinstalled and ready to load.

An online installation requires building and installing MatPL-2026.3 and its LAMMPS interface separately.

- MatPL-2026.3 is used for model training. The target machine must provide GCC `8.x or later`, CUDA `11.8 or later`, OpenMPI `4.1.4 or later`, and an NVIDIA GPU. The Python environment requires Python >= 3.11 and PyTorch >= 2.2.0+cu118.
- NN and Linear models additionally require the Intel toolchain (ifort, icc, and MKL). The `intel/2020` suite provides `ifort` and `icc` version `19.1.3` and MKL. If loading these components separately, use versions no older than these.
- The MatPL-2026.3 LAMMPS interface runs NEP and DP models. Installation and execution require OpenMPI; version `4.1.4` or later is recommended. Two interface variants are available:

  The open-source interface, whose performance is comparable to GPUMD;

  The closed-source interface, which is more than twice as fast as the open-source version.

2.3.1 Build and Install MatPL

To build and run MatPL-2026.3, download the source, create a Conda environment, install the required Python dependencies in that environment, and then compile the source.

In the commands below, `sh` means `bash`. If you invoke scripts with `sh`, verify that it links to `bash`; on some systems it links to `dash` instead.

2.3.1.1 Download the Source You can clone the repository online or download a source archive.

- Clone the MatPL repository from GitHub or Gitee:

```
git clone https://github.com/LonxunQuantum/MatPL.git MatPL-2026.3
or
git clone https://gitee.com/pfsuo/MatPL.git MatPL-2026.3
```

- Alternatively, download a release archive in a browser or with `wget`:

```
wget
 ↪  https://github.com/LonxunQuantum/MatPL/archive/refs/tags/MatPL-
 ↪  2026.3_update2.zip
or download from Gitee
wget https://gitee.com/pfsuo/MatPL/repository/archive/2026.3
```

Extract the downloaded release archive with `unzip`.

```
## Extraction creates a source directory named
 ↪  MatPL-MatPL-2026.3_update2
unzip MatPL-MatPL-2026.3_update2.zip
mv MatPL-MatPL-2026.3_update2 MatPL-2026.3 # Rename it for
 ↪  consistency with the rest of this guide
```

2.3.1.2 Create a Conda Virtual Environment Install Anaconda3 if it is not already available, then create a new Python virtual environment. Consult an Anaconda3 installation guide for Linux if necessary.

Download Anaconda3 directly to the server with:

```
curl https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/Anaconda3-
 ↪  2023.07-1-Linux-x86_64.sh -o Anaconda3-2023.07-1-Linux-x86_64.sh
## If the download fails, open the URL below in a browser and upload
 ↪  the downloaded file to the server
## https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/Anaconda3-
 ↪  2023.07-1-Linux-x86_64.sh
## Other versions are available at
 ↪  https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/
```

2.3.1.3 Create the Python Virtual Environment After installing Conda, create an environment with the `Python 3.11` interpreter. Other versions may cause dependency conflicts or unsupported syntax. Perform all subsequent compilation in this environment.

```
conda create -n matpl-2026.3 python=3.11 setuptools=68.0.0
## Python 3.11 is recommended; newer versions may cause build errors
 ↪  and are not yet supported. Use setuptools below 75.0.0 to avoid
 ↪  NumPy-Fortran data-conversion errors.
```

Activate the environment after creation:

```
conda activate matpl-2026.3
```

2.3.1.4 Install Dependencies in the Python Environment The required third-party packages are listed in `requirements.txt`. Run `pip` from the directory containing that file to install them. This step installs PyTorch and other packages and may take some time.

- `MatPL-2026.3/requirements.txt` defines the Python environment for the GPU version.
- `MatPL-2026.3/requirements_cpu.txt` defines the Python environment for the CPU version.

```
# Step 1: Activate the Conda environment
conda activate matpl-2026.3
# Step 2: Enter the source root
# For a cloned repository, enter MatPL-2026.3
cd MatPL-2026.3
# For the GPU version
pip install -r requirements.txt
# For the CPU version
pip install -r requirements_cpu.txt
```

2.3.1.5 Build and Install: Check the GPU Build Environment Enter the `src` directory alongside `requirements.txt`. For the GPU version, verify `GCC 8.x or later`, `CUDA 11.8 or later`, `OpenMPI 4.1.4 or later`, and NVIDIA GPU hardware. `NN and Linear models additionally require the Intel toolchain (ifort, icc, and MKL).` Because compiler-version problems cause most build failures, use `/src/check/check_env.sh` to verify the environment.

```
cd src
bash ./check/check_env.sh
```

The script prints information about the build environment.

```
========================================
      Environment Check Starting
========================================

=== Checking ifort compiler and MKL library ===
✓ ifort version: 19.1 (>= 19.1)
✓ MKL library is installed

=== Checking GCC version ===
✓ GCC version: 8 (>= 8.0)

=== Checking PyTorch installation ===
✓ PyTorch is installed

=== Checking PyTorch version ===
✓ PyTorch version: 2.2.0+cu118 (>= 2.0)

=== Checking PyTorch CUDA support ===
✓ PyTorch is compiled with CUDA 11.8

=== Checking CUDA version ===
✓ CUDA version: 11.8.89 (>= 11.8)

=== Checking nvcc availability ===
✓ nvcc command exists

========================================
        Environment Summary
========================================
✓ Environment check completed. All requirements are satisfied.
========================================
```

2.3.1.6 Build and Install: Check the CPU Build Environment Skip this step for the GPU version. The CPU version does not require CUDA. Run `/src/check/check_env_cpu.sh` to list the required and detected compiler versions:

```
========================================
      CPU Environment Check Starting
========================================

=== Checking ifort compiler and MKL library ===
✓ ifort version: 19.1 (>= 19.1)
```

```
✓ MKL library is installed

=== Checking GCC version ===
✓ GCC version: 8 (>= 8.0)

=== Checking PyTorch installation ===
✓ PyTorch is installed

=== Checking PyTorch version ===
✓ PyTorch version: 2.2.0+cu118 (>= 2.0)

========================================
        Environment Summary
========================================
✓ Environment check completed. All requirements are satisfied.
========================================
```

2.3.1.7 Build and Install: Compile the Code If the environment passes the checks above, compile the code with:

```
bash clean.sh
bash build.sh [-jN] [-m nn]
```

- `-jN`: N is the number of parallel compilation cores. For example, `sh build.sh -j4` uses four cores. The default `bash build.sh` uses one core.
- `-m nn`: Also compiles the Fortran code for Linear and NN models and requires the Intel compiler. `Fortran code is not compiled by default.`

If compilation fails, consult Common MatPL Installation Errors.

If the issue remains unresolved, email the machine-environment details, build log, and the commands you ran to `matpl@pwmat.com`, `wuxingxing@pwmat.com`, or `support@pwmat.com`.

After a successful build, output similar to the following appears:

```
[100%] Linking CXX shared library ../../lib/libCalcOps_bind.so
[100%] Built target CalcOps_bind
Operators built successfully
Creating symbolic links in bin directory...
Created symbolic link for main_MD.x

================================
```

```
MatPL has been successfully installed.
Please load the MatPL environment variables before use.

Recommended method:
 source /data/home/wuxingxing/codespace/MatPL-2025.12-tmp/env.sh

Or manually set environment variables:
     export  PYTHONPATH=/data/home/wuxingxing/codespace/MatPL-
2025.12-tmp:$PYTHONPATH
    export  PATH=/data/home/wuxingxing/codespace/MatPL-2025.12-
tmp/src/bin:$PATH
================================
```

The build creates `env.sh` in the source root. Load the MatPL environment variables with:

```
source /the/path/of/MatPL-2026.3/env.sh
```

You can also set them manually:

```
export PYTHONPATH=/the/path/of/MatPL-2026.3:$PYTHONPATH
export PATH=/the/path/of/MatPL-2026.3/src/bin:$PATH
```

2.3.1.8 Load and Use MatPL Before using MatPL, load the Python environment, CUDA, and the MatPL environment variables. The CPU version does not require CUDA.

```
conda activate matpl-2026.3
module load cuda/11.8-share
source /the/path/of/MatPL-2026.3/env.sh
```

You can then start training with MatPL commands. See the tutorials for usage examples.

2.3.2 Build and Install the Open-Source MatPL LAMMPS 2026.3 Interface

The MatPL-2026.3 LAMMPS interface supports NEP (including QNEP), DP, and D3. A Fortran interface is available for NN and Linear force fields; see the 2025.3 Fortran LAMMPS interface guide.

To install the interface, download the LAMMPS source, load the build environment, and compile it as described below.

2.3.2.1 Download the MatPL-2026.3 LAMMPS Interface Source

The open-source MatPL-2026.3 LAMMPS interface is hosted in this GitHub repository. Clone it with:

```
git clone https://github.com/LonxunQuantum/lammps-MatPL.git
cd lammps-MatPL
```

The source tree is organized as follows:

```
├── .gitignore
├── KOKKOS/
├── MATPLD3/
├── MATPLDP/
├── compute_qnep_bec_atom.cpp
├── compute_qnep_bec_atom.h
├── examples/
├── kknep-patch.sh
├── nep_cpu.cpp
├── nep_cpu.h
├── nep_gpu/
├── pair_nep.cpp
├── pair_nep.h
├── test/
├── test_qnep/
└── README.md
```

- KOKKOS, nep_gpu, nep_cpu.cpp, nep_cpu.h, pair_nep.cpp, and pair_nep.h contain the NEP LAMMPS interface with NEP Kokkos GPU acceleration. compute_qnep_bec_atom.cpp and compute_qnep_bec_atom.h output per-atom Born effective-charge tensors for QNEP.
- MATPLDP contains the DP interface; MATPLD3 contains the D3 interface. D3 requires CUDA and cannot be used together with matpl/nep/kk.
- examples contains force-field MD examples; test and test_qnep contain NEP and QNEP tests, respectively.
- kknep-patch.sh copies the interface source into the LAMMPS source tree and modifies cmake/CMakeLists.txt before compilation with CMake.
- Compared with MatPL-2025.3, the NEP Kokkos interface uses Kokkos to build neighbor lists on the GPU, while custom C++/CUDA operators evaluate energies and forces. This reduces GPU memory usage and improves inference performance.
- The repository supports the 2023, 2024, and 2025 LAMMPS releases. Recommended versions are stable_2Aug2023_update4, stable_29Aug2024_update4, and stable_22Jul2025_update4.

2.3.2.2 Copy the MatPL-2026.3 Interface into the LAMMPS Source Tree

Run `kknep-patch.sh` from the `lammps-MatPL` repository root. The script copies the interface files into the LAMMPS source tree and updates `cmake/CMakeLists.txt`:

```
bash kknep-patch.sh /the/rootpath/of/lammps
```

On success, it prints output similar to the following:

```
...
Patch process completed successfully!

Compilation Environment:
Recommended compilation environment: cuda/11.6 (with nvcc compiler)
 ↪  openmpi4.1.4 cmake/3.31.6 gcc8.n

Compilation Process:
cd /data/home/wuxingxing/codespace/suzhou/lmpversions/lammps-23-4-
 ↪  changerowcol/
mkdir build & cd build
cmake -C ../cmake/presets/basic.cmake \
    -DPKG_MESONT=no \
    -DPKG_JPEG=no \
    -DPKG_KOKKOS=yes \
    -DPKG_NEP_KK=yes \
    -DKokkos_ENABLE_CUDA=yes \
    -DKokkos_ENABLE_OPENMP=yes \
    -DKokkos_ENABLE_CUDA_LAMBDA=yes \
    -DFFT_KOKKOS=CUFFT \
    -DKokkos_ARCH_AMPERE86=ON \
    -DTEST_TIME=ON \
    ../cmake

cmake --build . --parallel 4 #(number of parallel compilation cores)


If you also need to compile the DP interface, please import the
 ↪  PyTorch path, import the MKL library, and enable the C++ STD17
 ↪  standard for compilation.
export Torch_DIR=$(python -c "import torch;
 ↪  print(torch.utils.cmake_prefix_path)")/Torch
Then, add the following option in cmake:
-DTorch_DIR=${Torch_DIR} \
-DCMAKE_CXX_STANDARD=17 \
-DPKG_MATPLDP=yes \
```

2.3.2.3 Load the Build Environment and Compile After copying the files, load the build environment. The LAMMPS interface requires CUDA, GCC, CMake, and OpenMPI. We recommend OpenMPI 4.1.4 or later, CUDA 11.6 or later, CMake 3.x or later, and GCC 8.x or later.

Following the instructions printed by the script, enter the LAMMPS root, create a `build` directory, and run the following CMake command:

```
# Enter the LAMMPS root
mkdir build & cd build
cmake -C ../cmake/presets/basic.cmake \
    -DPKG_MESONT=no \
    -DPKG_JPEG=no \
    -DPKG_KOKKOS=yes \
    -DPKG_NEP_KK=yes \
    -DKokkos_ENABLE_CUDA=yes \
    -DKokkos_ENABLE_OPENMP=yes \
    -DKokkos_ENABLE_CUDA_LAMBDA=yes \
    -DFFT_KOKKOS=CUFFT \
    -DKokkos_ARCH_AMPERE86=ON \
    -DTEST_TIME=ON \
    ../cmake

cmake --build . --parallel 4 #(number of parallel compilation cores)
```

Only the NEP interface is enabled by default. To build the DP interface, load MKL, provide the `LibTorch path from PyTorch`, and compile with the `C++17 standard`.

```
export Torch_DIR=$(python -c "import torch;
 ↪  print(torch.utils.cmake_prefix_path)")/Torch

# Enable the DP interface in the CMake command:
-DTorch_DIR=${Torch_DIR} \
-DCMAKE_CXX_STANDARD=17 \
-DPKG_MATPLDP=yes \
```

After compilation, output similar to the following appears and `env.sh` is generated in the LAMMPS source root. Load this file before using LAMMPS.

```
[ 98%] Building CUDA object CMake-
 ↪  Files/lammps.dir/data/home/wuxingxing/codespace/suzhou/lmpversions/lammps-
 ↪  23-4-opt/src/nep_gpu/utilities/gpu_vector.cu.o
[100%] Linking CXX static library liblammps.a
[100%] Built target lammps
[100%] Building CXX object CMake-
 ↪  Files/lmp.dir/data/home/wuxingxing/codespace/suzhou/lmpversions/lammps-
 ↪  23-4-opt/src/main.cpp.o
```

```
[100%] Linking CXX executable lmp
[100%] Built target lmp
```

2.3.2.4 Load and Use LAMMPS Before using MatPL-2026.3 LAMMPS, load OpenMPI, CUDA, and the LAMMPS environment variables.

```
module load cuda/11.8 openmpi/4.1.6
# Add the lmp executable to PATH
export PATH=/the/path/of/lammpsroot/dir/build:$PATH
```

2.3.3 Build and Install the Closed-Source MatPL LAMMPS 2026.3 Interface

The closed-source MatPL LAMMPS 2026.3 interface separates the LAMMPS interface code from the NEP GPU inference kernels. C++ interface components such as `pair_nep.cpp`, the Kokkos wrapper, and `nep_gpu_loader.h` are compiled into LAMMPS, while the optimized NEP GPU kernels are supplied as the precompiled `libnep_gpu.so` shared library. At runtime, the interface loads the library dynamically with `dlopen` and validates the license file.

You therefore do not compile the NEP GPU inference library. Download the LAMMPS interface source, the precompiled library matching the CUDA version, and a valid license, then compile only the C++ interface code.

2.3.3.1 Download the Closed-Source Interface Files Download the interface source, precompiled libraries, and evaluation license from the MatPL-pro-v2026.6 release.

File	CUDA / Precision
`libnep_gpu_cu118.so`	CUDA 11.8, default mixed precision
`libnep_gpu_cu118_fp64.so`	CUDA 11.8, FP64
`libnep_gpu_cu128.so`	CUDA 12.8, default mixed precision
`libnep_gpu_cu128_fp64.so`	CUDA 12.8, FP64
`summer_holiday.lic or 2026.lic`	License
Source code (zip)	LAMMPS interface source
Source code (tar.gz)	LAMMPS interface source

Select one shared library for the runtime environment:

- A `cu118` library requires the CUDA 11.x runtime (`libcudart.so.11.0`).

- A `cu128` library requires the CUDA 12.x runtime (`libcudart.so.12`).
- Libraries without `_fp64` use the default mixed precision. When using an `_fp64` library, compile the LAMMPS interface with `-DPREC_NEPINFER=ON`.

> **Note:** The CUDA major version of the shared library must match that of the runtime `libcudart`. CUDA 11.x and CUDA 12.x libraries are not interchangeable.

2.3.3.2 Closed-Source Interface Source Tree

After extracting the source archive, the root directory has the following structure:

```
├── .gitignore
├── examples/
├── kknep-patch-cpu.sh
├── kknep-patch.sh
├── KOKKOS/
├── matpl-patch.sh
├── MATPLD3/
├── MATPLDP/
├── MatPLPackages.cmake
├── nep_cpu.cpp
├── nep_cpu.h
├── nep_gpu/
│   ├── cuda-11.8/
│   ├── cuda-12.8/
│   └── 2026.lic # Expires at the end of 2026
│   └── summer_holiday.lic # Expires at the end of August 2026
├── nep_gpu_loader.h
├── pair_nep.cpp
├── pair_nep.h
├── README.md
└── test/
```

The main components are:

- `kknep-patch.sh` backs up and modifies LAMMPS `cmake/CMakeLists.txt`, then copies the NEP CPU interface, `nep_gpu_loader.h`, the Kokkos wrapper, and MatPL heat-flux interface files.
- `nep_gpu_loader.h` loads `libnep_gpu.so` at runtime, so the closed-source library does not need to be linked directly into LAMMPS during compilation.
- `KOKKOS` contains the NEP Kokkos interface, the MatPL heat-flux compute, and the time-averaging fix.
- `examples` and `test` contain usage examples and test scripts, respectively.

2.3.3.3 Build Environment

The following software versions are recommended:

Component	Recommended version	Notes
LAMMPS	`stable_2Aug2023_update4`, `stable_29Aug2024_update4`, or `stable_22Jul2025_update4`	`kknep-patch.sh` supports the 2023–2025 releases
CUDA Toolkit	11.8 or 12.8	Must match the major version of the downloaded library
GCC	8.3 or later	C++17 support is required
OpenMPI	4.1.x	Used for multi-process and multi-node parallelism
CMake	3.18 or later	LAMMPS build system

For example, load build environments for CUDA 11.8 or CUDA 12.8 as follows:

```
# CUDA 11.8, for libnep_gpu_cu118*.so
module --ignore-cache load openmpi/4.1.6 cuda/11.8-share gcc/8.3.1
 ↪  cmake/3.31.6

# CUDA 12.8, for libnep_gpu_cu128*.so
module --ignore-cache load openmpi/4.1.6 cuda/12.8-share gcc/11.2.1
 ↪  cmake/3.31.6
```

> The `module` names above are examples for a cluster environment. Adjust them to match the installed software.

2.3.3.4 Copy the Interface Files into the LAMMPS Source Tree

Enter the root of the extracted source and run:

```
cd /the/path/of/lammps-MatPL-MatPL-pro-v2026.6
bash kknep-patch.sh /the/path/of/lammps
```

The script copies the interface files into the LAMMPS `src` directory and adds the `PKG_NEP_KK` build option to `cmake/CMakeLists.txt`. The message `Patch process completed successfully!` confirms completion.

2.3.3.5 Build LAMMPS with CMake Enter the LAMMPS source directory and create a separate build directory:

```
cd /the/path/of/lammps
mkdir build && cd build

cmake -C ../cmake/presets/basic.cmake \
    -DPKG_MESONT=no \
    -DPKG_JPEG=no \
    -DPKG_KOKKOS=yes \
    -DPKG_NEP_KK=yes \
    -DKokkos_ENABLE_CUDA=yes \
    -DKokkos_ENABLE_OPENMP=yes \
    -DKokkos_ENABLE_CUDA_LAMBDA=yes \
    -DFFT_KOKKOS=CUFFT \
    -DKokkos_ARCH_AMPERE86=ON \
    ../cmake

cmake --build . --parallel 8
```

`-DKokkos_ARCH_AMPERE86=ON` is an example for SM 8.6 GPUs such as the A6000 and RTX 3090. Select the Kokkos architecture for the actual GPU: `-DKokkos_ARCH_AMPERE80=ON` for A100, `-DKokkos_ARCH_ADA89=ON` for RTX 4090, or `-DKokkos_ARCH_HOPPER90=ON` for H100.

When using an `_fp64.so` library, add the following option to the CMake command:

```
-DPREC_NEPINFER=ON \
```

2.3.3.6 Configure the Shared Library and License After compilation, set `NEP_GPU_LIB_PATH` to the full path of the precompiled library and `NEP_LICENSE_PATH` to the license file:

```
# LAMMPS executable
export PATH=/the/path/of/lammps/build:$PATH

# CUDA 11.8 default mixed-precision example
export NEP_GPU_LIB_PATH=/the/path/of/libnep_gpu_cu118.so
export NEP_LICENSE_PATH=/the/path/of/summer_holiday.lic
```

Alternatively, rename the library to `libnep_gpu.so` and add its directory to `LD_LIBRARY_PATH`:

```
export LD_LIBRARY_PATH=/the/path/of/libnep_gpu_dir:$LD_LIBRARY_PATH
```

`NEP_GPU_LIB_PATH` takes precedence over the system library search path, so specifying the `.so` file directly is recommended. Similarly, point `NEP_LICENSE_PATH` directly to the downloaded `summer_holiday.lic`.

> **License:** The `summer_holiday.lic` included in the release is a non-commercial evaluation license for up to 16 GPUs and expires on August 31, 2026. For commercial use or renewal, contact `matpl@pwmat.com`.

2.3.4 Run LAMMPS

2.3.4.1 Run Commands Before running LAMMPS, make sure `NEP_GPU_LIB_PATH` and `NEP_LICENSE_PATH` are set and that the input, structure, and NEP force-field files are ready.

```
# One GPU
mpirun -np 1 --bind-to numa lmp -k on g 1 -sf kk -pk kokkos -in
 ↪  input.in

# Multiple GPUs on one node (four GPUs)
mpirun -np 4 --bind-to numa lmp -k on g 4 -sf kk -pk kokkos -in
 ↪  input.in

# Multiple nodes (two nodes with four GPUs each)
mpirun -np 8 --bind-to numa --map-by ppr:4:node lmp -k on g 4 -sf kk
 ↪  -pk kokkos -in input.in
```

For multi-node execution, set the same `NEP_GPU_LIB_PATH` and `NEP_LICENSE_PATH` on every compute node and ensure that all nodes can access the shared library, license, input files, and force-field files.

2.3.4.2 LAMMPS Input Script The NEP Kokkos interface requires a half neighbor list with Newton communication enabled:

```
package kokkos neigh half comm device
newton on
```

For single-model inference, configure `pair_style` as follows:

```
pair_style    matpl/nep/kk nep.txt
pair_coeff    * * Hf O
```

The element names after `pair_coeff * *` correspond one-to-one with the atom types in the data file and map to the element order in the NEP force-field file.

2.3.4.3 Multi-Model Deviation Calculation In an active-learning workflow, load several NEP force fields simultaneously to output model deviations:

```
pair_style   matpl/nep/kk nep1.txt nep2.txt nep3.txt out_freq 10
 ↪  out_file explr.error
pair_coeff   * * Hf O
```

- Multiple `nep*.txt` files enable model-deviation calculation.
- `out_freq N` outputs the deviation every N steps; the default is 1.
- `out_file name` sets the deviation-output file; the default is `explr.error`.

2.3.4.4 NPT Example The following is a complete NPT input script for an HfO2 NEP force field:

```
package kokkos neigh half comm device
newton on

units           metal
boundary        p p p
atom_style      atomic
neighbor        1.0 bin
neigh_modify    delay 10

read_data       structure.lmp
replicate       10 10 10

mass            1 178.49
mass            2 15.999

pair_style      matpl/nep/kk nep.txt
pair_coeff      * * Hf O

velocity        all create 300.0 12345
fix             1 all npt/kk temp 300 300 0.1 iso 0.0 0.0 0.5

thermo_style    custom step temp pe ke etotal press vol
thermo          10
timestep        0.002

run             1000
```

2.3.4.5 MatPL Heat-Flux Calculation

> **Interface distinction:** The open-source version supports only `matpl/heatflux`; the closed-source version supports the Kokkos GPU heat-flux interface `matpl/heatflux/kk`. The commands below apply only to the closed-source version.

`matpl/heatflux/kk` calculates MatPL heat flux directly on the GPU and does not require the conventional `ke/atom + pe/atom + centroid/stress/atom + heat/flux` post-processing chain:

```
package kokkos neigh half comm device
newton on

pair_style   matpl/nep/kk nep.txt
pair_coeff   * * C

compute      flux all matpl/heatflux/kk
fix          fluxout all matpl/heatflux/ave/kk 10 100 1000 flux file
 ↪  compute_HeatFlux.out
```

`compute matpl/heatflux/kk` outputs a global vector with six components:

1. `Jx`: total heat flux in the x direction.
2. `Jy`: total heat flux in the y direction.
3. `Jz`: total heat flux in the z direction.
4. `Jconv,x`: convective heat flux in the x direction.
5. `Jconv,y`: convective heat flux in the y direction.
6. `Jconv,z`: convective heat flux in the z direction.

The virial contribution to heat flux is therefore the total heat flux minus the convective heat flux: `(1-4, 2-5, 3-6)`. `fix matpl/heatflux/ave/kk` time-averages the heat-flux vector and writes the result to `compute_HeatFlux.out`.

2.3.4.6 More Examples

- H2O multi-node, multi-GPU example
- HfO2 multi-node, multi-GPU example
- Graphene heat-flux calculation and validation example

2.3.4.7 Troubleshooting

Error or symptom	Cause	Solution
`Failed to load NEP GPU library`	The shared library cannot be found	Check `NEP_GPU_LIB_PATH` or add the library directory to `LD_LIBRARY_PATH`

Error or symptom	Cause	Solution
`Symbol not found: nep_*`	The interface and library versions do not match	Use the source and `libnep_gpu*.so` from the same release
`NEP GPU license check failed` or `License file not found`	The license is missing, incorrectly addressed, or expired	Check `NEP_LICENSE_PATH`, file permissions, and the expiration date
A `libcudart` error occurs while loading the `.so`	CUDA major versions do not match	Use a `cu118` library with CUDA 11.x and a `cu128` library with CUDA 12.x
The GPU does not match the license or the GPU count exceeds the limit	The current hardware is outside the licensed scope	Inspect GPUs with `nvidia-smi --query-gpu=uuid`; restrict `CUDA_VISIBLE_DEVICES` or update the license if necessary

2.4 Common Installation Errors

2.4.1 Build Environment

Most installation failures are caused by incompatible build-environment versions or missing environment variables. First, verify that the following compilers and tools are installed correctly and have compatible versions.

We recommend `intel2020`, `cuda/11.8`, `CMake >= 3.21`, and `GCC 8.x`. MatPL uses `PyTorch >= 2.0` and requires `CUDA 11.8` or later.

The `intel/2020` toolchain provides the `ifort` and `icc` compilers (`19.1.3`), `MPI 2019`, and `MKL 2020`. If you load these components separately, use versions no older than these.

You can check the environment with `src/check/check_env.sh` in the source root. A valid environment produces output similar to the following.

```
1. CUDA version is 11.8.
2. nvcc command exists.
3. ifort version is no less than 19.1, current version is 19.1.
```

```
4. MKL library is installed.
5. GCC version is not 8.x, current version is 8.
6. PyTorch is installed.
7. PyTorch version is 2.0 or above, current version is 2.2.
```

2.4.2 Issue: `cuda_runtime.h` Header Not Found

If `cuda_runtime.h` cannot be found during compilation, replace line 24 of `src/MAKE/Makefile.mpi` with the path to your CUDA installation, such as `/the/path/cuda/cuda-11.8`. The header is located in the `include` subdirectory.

```
CUDA_HOME = $(CUDADIR)
Replace with CUDA_HOME = /the/path/cuda/cuda-11.8
```

2.4.3 Issue: NeighConst.so Compilation Error

2.4.3.1 Error Description The following error occurs while compiling the Fortran code:

```
ifort -O3 least_squares.f90 counts_atom.f90 scan_title.f90 transform_to_upp
      find_neighbore00.f90 find_neighbore.f90 find_feature_deepMD2.f90 \
         gen_deepMD2_feature.f90 \
    -o gen_deepMD2_feature.x -mkl
python3 -m numpy.f2py -c -m NeighConst --fcompiler=intelem --
compiler=intelem -L/share/app/intel2020ucompilers_and_libraries_2020.4.304
lmkl_rt NeighConst.f90
Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
 File "/data/home/wuxingxing/anaconda3/envs/pwmlff-2024.5/lib/python3.11/
packages/numpy/f2py/__in__.py", line 5, in <module>
    main()
 File "/data/home/wuxingxing/anaconda3/envs/pwmlff-2024.5/lib/python3.11/
packages/numpy/f2py/f22e.py", line 766, in main
    run_compile()
 File "/data/home/wuxingxing/anaconda3/envs/pwmlff-2024.5/lib/python3.11/
packages/numpy/f2py/f22e.py", line 594, in run_compile
    build_backend = f2py_build_generator(backend_key)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
 File "/data/home/wuxingxing/anaconda3/envs/pwmlff-2024.5/lib/python3.11/
packages/numpy/f2py/_bkends/__init__.py", line 6, in f2py_build_generator
    from ._distutils import DistutilsBackend
```

```
  File "/data/home/wuxingxing/anaconda3/envs/pwmlff-2024.5/lib/python3.11/
packages/numpy/f2py/_bkends/_distutils.py", line 3, in <module>
    from numpy.distutils.core import setup, Extension
  File "/data/home/wuxingxing/anaconda3/envs/pwmlff-2024.5/lib/python3.11/
packages/numpy/distuti/core.py", line 24, in <module>
   from numpy.distutils.command import config, config_compiler, \
  File "/data/home/wuxingxing/anaconda3/envs/pwmlff-2024.5/lib/python3.11/
packages/numpy/distuti/command/config.py", line 19, in <module>
   from numpy.distutils.mingw32ccompiler import generate_manifest
  File "/data/home/wuxingxing/anaconda3/envs/pwmlff-2024.5/lib/python3.11/
packages/numpy/distuti/mingw32ccompiler.py", line 27, in <module>
   from distutils.msvccompiler import get_build_version as get_build_msvc_v
ModuleNotFoundError: No module named 'distutils.msvccompiler'
make: *** [NeighConst.so] Error 1
make: Leaving directory `/data/home/wuxingxing/codespace/PWMLFF_gpu/src/pr
make: Entering directory `/data/home/wuxingxing/codespace/PWMLFF_gpu/src/p
```

2.4.3.2 Cause This error is caused by an incompatible `setuptools` version, usually one that is too recent. Downgrade `setuptools` as follows:

```
# Uninstall setuptools
$ pip uninstall setuptools
# Clear the local cache
$ pip cache purge
# Reinstall setuptools
$ pip install setuptools==68.0.0
# In our tests, version 68.0.0 or earlier works
```

2.4.4 Issue: Unknown `CUDA Architecture Name 9.0a` When Building the Offline Package

2.4.4.1 1. Symptom When compiling custom operators in the MatPL offline package, the PyTorch CMake configuration fails during `cmake ..` with the following error:

```
-- PyTorch built with CUDA support: TRUE
-- CUDA toolkit found: /share/app/cuda/cuda-12.4/bin/nvcc
-- Enabling GPU compilation: PyTorch has CUDA support AND CUDA
 ↪  toolkit is available
-- Automatic GPU detection failed. Building for common architectures.
-- Autodetected CUDA architecture(s): 3.5;5.0;8.0;8.6;8.9;9.0;9.0a
```

```
CMake Error at
 ↪  .../torch/share/cmake/Caffe2/Modules_CUDA_fix/upstream/FindCUDA/select_compu
 ↪  (message):
  Unknown CUDA Architecture Name 9.0a in CUDA_SELECT_NVCC_ARCH_FLAGS
```

This issue commonly occurs with the following environment:

```
module load cuda/12.4-share openmpi/4.1.6 cmake/3.31.6
source /opt/rh/devtoolset-8/enable
source /path/to/MatPL-2026.3/matpl-2026.3/bin/activate
```

Here, the PyTorch version is `2.2` and the CUDA Toolkit version is `12.4`.

2.4.4.2 2. Cause This is not a compilation error in the MatPL operator source. It occurs because the CMake CUDA architecture-selection script bundled with PyTorch fails during configuration.

When PyTorch cannot automatically identify the current GPU architecture, it falls back to a list of common CUDA architectures. Under CUDA 12.4, that list may include:

```
9.0a
```

However, the `select_compute_arch.cmake` script bundled with PyTorch 2.2 cannot recognize the letter-suffixed architecture `9.0a`, so configuration fails at `find_package(Torch REQUIRED)`.

2.4.4.3 3. Solution Before running `cmake ..`, explicitly specify the GPU architectures to compile and bypass PyTorch's automatic detection.

For an NVIDIA RTX 4090, use:

```
export TORCH_CUDA_ARCH_LIST="8.9"
cmake ..
```

To support common GPUs such as the A100, RTX 3090, and RTX 4090 in the same offline package, use:

```
export TORCH_CUDA_ARCH_LIST="8.0;8.6;8.9"
cmake ..
```

To support the H100, use:

```
export TORCH_CUDA_ARCH_LIST="9.0"
cmake ..
```

Do not set the value to `9.0a`.

You can also specify it directly in the CMake command:

```
cmake .. -DTORCH_CUDA_ARCH_LIST="8.0;8.6;8.9"
```

2.4.5 Issue: NVCC Reports `fatal error: math.h: No such file or directory`

2.4.5.1 1. Symptom While `make` is compiling `.cu` files, it reports a low-level error indicating that a standard-library header cannot be found:

```
/usr/include/c++/11/cmath:45:15: fatal error: math.h: No such file or
 ↪  directory
   45 | #include_next <math.h>
      |               ^~~~~~~~
```

2.4.5.2 2. Cause

- **Root cause (path contamination):** While detecting or configuring the environment, CMake passes the system `/usr/include` directory explicitly to `nvcc`, through either `-isystem /usr/include` or `-I`.
- **Broken `#include_next` behavior:** When processing `cmath`, GCC uses `#include_next <math.h>`, meaning "skip the current directory and search later system paths for the next `math.h`." Once `/usr/include` is supplied explicitly, GCC incorrectly treats that directory as already traversed and cannot find the actual C `math.h` in subsequent paths.
- **Trigger A (custom operators):** An older system CUDA 11.5 installation is mixed with GCC 11, exposing a bug in which the older NVCC cannot interpret the GCC 11 option set.
- **Trigger B (LAMMPS build):** A custom package calls `find_package(CUDAToolkit)`, and the returned `CUDAToolkit_INCLUDE_DIRS` resolves to `/usr/include`, contaminating the include paths of global targets.

2.4.5.3 3. Solution Edit **`CMakeLists.txt`** in the LAMMPS root and add the following workaround immediately after `project(lammps CXX CUDA)`:

```
project(lammps CXX CUDA)

# ================ Work around the missing math.h issue
 ↪  ================
list(APPEND CMAKE_CUDA_IMPLICIT_INCLUDE_DIRECTORIES "/usr/include")
string(APPEND CMAKE_CUDA_FLAGS " -Xcompiler -idirafter,/usr/include")
# ============================================================
```

If multiple CUDA compilers are present, explicitly specify the CUDA compiler and Toolkit root.

```
cmake -C ../cmake/presets/basic.cmake \
```

```
    -
     ↪ DCMAKE_CUDA_COMPILER=/opt/nvidia/hpc_sdk/Linux_x86_64/23.9/compilers/bin/
     ↪ \
    -DCUDAToolkit_ROOT=/opt/nvidia/hpc_sdk/Linux_x86_64/23.9/cuda/12.2
     ↪ \
    -DPKG_KOKKOS=yes \
    -DPKG_NEP_KK=yes \
    -DKokkos_ENABLE_CUDA=yes \
    -DKokkos_ARCH_VOLTA70=ON \
    ../cmake
make -j4

# VOLTA70 targets V100 GPUs; for other GPUs, see
 ↪ https://docs.lammps.org/Build_extras.html#available-architecture-
 ↪ settings
```

2.5 Common Runtime Errors

2.5.1 Check Environment Variables

Runtime errors caused by missing or incorrectly loaded environment variables usually appear as a missing `MatPL` command or missing `***.so` shared libraries. Verify that all of the following environments have been loaded.

```
# Python environment: verify that it has been activated
source /the/path/etc/profile.d/conda.sh
conda activate matpl-2025.3

# Intel and CUDA toolchains
module load intel/2020 cuda/11.8

# MatPL environment variables
source /the/path/MatPL-2025.3/env.sh
```

2.5.2 Issue: CUDA driver version is insufficient for CUDA runtime version

2.5.2.1 1. Symptom When running GPU features or custom CUDA operators in MatPL, the program reports the following error:

```
CUDA Error:
    File:
 ↪ /data/home/wuxingxing/pack/pack-2026.3/gpu-install/MatPL-
 ↪ 2026.3/MatPL-2026.3/src/op/kernel/calculateNepNeighbor.cu
```

```
    Line:       331
    Error code: 35
    Error text: CUDA driver version is insufficient for CUDA runtime
↪   version
```

`Error code: 35` indicates that the NVIDIA driver installed on the machine is too old to support the CUDA Runtime used by the program.

2.5.2.2 2. Cause Running a CUDA program requires both of the following conditions:

1. The CUDA Runtime loaded in the current environment is compatible with the CUDA version used to compile and package the program.
2. The NVIDIA driver is new enough to support that CUDA Runtime.

If the driver is too old, CUDA kernel startup fails even when the Conda environment, CUDA module, and MatPL environment are loaded correctly.

Use the following command to inspect the current driver:

```
nvidia-smi
```

For example, the following older driver supports CUDA only up to version 11.6:

```
NVIDIA-SMI 510.39.01    Driver Version: 510.39.01    CUDA Version:
↪   11.6
```

If the MatPL offline package or build environment uses CUDA 12.4, this driver is insufficient and triggers the error above.

2.5.2.3 3. Solution Upgrade the NVIDIA driver to a version that supports the CUDA Runtime used by the current MatPL environment.

After upgrading, the driver information may look like this:

```
NVIDIA-SMI 550.100      Driver Version: 550.100      CUDA Version: 12.4
```

After the upgrade, log in to the node again or restart the relevant services, then run:

```
nvidia-smi
```

Confirm that `Driver Version` and CUDA `Version` now show compatible versions before rerunning MatPL.

Switching Conda environments or reloading the CUDA module alone cannot resolve this issue. CUDA `Version` indicates the compatibility ceiling imposed by the NVIDIA driver. If the driver is too old, a system administrator must upgrade it.

2.5.3 Shared-Library Loading Error: MKL

```
    exec(code, run_globals)
  File "/the/path/MatPL-2025.3/main.py", line 6, in <module>
    from src.user.dp_work import dp_train, dp_test
  File "/the/path/MatPL-2025.3/src/user/dp_work.py", line 6, in
  ↪  <module>
    from src.PWMLFF.dp_network import dp_network
  File "/the/path/MatPL-2025.3/src/PWMLFF/dp_network.py", line 42, in
  ↪  <module>
    import src.pre_data.dp_mlff as dp_mlff
  File "/the/path/MatPL-2025.3/src/pre_data/dp_mlff.py", line 11, in
  ↪  <module>
    from src.lib.NeighConst import neighconst
ImportError: libmkl_rt.so: cannot open shared object file: No such
 ↪  file or directory
```

Solution: The Intel Math Kernel Library (MKL) has not been loaded. The `intel/2020` module—typically provided by Intel Parallel Studio XE 2020 or the Intel oneAPI Toolkits 2020—usually includes Intel MKL. Loading this module makes MKL available in the build and run-time environments.

```
module load intel/2020
```

2.5.4 Common LAMMPS Interface Runtime Errors

2.5.4.1 Check Environment Variables Runtime errors caused by missing or incorrectly loaded environment variables usually appear as a missing `lmp_mpi` command or missing `***.so` shared libraries. Verify that the following environments have been loaded.

```
# 1. Required for the mpirun command
module load intel/2020

# 2. Load the LAMMPS environment variables
source /the/path/of/lammps/env.sh

# 3. Run LAMMPS
# --bind-to numa binds processes to available cores on each socket,
 ↪  avoiding contention on default cores when multiple jobs run
mpirun -np 4 --bind-to numa lmp_mpi -in in.lammps
```

2.5.4.2 LAMMPS NEP Model After an NEP model has run through the GPU interface for some time, the following error may occur:

```
    ......
    97000   1293.8659     -70999.672      35.957737     -70963.715
 ↪  13.66254       13.66254       12.50455       2334.1619
    98000   1191.7602     -71009.412      33.120127     -70976.292
 ↪  13.577541      13.577541      12.426755      2290.8676
    99000   1219.1286     -71013.893      33.880718     -70980.012
 ↪  13.488421      13.48842       12.345188      2246.0524
CUDA Error:
    File:       utilities/gpu_vector.cu
    Line:       117
    Error code: 700
    Error text: an illegal memory access was encountered

=====================================================================================
=   BAD TERMINATION OF ONE OF YOUR APPLICATION PROCESSES
=   RANK 0 PID 5490 RUNNING AT gn59
=   KILLED BY SIGNAL: 9 (Killed)
=====================================================================================

=====================================================================================
=   BAD TERMINATION OF ONE OF YOUR APPLICATION PROCESSES
=   RANK 1 PID 5491 RUNNING AT gn59
=   KILLED BY SIGNAL: 9 (Killed)
=====================================================================================

=====================================================================================
=   BAD TERMINATION OF ONE OF YOUR APPLICATION PROCESSES
=   RANK 2 PID 5492 RUNNING AT gn59
=   KILLED BY SIGNAL: 9 (Killed)
=====================================================================================
```

`Solution`: This error usually occurs after the MD simulation has run for some time and the force-field fit becomes less accurate. The neighbor lists of some atoms then grow beyond the configured maximum. Increase the maximum number of neighbors as follows. In the NEP force-field file—for example, `nep_to_lmps.txt` shown below:

```
nep4   2 O Si
cutoff 6.0 5.0
n_max  4 4
basis_size 12 12
l_max  4 2 1
......
```

Change the `cutoff` line to

```
cutoff 6.0 5.0 500 400
```

Here, `500` is the maximum neighbor count for the two-body (`radial_cutoff`) cutoff, and `400` is the maximum neighbor count for the many-body (`angular_cutoff`) term.

If this error occurs, first check whether the MD trajectory is physically reasonable and whether the simulation itself has become unstable.

3 MatPL Commands

You can invoke MatPL with `matpl`, `MatPL`, `MATPL`, or `PWMLFF`. `PWMLFF` was the command used before MatPL-2025.3 and remains compatible with version 2026.3.

MatPL provides the `train` command for training, the `test` command for inference and evaluation, and several `model-specific utility commands`. Run `matpl -h` to list all supported commands.

```
MatPL -h
or MatPL --help
```

3.1 train

The `train` command trains a MatPL model and requires a prepared JSON training configuration.

```
MatPL train input.json
```

- For NEP training, see NEP training.
- For DP training, see DP training.
- For NN training, see NN training.
- For LINEAR training, see LINEAR training.

train output directory

After force-field training, the following files and directories are generated:

```
├── model_record
│   ├── epoch_train.dat
│   ├── epoch_valid.dat
│   ├── nep_model.ckpt
│   └── nep5.txt
├── std_input.json
├── train.json
└── forcefield/
    └── forcefield.ff
```

- `std_input.json` contains all settings used for training, including user-defined and default parameters.
- `model_record/nep_model.ckpt` is the force-field checkpoint saved after the latest epoch. `.ckpt` is a PyTorch-readable format. The corresponding files are `dp_model.ckpt` for DP and `nn_model.ckpt` for NN.
- `model_record/nep5.txt` is the text-format force field extracted from `nep_model.ckpt` for MD in LAMMPS or GPUMD. Other model types do not generate this file.

- `model_record/epoch_train.dat` summarizes the loss on `train_data` for each epoch, as shown below. From left to right, the columns are: epoch; total loss; L2 loss, omitted when L2 regularization is disabled (`lambda_2` for ADAM and `po_weight` for LKF); atomic-energy RMSE (eV/atom); force RMSE (eV/Å); atomic-virial RMSE (eV/atom), omitted when `train_virial` is disabled; learning rate; and epoch time in seconds.

```
# epoch              loss           Loss_l2   RMSE_Etot(eV/atom)
 ↪  RMSE_F(eV/Å)   RMSE_virial(eV/atom)           real_lr
 ↪  time(s)
    1    4.7907987747e+04  1.3987758802e-01     2.2508174106e+00
 ↪  7.7088011034e-01       5.4796850561e+00  1.0000000000e-03
 ↪  3.5540
    ......
```

- `model_record/epoch_valid.dat` summarizes the validation loss on `valid_data` after each epoch. It is not generated when no validation set is configured. From left to right, the columns are: epoch; total loss; atomic-energy RMSE (eV/atom); force RMSE (eV/Å); atomic-virial RMSE (eV/atom), omitted when `train_virial` is disabled; learning rate; and epoch time in seconds.

```
# epoch              loss    RMSE_Etot(eV/atom)         RMSE_F(eV/Å)
 ↪  RMSE_virial(eV/atom)
    1    2.9945036197e+04     1.8128180972e+00     6.5953191220e-01
 ↪  5.2145886493e+00
```

- The `forcefield` directory contains text-format NN or Linear force fields for the Fortran LAMMPS interface.

3.2 test

The `test` command evaluates a MatPL model and requires a prepared JSON inference configuration. When successful, it outputs predicted energies and forces for the test data.

```
MatPL test input.json
```

- For NEP testing, see NEP testing.
- For DP testing, see DP testing.
- For NN testing, see NN testing.
- For LINEAR testing, see LINEAR testing.

test output directory

After testing, a `test_result` directory is created in the current directory to store the evaluated structures and results:

```
test_result/
│    ├──image_atom_nums.txt
│    ├── dft_total_energy.txt
│    ├── dft_force.txt
│    ├── dft_virial.txt
│    ├── dft_atomic_energy.txt
│    ├── inference_total_energy.txt
│    ├── inference_force.txt
│    ├── inference_virial.txt
│    ├── inference_atomic_energy.txt
│    ├── inference_summary.txt
│    ├── Energy.png
│    └── Force.png
└── std_input.json
```

- `image_atom_nums.txt` stores the atom count of each structure in the test set.
- `dft_total_energy.txt` stores the energy label for each structure.
- `dft_force.txt` stores per-atom force labels. Each line contains the x, y, and z force components of one atom.
- `dft_virial.txt` stores one virial label per structure. If virial information is unavailable, the line contains nine `-e6` placeholders.
- `dft_atomic_energy.txt` stores per-atom energy labels, a PWmat-specific label, with one structure per line.
- `inference_total_energy.txt` stores predicted energies in the same structure order as `dft_total_energy.txt`.
- `inference_virial.txt` stores one predicted virial per structure, aligned with `dft_virial.txt`.
- `inference_atomic_energy.txt` stores predicted per-atom energies, one structure per line, aligned with `dft_atomic_energy.txt`.
- `Energy.png` compares labeled and predicted energies.
- `Force.png` compares labeled and predicted forces.
- `Virial.png` compares labeled and predicted virials. It is omitted when the labels contain no virial data.
- `inference_summary.txt` contains the evaluation summary, as shown below.

```
For 1140 images:
Average RMSE of Etot per atom: 0.029401988821789057
Average RMSE of Force: 0.045971754863441294
Average RMSE of Virial per atom: None
```

```
More details can be found under the file directory:
/the/path/test/test_result
```

3.3 ASE Interface

NEP and DP models support ASE operations on CPUs or GPUs: - NEP ASE example - DP ASE example

3.4 Other Utility Commands

MatPL provides different utility commands for each model type.

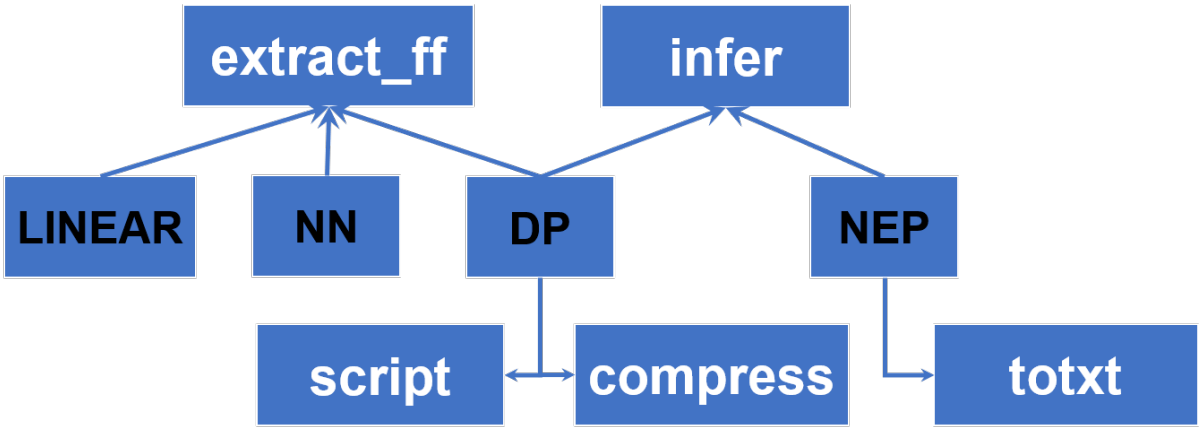


Figure 1: MatPL utility commands

3.4.1 extract_ff

This command extracts an NN force-field checkpoint to text format for use with the Fortran LAMMPS interface.

```
# Extract the NN force-field model
MatPL extract_ff nn_model.ckpt
```

For usage, see: - Extract an NN force field

3.4.2 infer

This command uses an NEP or DP model to predict the energy and forces of a single structure.

```
# NEP inference on a PWmat atom.config structure
MatPL infer nep_to_lmps.txt atom.config pwmat/config
MatPL infer nep_modek.ckpt atom.config pwmat/config
# DP inference on a LAMMPS dump structure
MatPL infer dp_model.ckpt 0.lammpstrj lammps/dump Hf O
```

For usage, see: - Single-structure inference with an NEP force field - Single-structure inference with a DP force field

3.4.3 totxt

This NEP-specific command converts an NEP checkpoint to a text-format force field for LAMMPS or GPUMD.

```
MatPL totxt nep_model.ckpt
```

For usage, see: - Convert an NEP force field to LAMMPS or GPUMD format

3.4.4 compress

This DP-specific command accelerates inference by fitting the DP embedding network with polynomials. It provides a significant speedup when the training set contains many atom types. The complete command is:

```
MatPL compress dp_model.ckpt -d 0.01 -o 3 -s cmp_dp_model
```

- `compress` is the compression command.
- `dp_model.ckpt` is the required model file to compress.
- `-d` sets the S_{ij} grid spacing; the default is `0.01`.
- `-o` sets the compression order: 3 for third order or 5 for fifth order. The default is 3.
- `-s` sets the compressed model name. The default is `cmp_dp_model`.

After compression, `cmp_dp_model.ckpt` is created in the current directory. For usage, see: - Polynomial compression of a DP force field

3.4.5 script

This DP-specific command converts a DP checkpoint to LibTorch format for use in LAMMPS simulations.

```
MatPL script dp_model.ckpt
# Creates jit_dp.pt in the current directory
# Convert a compressed force-field file
MatPL script cmp_dp_model.ckpt
# Creates jit_cmp_dp.pt in the current directory
```

For usage, see: - Convert a DP force field to LibTorch format

3.5 Using Force Fields in LAMMPS

MatPL provides LAMMPS force-field interfaces. For installation, see `Online Installation` or the offline installation guide.

Run LAMMPS as follows:

```
# Load the LAMMPS env.sh file from the LAMMPS source root
source /the/path/of/lammps/env.sh

# Run LAMMPS
# NEP supports Kokkos acceleration with pair style matpl/nep/kk
# Multiple GPUs on one node (four GPUs here)
mpirun -np 4 --bind-to numa lmp -k on g 4 -sf kk -pk kokkos -in
 ↪  kkin.lmp

# Multiple nodes and GPUs (two nodes with four GPUs each)
mpirun -np 8 --map-by ppr:4:node lmp -k on g 4 -sf kk -pk kokkos -in
 ↪  kkin.lmp

# Use the following form for the CPU version of matpl/nep or for
 ↪  matpl/dp
mpirun -np N lmp -in in.lammps
```

Specify the force-field path in the LAMMPS input file as shown below.

For the Kokkos-accelerated LAMMPS NEP interface:

```
# LAMMPS 2024 requires neigh half; LAMMPS 2023 accepts either half or
 ↪  full
package kokkos neigh half comm device
newton on

pair_style    matpl/nep/kk    /path/to/force-field
pair_coeff    * *      O Hf
```

Here: - LAMMPS 2024 requires `neigh half`; LAMMPS 2023 accepts either `half` or `full`.

- `pair_style` sets the force-field path. `matpl/nep/kk` selects MatPL's Kokkos GPU acceleration for NEP, while `matpl/nep` uses only the CPU. For a DP model, use `matpl/dp`; it automatically uses a GPU when available and otherwise runs on the CPU.

The interface also supports model-deviation output from multiple models, commonly used in active learning. The first model drives the MD simulation, while the others participate in the deviation calculation. Configure `pair_style` as follows:

```
# LAMMPS 2024 requires neigh half; LAMMPS 2023 accepts either
↪ half or full
package kokkos neigh half comm device
newton on

pair_style   matpl/nep/kk   0_nep.txt 1_nep.txt 2_nep.txt
↪ 3_nep.txt  out_freq DUMP_FREQ_VALUE out_file model_devi.out
```

- LAMMPS 2024 requires `neigh half`; LAMMPS 2023 accepts either `half` or `full`.
- `pair_coeff` maps atom types in the simulated structure to elements. For example, if type 1 is O and type 2 is Hf, use `pair_coeff * * 8 72`. Atomic numbers or element symbols are both accepted, provided their order matches the input structure.

For LAMMPS configuration examples for DP and NEP force fields, see: - NEP LAMMPS MD - DP LAMMPS MD

The LAMMPS `pair_style` configuration is slightly different for NN and LINEAR force fields:

```
pair_style   matpl
pair_coeff   * * 3 1 forcefield.ff 29
```

- `pair_style` selects the MatPL force field.
- `pair_coeff` specifies the force-field file and atom type. Here, 3 selects a Neural Network force field; use 1 for a Linear force field. The second 1 means that one force-field file is read, `forcefield.ff` is generated by MatPL, and 29 is the atomic number of Cu.
- NN lammps MD
- LINEAR lammps MD

The MatPL-2026.3 LAMMPS interface is built with CMake and produces an executable named `lmp` by default. The MatPL-2025.3 interface is built with Make and produces `lmp_mpi` by default.

4 MatPL Parameters

This section describes user-configurable parameters for all models. `Basic parameters` must be supplied, while `advanced parameters` have defaults that can be overridden in JSON. A relative path is resolved from the current working directory; an absolute path begins at the filesystem root.

4.1 Basic Parameters

4.1.1 model_type

Selects the model type: `LINEAR`, `NN`, `DP`, or `NEP`.

4.1.2 atom_type

Sets the elements in any chosen order using atomic numbers or symbols. Examples: `[29]` or `["Cu"]` for copper, and `[1,6]` or `["H","C"]` for CH4.

4.1.3 train_data

Specifies training-data paths, either relative or absolute. - DP and NEP support `extxyz`, `pwmlff/npy`, `deepmd/npy`, `deepmd/raw`, `pwmat/movement`, `vasp/outcar`, and `cp2k/md`. - LINEAR and NN support only `pwmat/movement`.

4.1.4 valid_data

Specifies validation-data paths, either relative or absolute. - DP and NEP support `extxyz`, `pwmlff/npy`, `deepmd/npy`, `deepmd/raw`, `pwmat/movement`, `vasp/outcar`, and `cp2k/md`. - LINEAR and NN support only `pwmat/movement`.

4.1.5 test_data

Specifies test-data paths for the `test` command, either relative or absolute. - DP and NEP support `extxyz`, `pwmlff/npy`, `deepmd/npy`, `deepmd/raw`, `pwmat/movement`, `vasp/outcar`, and `cp2k/md`. - LINEAR and NN support only `pwmat/movement`.

4.1.6 format

Sets the format of `train_data`, `valid_data`, and `test_data`. Supported values include `extxyz`, `pwmlff/npy`, `deepmd/npy`, `deepmd/raw`, and direct trajectory formats `pwmat/movement`, `vasp/outcar`, and `cp2k/md`. The default is `pwmat/movement`. See pwdata. :::info All input datasets must use the same format. :::

4.1.7 model_load_file

- For fine-tuning or continued training, specifies an initial `.ckpt` model.
- For `test`, specifies the model path, either relative or absolute.

4.1.8 nep_txt_file

- For fine-tuning or continued training, specifies a `.txt` model such as GPUMD `nep4.txt` or `nep5.txt`.
- The text file may be a universal force field covering 89 elements. MatPL extracts parameters for the elements in `atom_type`, enabling fine-tuning to a smaller model. This can be combined with `fix_cij`, `fix_hiddenlayer`, and `fix_outlayer` under `model->fitting_net`.

4.1.9 recover_train

Whether to resume an interrupted training task. The default is `true`.

4.1.10 reserve_work_dir

For LINEAR and NN, whether to retain `work_dir` after execution. The default is `false`.

4.1.11 save_step

Sets the model-save interval in iterations. The default is None, so models are saved only after each epoch.

4.1.12 max_save_num

With `save_step`, sets the maximum number of recent models retained. The default is `10`.

4.2 NEP Model Hyperparameters

The complete NEP model configuration is:

```
"model": {
    "descriptor": {
        "cutoff": [6.0,6.0],
        "n_max": [4,4],
        "basis_size": [12,12],
        "l_max": [4,2,1],
        "zbl": 2.0
    },
    "fitting_net": {
        "network_size": 40,
        "fix_cij":false,
        "fix_hiddenlayer":false,
        "fix_outlayer":false
    }
}
```

4.2.1 cutoff

Sets radial and angular cutoff radii. The default is `[8.0, 4.0]`.

4.2.2 n_max

Sets `n_max` for radial and angular descriptors. Each value must be between 0 and 19. The default is `[4,4]`.

4.2.3 basis_size

Sets the radial and angular basis sizes. Each value must be between 0 and 19. The default is `[8,8]`.

4.2.4 l_max

Sets angular expansion orders and enables four- and five-body descriptors. The default `[4,2,1]` contains the three-, four-, and five-body orders. Use `[4,0,0]` for three-body only or `[4,2,0]` for three- and four-body descriptors.

The numbers of two-, three-, four-, and five-body descriptors are `n_max[0]+1`, `(n_max[1]+1)*l_max[0]`, `n_max[1]+1`, and `n_max[1]+1`, respectively.

4.2.5 network_size

Sets the number of neurons in NEP' s single hidden layer. The default is `40`.

4.2.6 zbl

Sets the outer cutoff of the Ziegler–Biersack–Littmark (ZBL) potential (`DOI: 10.1007/978-1-4615-8103-1_3`); the inner cutoff is fixed at half the outer cutoff. It is disabled by default. A value in $1.0 \leq zbl \leq 3.0$ is recommended.

4.2.7 use_typewise_cutoff_zbl

Enables type-wise ZBL cutoffs. The outer cutoff for a pair is `min(zbl, use_typewise_cutoff_zbl × sum of covalent radii)`, and the inner cutoff is fixed at 0.0. Disabled by default; introduced in `MatPL-2026.3 Update 1`.

4.2.8 fix_cij

Fine-tuning option that freezes NEP two- and three-body feature coefficients. The default is `false`.

4.2.9 fix_hiddenlayer

Fine-tuning option that freezes NEP hidden-layer parameters `W0` and `B0`. The default is `false`.

4.2.10 fix_outlayer

Fine-tuning option that freezes NEP output-layer parameters `W1` and `B1`. The default is `false`.

4.2.11 Interpreting NEP.txt

A standard NEP `.txt` header is shown below.

```
nep5   2 O Hf           # Number of element types, followed by their
 ↪  symbols
zbl 1 2                 # Present only when ZBL is enabled during
 ↪  training
```

```
cutoff 6.0 6.0 108 108  # Two-body cutoff, many-body cutoff, and
 ↪  their maximum neighbor counts
n_max  4 4              # Two-body and many-body n_max
basis_size 12 12        # Two-body and many-body basis_size
l_max  4 2 1            # Three-, four-, and five-body l_max values
ANN    40 0             # 40 hidden neurons; 0 is a placeholder
```

The remaining lines contain four blocks: `network parameters`, `two-body coefficients`, `three-body coefficients`, and `normalization values`.

- Block 1 contains network parameters for $E_i = \tanh(q_n W0 + B0)W1 + b1$. Its size is `number of elements` × (`number of features` × `ANN[0]` + `ANN[0]` + `ANN[0]`) + `number of elements`. For each element in header order, it stores `W0`, `B0`, and `W1`, followed by one `b1` per element. A GPUMD-trained NEP4 file stores only one `b1`, the mean across elements. `W0` is flattened row-major from `[ANN hidden units, number of features]`.
- Block 2 contains two-body coefficients: `number of elements`² × (`n_max[0]+1`) × (`basis_size[0]+1`).
- Block 3 contains three-body coefficients: `number of elements`² × (`n_max[1]+1`) × (`basis_size[1]+1`).

  In either coefficient block, the matrix order is `[I,J,N,K]`: `I` is the central element, `J` is the neighbor element, both following header order; `N=n_max+1`; and `K=basis_size+1`.
- Block 4 contains one normalization value per feature, ordered by two-, three-, four-, and five-body terms. Their counts are `n_max[0]+1`, `(n_max[1]+1)*l_max[0]`, `n_max[1]+1`, and `n_max[1]+1`.

4.3 DP Model Hyperparameters

The complete DP model configuration is:

```
    "type_embedding":false,
    "model": {
        "type_embedding":{
            "physical_property":["atomic_number", "atom_mass",
             ↪  "atom_radius", "molar_vol", "melting_point",
             ↪  "boiling_point", "electron_affin", "pauling"]
        },
        "descriptor": {
            "Rmax": 6.0,
            "Rmin": 0.5,
```

```
            "M2": 16,
            "network_size": [25,25,25]
        },
        "fitting_net": {
            "network_size": [50,50,50,1]
        }
    }
```

4.3.1 type_embedding

Configures type embedding for DP training. You may also set `"type_embedding":true` alongside `model`, which uses `["atomic_number","atom_radius","atom_mass","electron_` The default is `false`.

4.3.1.1 physical_property Selects physical properties for DP type embedding. Eight properties are available:

```
- `atomic_number`: atomic number
- `atom_mass`: atomic mass
- `atom_radius`: atomic radius
- `molar_vol`: molar volume
- `melting_point`: melting point
- `boiling_point`: boiling point
- `electron_affin`: electron affinity
- `pauling`: Pauling electronegativity
```

```
The default `physical_property` is `["atomic_number","atom_radius","atom_ma
```

4.3.2 Rmax

Maximum cutoff radius of the DP smoothing function. The default is 6.0Å.

4.3.3 Rmin

Minimum cutoff radius of the DP smoothing function. The default is 0.5Å.

4.3.4 M2

Sets the DP embedding-network output dimension and therefore the fitting-network input dimension. In the example, these are `25 × 16` and `25 × 16 = 400`. The default is `16`.

4.3.5 network_size

Sets the embedding- and fitting-network structures. Defaults are `[25,25,25]` and `[50,50,50,1]`.

Embedding network: Input -> hidden layer 1 (25 neurons) -> hidden layer 2 (25 neurons) -> output layer (25 neurons)

Fitting network: Input (`M2 × 25`) -> three hidden layers (50 neurons each) -> output (1 neuron)

4.4 NN Model Hyperparameters

The complete NN model configuration is:

```
"model": {
    "descriptor": {
        "Rmax": 6.0,
        "Rmin": 0.5,
        "feature_type": [3,4]
    },
    "fitting_net": {
        "network_size": [15,15,1]
    }
}
```

4.4.1 Rmax

Maximum feature cutoff radius. The default is 6.0Å.

4.4.2 Rmin

Minimum feature cutoff radius. The default is 0.5Å.

4.4.3 feature_type

Selects feature types. Supported values are `[1,2]`, `[3,4]`, `[5]`, `[6]`, `[7]`, and `[8]`. The default `[3,4]` selects two- and three-body Gaussian features. See Appendix 1.

4.4.4 network_size

Sets the fitting-network structure. The default `[15,15,1]` gives: Input -> hidden layer 1 (15 neurons) -> hidden layer 2 (15 neurons) -> output (1 neuron)

4.5 Linear Model Hyperparameters

The complete Linear model configuration is:

```
"model": {
    "descriptor": {
        "Rmax": 6.0,
        "Rmin": 0.5,
        "feature_type": [3,4]
    }
}
```

4.5.1 Rmax

Maximum feature cutoff radius. The default is 6.0Å.

4.5.2 Rmin

Minimum feature cutoff radius. The default is 0.5Å.

4.5.3 feature_type

Selects feature types using the same settings as the NN model. Supported values are `[1,2]`, `[3,4]`, `[5]`, `[6]`, `[7]`, and `[8]`. The default `[3,4]` selects two- and three-body Gaussian features. See Appendix 1.

4.6 ADAM Optimizer Hyperparameters

The complete ADAM configuration is:

```
"optimizer": {
    "optimizer": "ADAM",
    "epochs": 30,
    "reset_epoch":false,
    "batch_size": 1,
    "print_freq": 10,
    "lambda_2" : 0.1,
    "learning_rate": 0.001,
    "stop_lr": 3.51e-08,
    "stop_step": 1000000,
    "decay_step": 5000,
    "max_norm": 0.5,
```

```
        "norm_type": 2,
        "t_0": 6,
        "t_mult": 1,
        "train_energy": true,
        "train_force": true,
        "train_virial": false,
        "start_pre_fac_force": 1000,
        "start_pre_fac_etot": 0.02,
        "start_pre_fac_virial": 50.0,
        "end_pre_fac_force": 1.0,
        "end_pre_fac_etot": 1.0,
        "end_pre_fac_virial": 1.0
    }
```

4.6.1 optimizer

Selects the optimizer. The default is ADAM; use LKF for the LKF optimizer. See the LKF paper.

4.6.2 reset_epoch

Whether restarted training begins at epoch 0. The default is `true`. Set `false` to resume from a checkpoint and restore ADAM′ s first- and second-moment states.

4.6.3 epochs

Sets the number of epochs. Each epoch processes the entire dataset in mini-batches with forward propagation, loss calculation, and backpropagation. The default is 30.

Choose the epoch count by monitoring training and validation. Too few epochs may underfit; too many may overfit and reduce generalization.

4.6.4 batch_size

Sets the number of samples in each mini-batch. The default is 1.

4.6.5 print_freq

Sets the number of mini-batch iterations between training-error reports. The default is 10.

4.6.6 train_energy

Whether to train on total energy. The default is `true`.

4.6.7 train_force

Whether to train on forces. The default is `true`.

4.6.8 train_virial

Whether to train on virials. The default is `false`.

4.6.9 lambda_2

Sets ADAM L2 regularization. It is disabled by default and may reduce overfitting.

4.6.10 learning_rate

Sets the base learning rate, default `0.001`. For NEP Adam/AdamW training, the actual peak rate is `learning_rate × lr_scale`; `lr_scale=1` when `scale_lr=false`, or is determined by `scale_method` when scaling is enabled.

When both `t_0` and `t_mult` are set, NEP uses cosine annealing with warm restarts. `t_0` is the first cycle length in epochs (internally multiplied by the number of updates per epoch), and each subsequent cycle length is multiplied by `t_mult`. Within a cycle, the learning rate approaches `stop_lr` before returning to the peak at the restart. Without these parameters, the stepwise exponential decay described under `decay_step` applies.

4.6.11 stop_lr

The minimum learning rate, default `3.51e-08`. Without periodic restarts, the rate stays at `stop_lr` after the cumulative update count exceeds `stop_step`. With periodic restarts, it is the minimum of each cosine cycle.

4.6.12 stop_step

Effective only when periodic restarts are disabled. The default is `1000000` cumulative optimizer updates. After this count, the learning rate stays at `stop_lr`; NEP Adam/AdamW loss prefactors also reach their `end_pre_fac_*` values. With periodic restarts enabled, this parameter does not enter the learning-rate or loss-prefactor calculation.

4.6.13 decay_step

Effective only when periodic restarts are disabled; default `5000`. The learning rate drops one level after every `decay_step` optimizer updates. Let k be the zero-based cumulative update count and `peak_lr = learning_rate × lr_scale`:

```
decay_rate = exp(log(stop_lr / peak_lr) / (stop_step / decay_step))
if k > stop_step:
    real_lr = stop_lr
else:
    real_lr = peak_lr * decay_rate ** (k // decay_step)
```

4.6.14 Gradient Clipping

4.6.14.1 Norm-Based Clipping: max_norm and norm_type `max_norm` and `norm_type` configure norm-based clipping. The default `max_norm=None` disables it.

If the norm of all parameter gradients exceeds `max_norm`, gradients are scaled proportionally until the norm equals `max_norm`.

This preserves relative gradient directions while preventing exploding gradients. `norm_type` selects the norm.

`norm_type` is integer 1 for L1 or 2 for L2. The default is 2.

The L1 norm is the sum of absolute gradient values: $\|g\|_1 = \sum_{i=1}^{n} |g_i|$, If $\|g\|_1 >$ max_norm, gradients are scaled to

$$g_{\text{clipped}} = g \cdot \frac{\text{max_norm}}{\|g\|_1}$$

The L2 norm is the Euclidean gradient norm: $\|g\|_2 = \sqrt{\sum_{i=1}^{n} g_i^2}$, If $\|g\|_2 >$ max_norm, gradients are scaled to

$$g_{\text{clipped}} = g \cdot \frac{\text{max_norm}}{\|g\|_2}$$

4.6.14.2 Value-Based Clipping: clip_value Clips each gradient component to `[-clip_value, clip_value]`. The default `None` disables it.

$$g_i^{(\text{clipped})} = \begin{cases} \text{clip_value}, & \text{if } g_i > \text{clip_value} \\ -\text{clip_value}, & \text{if } g_i < -\text{clip_value} \\ g_i, & \text{otherwise} \end{cases}$$

4.6.15 Cosine Annealing with Warm Restarts: t_0 and t_mult

Positive integers `t_0` and `t_mult` enable cosine annealing with warm restarts for ADAM. Once enabled, `optim.lr_scheduler.CosineAnnealingWarmRestarts` controls the learning rate and the `decay_step` schedule is disabled.

- `t_0`: length of the first cycle, in epochs.
- `t_mult`: factor by which the cycle length grows after each restart.
- The epoch E_n at which cycle n ends and the learning rate reaches its minimum is calculated as follows:
  - For `t_mult = 1`, cycles have fixed length `t_0`.
  - For `t_mult > 1`, cycle lengths grow geometrically:

$$E_n = t_0 \times \frac{T_{mult}^n - 1}{T_{mult} - 1} \quad (n = 1, 2, 3, \dots)$$

- When cosine annealing is enabled, the model at each E_n, immediately before restart, is saved under `model_record/saved_models`.

In the figure below, learning_rate is 0.001, `t_0=1`, `t_mult=2`, and stop_lr is 3.51e-08. Restarts occur at epochs 1, 3, 7, 15, and so on.

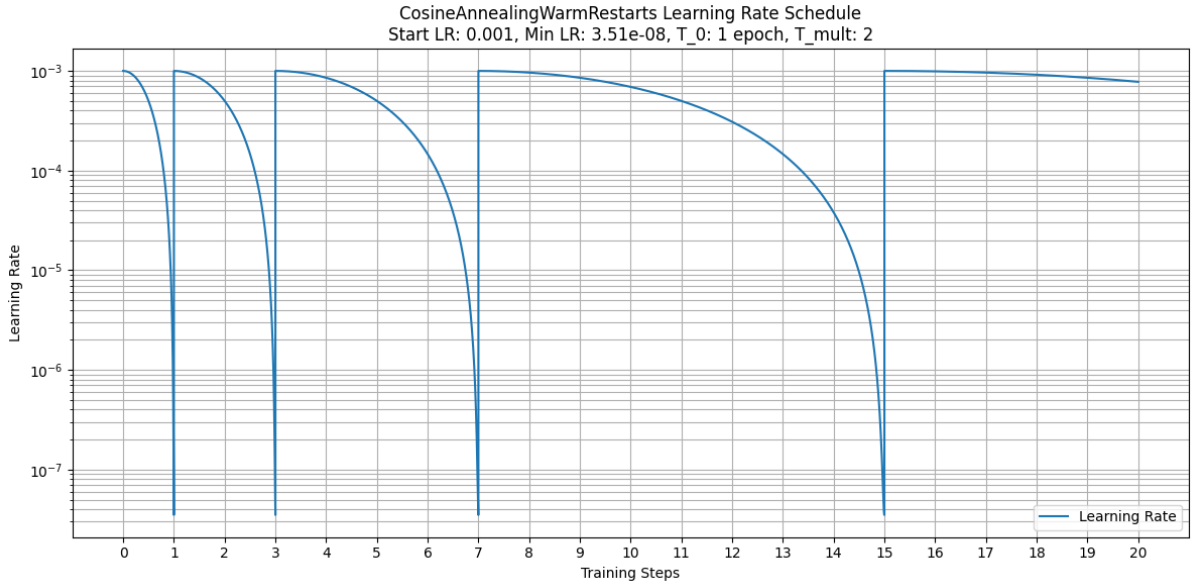


Figure 2: AL_T0_T_mult

4.6.16 Statistics of the first five restart epochs for each group

Combination (T_0, T_mult)	1st restart	2nd restart	3rd restart	4th restart	5th restart
(1, 2)	1	3	7	15	31
(1, 6)	1	7	43	259	1555
(2, 2)	2	6	14	30	62
(2, 4)	2	10	42	170	682
(2, 1)	2	4	6	8	10

4.6.17 Detailed calculation process

1. **(1, 2)**: The cycle lengths are `1, 2, 4, 8, 16...` respectively
   → Restart points: 1, `1+2=3`, `3+4=7`, `7+8=15`, `15+16=31`
2. **(1, 6)**: The cycle lengths are `1, 6, 36, 216, 1296...` respectively
   → Restart points: 1, `1+6=7`, `7+36=43`, `43+216=259`, `259+1296=1555`
3. **(2, 2)**: The cycle lengths are `2, 4, 8, 16, 32...` respectively
   → Restart points: 2, `2+4=6`, `6+8=14`, `14+16=30`, `30+32=62`
4. **(2, 4)**: The cycle lengths are `2, 8, 32, 128, 512...` respectively
   → Restart points: 2, `2+8=10`, `10+32=42`, `42+128=170`, `170+512=682`
5. **(2, 1)**: The cycle length is constant at `2, 2, 2, 2, 2...`
   → Restart points: `2, 4, 6, 8, 10` (arithmetic progression)

4.6.18 Learning-Rate Scaling (scale_lr / scaling_method)

`scale_lr` controls whether the base learning rate is scaled; its default is `false`. When disabled, the scale factor is 1. When enabled, a factor is calculated once at training initialization, and the actual peak rate is `learning_rate × scale factor`. This does not change `stop_lr`.

`scaling_method` selects the scaling rule and defaults to `sqrt`. The current code also accepts the synonymous key `scale_method`. Prefer `scale_method` in new configurations; existing `scaling_method` configurations remain valid. Setting both keys to different values raises an error.

Let B be the local batch size per training process, P the total number of training processes (usually the total GPU count in GPU training), and A the mean atom count in the training set. The supported values are:

scaling_method / scale_method	Scale factor
linear_gpu	P
sqrt_batch	$\sqrt{B}$
sqrt_gpu	$\sqrt{P}$
sqrt (default)	$\sqrt{BP}$
sqrt_batch_gpu_atom	$\sqrt{BPA}$

For NEP LMDB data, when batch_size specifies a number of structures, B is that per-process frame count. With "mix:<atom count>", the code estimates B as max(1, mix value / mean atom count in the training set); do not substitute the raw mix value for B in the table. The selected method has no effect unless scale_lr=true.

```
"scale_lr": true,
"scale_method": "sqrt"
```

4.6.19 Energy and Force Prefactors in the Loss

The following parameters set the initial and final weights of force, total-energy, and virial losses. All values must be nonnegative:

- start_pre_fac_force: default 100; end_pre_fac_force: default 1.0.
- start_pre_fac_etot: default 1.0; end_pre_fac_etot: default 1.0.
- start_pre_fac_virial: default 0.1; end_pre_fac_virial: default 0.1.

With periodic restarts enabled, Adam/AdamW training uses the learning rate η_k actually applied to the current optimizer update. Let $\eta_{\text{peak}} = \texttt{learning_rate} \times \texttt{lr_scale}$. Then:

$$r_k = \mathrm{clip}\left(\frac{\eta_k}{\eta_{\text{peak}}}, 0, 1\right), \qquad p_x(k) = p_x^{\text{end}} + \left(p_x^{\text{start}} - p_x^{\text{end}}\right) r_k.$$

At each learning-rate restart, the prefactor returns to its initial weight; near the end of a cycle, it approaches its final weight. Because stop_lr is usually positive, the prefactor near the cycle minimum may remain slightly above end_pre_fac_*.

Without periodic restarts, let k be the zero-based cumulative optimizer update count. The prefactor changes linearly with stop_step:

$$q_k = \mathrm{clip}\left(\frac{k}{\texttt{stop_step}}, 0, 1\right), \qquad p_x(k) = p_x^{\text{start}} + \left(p_x^{\text{end}} - p_x^{\text{start}}\right) q_k.$$

The final weight is maintained after `stop_step`. For example, the default force weight changes from `100` to `1.0`, giving $p_F(k) = 100 - 99q_k$.

For enabled training targets, the total loss is:

$$L = p_F L_F^{\mathrm{MSE}} + p_{\mathrm{etot}} \frac{L_{\mathrm{Etot}}^{\mathrm{MSE}}}{N_{\mathrm{avg}}} + p_{\mathrm{virial}} \frac{L_{\mathrm{Virial}}^{\mathrm{MSE}}}{N_{\mathrm{avg}}},$$

where N_{avg} is the mean atom count per structure in the current batch. Disabled targets do not contribute. The numerical size of a prefactor alone does not establish the corresponding term′ s contribution to the parameter gradients.

4.7 KF Optimizer Hyperparameters

The complete KF configuration is:

```
"optimizer": {
    "optimizer": "LKF",
    "epochs": 30,
    "batch_size": 1,
    "print_freq": 10,
    "block_size": 5120,
    "p0_weight": 0.01,
    "kalman_lambda": 0.98,
    "kalman_nue": 0.9987,
    "train_energy": true,
    "train_force": true,
    "train_virial": false,
    "pre_fac_force": 2.0,
    "pre_fac_etot": 1.0,
    "pre_fac_virial": 1.0
}
```

`optimizer`, `epochs`, `batch_size`, `print_freq`, `train_energy`, `train_force`, and `train_virial` have the same meanings as for ADAM.

4.7.1 block_size

Sets the block size of LKF covariance matrix P. Larger blocks consume more memory and train more slowly; smaller blocks may reduce convergence speed and accuracy. The default is `5120`; `10240` is recommended for A100 or H100 GPUs.

4.7.2 p0_weight

LKF regularization parameter. The default `0.01` reduces overfitting and must be below `1`; setting it to `1` disables regularization.

4.7.3 kalman_lambda

LKF memory factor controlling the weight of previous data. Larger values retain more history. The default is `0.98`.

4.7.4 kalman_nue

LKF forgetting rate controlling how quickly `kalman_lambda` changes. The default is `0.9987`.

4.7.5 pre_fac_etot

Weight of total energy in the loss. The default is `1.0`.

4.7.6 pre_fac_force

Weight of force in the loss. The default is `2.0`.

4.7.7 pre_fac_virial

Weight of virial in the loss. The default is `1.0`.

1. Multi-GPU NEP training does not support LKF or GKF.
2. The size of covariance matrix P scales with the square of `N/block_size`, where N is the parameter count. Training many element types can therefore exhaust GPU memory and converge slowly.

5 MatPL Tutorials

Source repository

MatPL includes 8 descriptor types with translational, rotational, and permutational invariance:

1. 2-body(2b)
2. 3-body(3b)
3. 2-body Gaussian(2b gauss)
4. 3-body Cosine(3b cos)
5. Moment Tensor Potential(MTP)
6. Spectral Neighbor Analysis Potential(SNAP)
7. DP-Chebyshev(dp1)
8. DP-Gaussian(dp2)

4 trainable model types:

1. Linear
2. Neural Network(NN)
3. DP se_e2_a(Pytorch)
4. Neuroevolution Potential (NEP) (supports multi-GPU, multi-node training and ultra-fast large-scale LAMMPS MD)

2 efficient training optimizers:

1. Adaptive Moment Estimation (ADAM)
2. Reorganized Layer Extended Kalman Filtering (LKF)

5.1 NEP Model

Tutorial

5.1.1 Model Overview

The NEP model was originally implemented in the GPUMD package (GPUMD 2022b). GPUMD trains NEP models using the separable natural evolution strategy (SNES), which is straightforward to implement because it does not rely on gradient information. For standard supervised-learning tasks, however—especially deep learning—gradient-based optimization algorithms are generally more suitable. Beginning with `MatPL 2025.3`, MatPL implements the NEP model (NEP5; its network architecture is shown in Figure 1) and supports model training with the `gradient-based LKF or ADAM optimizer`. We have also optimized the most time-consuming parts of gradient computation with C++/CUDA operators, substantially improving training speed.

We compared the training efficiency of LKF and SNES across multiple systems. The results show that LKF delivers superior training accuracy and convergence speed for NEP models.

Because an NEP network contains only one hidden layer, inference is already extremely fast; introducing the LKF optimizer also greatly improves training efficiency. MatPL therefore enables users to obtain high-quality NEP models at relatively low training cost and use them for efficient machine-learning molecular dynamics, which is especially valuable when computational resources or budgets are limited.

We have also implemented a LAMMPS molecular-dynamics interface for NEP models that supports both CPU and GPU devices. Thanks to NEP's compact network architecture and streamlined feature design, the model provides extremely fast inference in LAMMPS and supports multi-node execution, including GPUs across nodes.

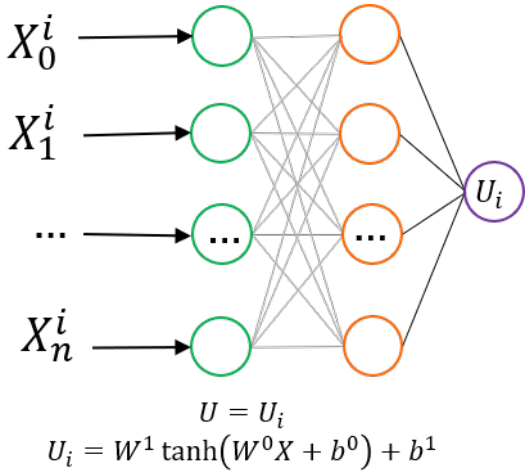


Figure 3: NEP network architecture

`In MatPL-2026.3`, NEP has been extensively optimized.

For training on a single GPU, optimized gradient operators provide a `more than three-fold speedup` over MatPL-2025.3. Multi-node, multi-GPU large-batch training has also been introduced, together with measures that mitigate the loss of accuracy commonly associated with large batches, delivering a `major improvement` in training efficiency.

For LAMMPS simulations, the most time-consuming neighbor calculations are offloaded from the CPU to the GPU through Kokkos. The inference kernels have also been optimized, reducing GPU memory usage by more than one third while making simulations `over an order of magnitude faster` than MatPL-2025.3.

Here, `nep4.txt` refers to a force-field file trained with GPUMD, while `nep5.txt` is trained with MatPL. In `nep5.txt`, the output layer of each subnet has its own independent bias parameter; in `nep4.txt`, all subnet output layers share a single bias term.

5.1.2 NEP Training Benchmarks

The following figures show loss convergence over time during multi-node, multi-GPU training with MatPL. The horizontal axis represents training time, and the vertical axis represents the training loss.

Training the FeC dataset (16,084 structures) on two nodes with eight GPUs:

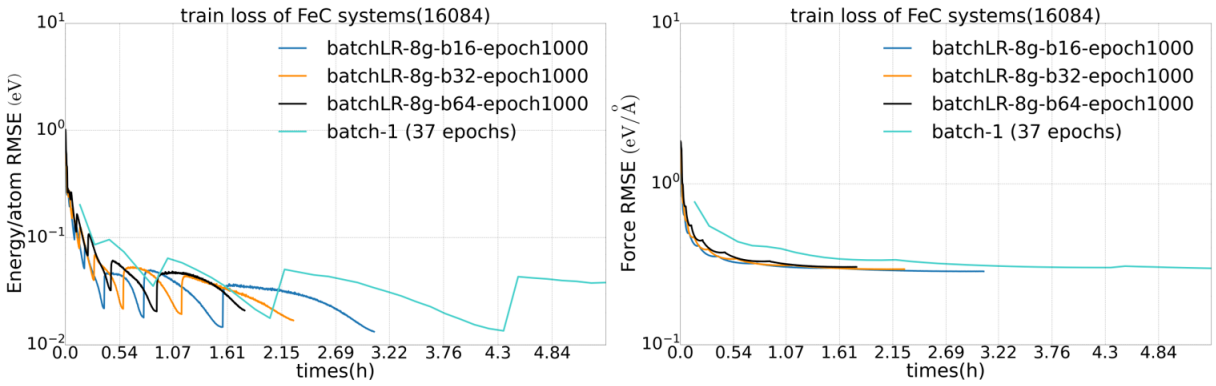


Figure 4: NEP training on the FeC dataset

Training the AuAg alloy dataset (5,762 structures) on two nodes with eight GPUs:

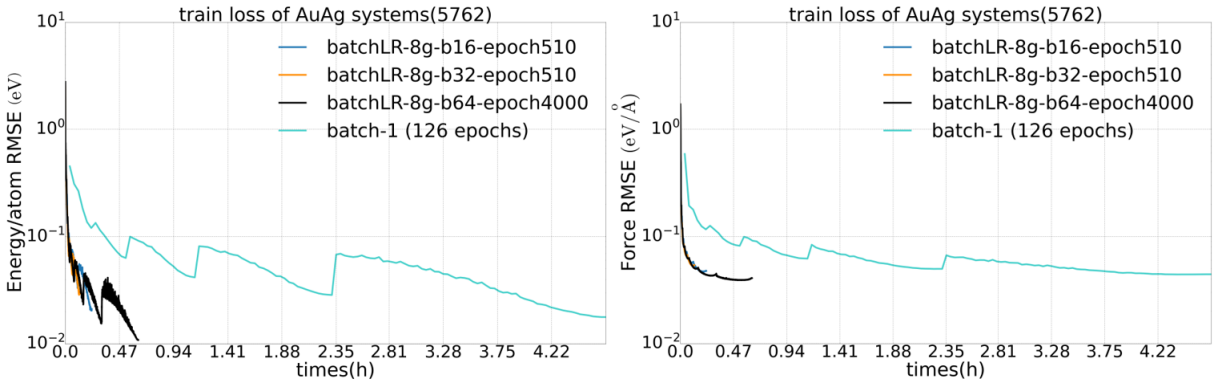
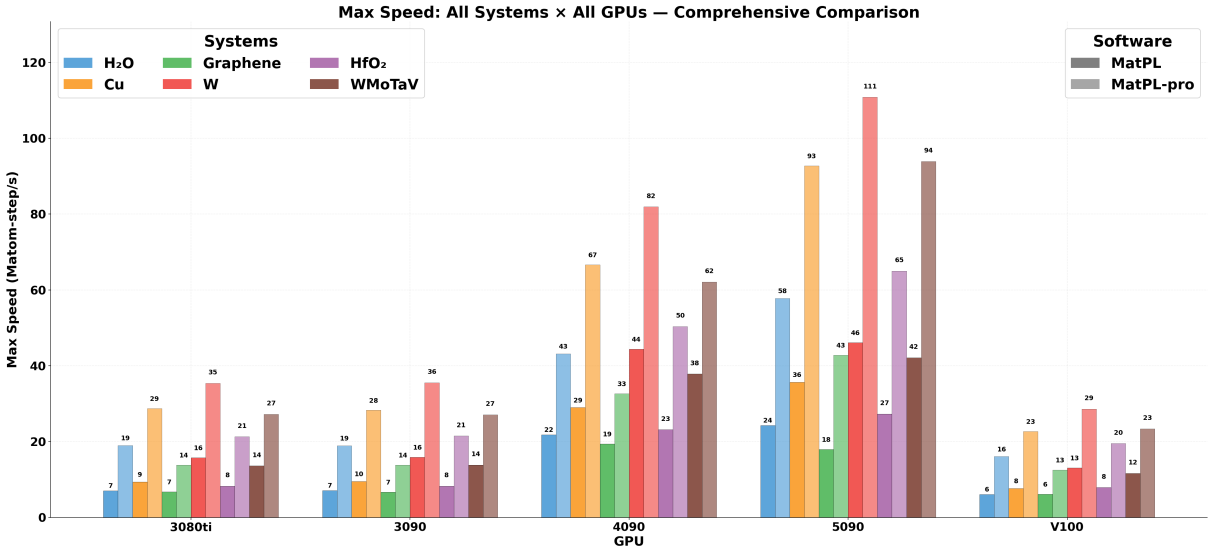
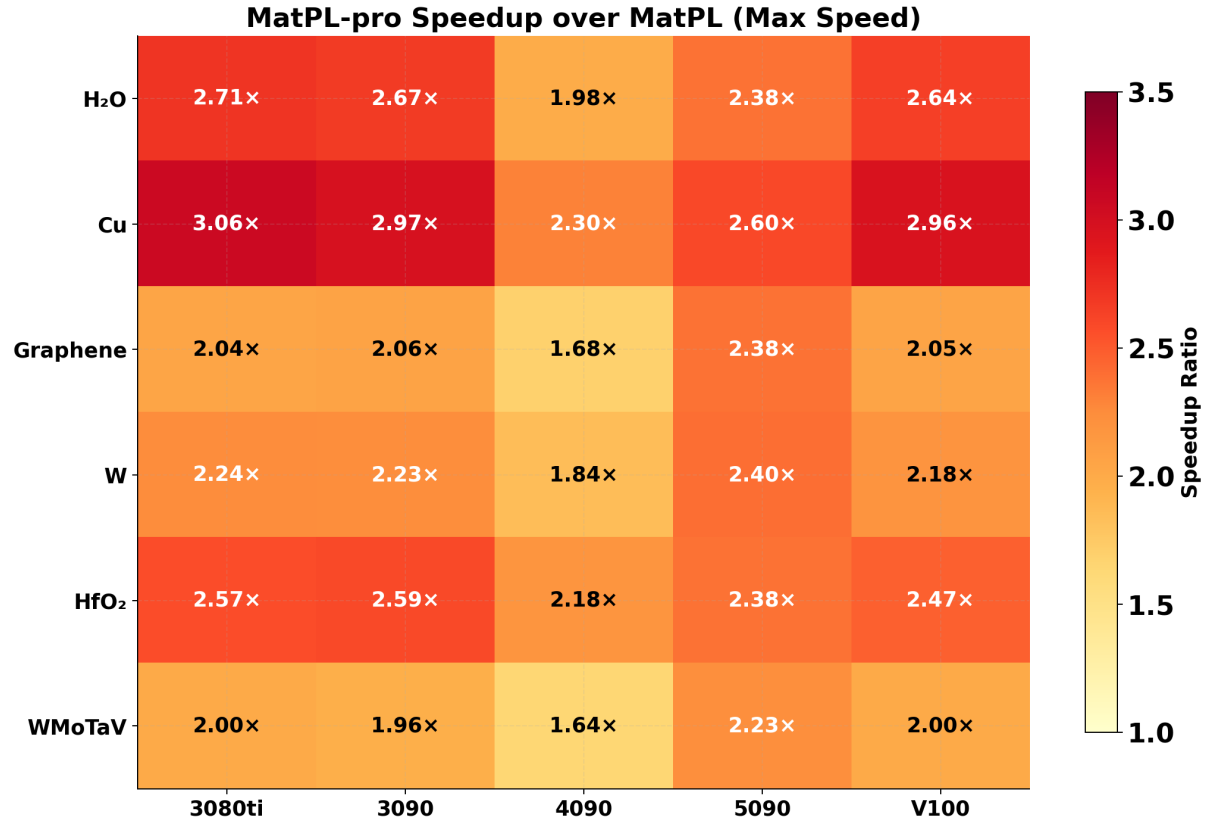


Figure 5: NEP training on the AuAg dataset

5.1.3 LAMMPS Interface Benchmarks

The following figures show the LAMMPS simulation speed of the NEP Kokkos version. The NEP/KK LAMMPS Kokkos interface is now available. The Pro version is two to three times faster than the standard version.





5.2 NEP Tutorial

This tutorial uses the MatPL `source root/example/HfO2/nep_demo` example (source of the HfO2 training dataset) to demonstrate training, testing, LAMMPS simulation, and other operations for an NEP model. The current example directory is organized as follows:

```
nep_demo/
├── nep_kokkos_lmps/
│   ├── kkin.lmp
```

```
│   ├── lmp.config
│   ├── nep_to_lmps.txt
│   ├── rungpu_mcloud.job
│   └── rungpu_station.job
├── nep_lmps/
│   ├── in.lmp-cpu-25
│   ├── in.lmp-kk
│   ├── lmp.config
│   ├── nep_to_lmps.txt
│   ├── runcpu_mcloud.job
│   ├── runcpu_station.job
│   ├── rungpu_mcloud.job
│   └── rungpu_station.job
├── nep_lmps_deviation/
│   ├── in.lmp-kk
│   ├── lmp.config
│   ├── nep0.txt
│   ├── nep1.txt
│   ├── nep2.txt
│   ├── nep3.txt
│   ├── rungpu_mcloud.job
│   └── rungpu_station.job
├── nep_test.json
├── nep_train.json
├── train_mcloud.job
└── train_station.job
```

- `nep_train.json` and `nep_test.json` are the NEP training and testing configurations. Their `../pwdata/` paths refer to the `Hf02/pwdata` training-data directory.
- `train_mcloud.job` and `train_station.job` are example Slurm training scripts for Mcloud and a self-managed cluster, respectively.
- `nep_lmps` provides input files and job scripts for both CPU and NEP Kokkos GPU simulations.
- `nep_kokkos_lmps` provides an NEP Kokkos GPU simulation example.
- `nep_lmps_deviation` provides a four-model deviation example for active-learning workflows.
- In each LAMMPS example directory, `lmp.config` is the initial structure; `nep_to_lmps.txt` or `nep*.txt` are NEP force-field files; `*.lmp*` are LAMMPS input files; and `*.job` are Slurm job scripts.

5.2.1 train

Run the following command in the `nep_demo` directory to start training:

```
MatPL train nep_train.json
# Alternatively, adapt the Slurm script to the environment and submit
 ↪  it
sbatch train_mcloud.job
# On a self-managed cluster, use sbatch train_station.job
```

Input file description

The contents of `nep_train.json` are shown below. For descriptions of the NEP parameters, see the NEP Parameter Reference:

```
{
    "model_type": "NEP",
    "atom_type": [
        8, 72
    ],
    "optimizer": {
        "optimizer": "ADAM",
        "epochs": 30,
        "batch_size": 1,
        "print_freq": 10,
        "train_energy": true,
        "train_force": true,
        "train_virial": true
    },

    "format": "pwmlff/npy",
    "train_data": [
        "../pwdata/init_000_50/", "../pwdata/init_002_50/",
        "../pwdata/init_004_50/", "../pwdata/init_006_50/",
        "../pwdata/init_008_50/", "../pwdata/init_010_50/",
        "../pwdata/init_012_50/", "../pwdata/init_014_50/",
        "../pwdata/init_016_50/", "../pwdata/init_018_50/",
        "../pwdata/init_020_20/", "../pwdata/init_022_20/",
        "../pwdata/init_024_20/", "../pwdata/init_026_20/",
        "../pwdata/init_001_50/", "../pwdata/init_003_50/",
        "../pwdata/init_005_50/", "../pwdata/init_007_50/",
        "../pwdata/init_009_50/", "../pwdata/init_011_50/",
        "../pwdata/init_013_50/", "../pwdata/init_015_30/",
        "../pwdata/init_017_50/", "../pwdata/init_019_50/",
        "../pwdata/init_021_20/", "../pwdata/init_023_20/",
        "../pwdata/init_025_20/", "../pwdata/init_027_20/"
    ],
    "valid_data":[
        "../pwdata/init_000_50/", "../pwdata/init_004_50/",
```

```
        "../pwdata/init_008_50/"
    ]
}
```

For details about the force-field directory generated after training, see model_record details.

5.2.2 Multi-Node, Multi-GPU Training

Multi-node, multi-GPU training uses the same directory structure. See the MatPL `source root/example/parallelnep` example (source of the HfO2 training dataset).

The directory provides three example launch scripts for Mcloud users: `1node-1g-run.job` for one GPU on one node, `1node-4g-run.job` for multiple GPUs on one node, and `2node-8g-run.job` for multiple nodes and GPUs. For an online installation, see `env.sh` for the MatPL-2026.3 environment setup.

Multi-node, multi-GPU training requires the `address of the primary node` and an `available port`. We recommend obtaining them automatically with the following shell commands:

```
MASTER_ADDR=$(scontrol show hostnames $SLURM_JOB_NODELIST | head -n 1)
# Dynamically allocate an available port
function get_free_port() {
    python -c 'import socket; s = socket.socket(socket.AF_INET,
     ↪  socket.SOCK_STREAM); s.bind(("", 0));
     ↪  print(s.getsockname()[1]); s.close()'
}
MASTER_PORT=$(get_free_port)

export MASTER_ADDR=$MASTER_ADDR
export MASTER_PORT=$MASTER_PORT

echo "addrs: $MASTER_ADDR"
echo "port:  $MASTER_PORT"
echo "tasks: $SLURM_NTASKS"

srun MATPL train train.json
```

Multi-GPU NEP training supports only the ADAM optimizer; LKF and GKF are not supported.

5.2.3 test

The `test` command accepts a MatPL `nep_model.ckpt` force field as well as `nep5.txt` and `nep4.txt` files used by LAMMPS or GPUMD.

```
MatPL test nep_test.json
```

The contents of `test.json` are shown below. For parameter descriptions, see the Parameter Reference.

```
{
    "model_type": "NEP",
    "format": "pwmlff/npy",
    "model_load_file": "./model_record/nep_model.ckpt",
    "test_data": [
        "../init_000_50", "../init_004_50", "../init_008_50",
        "../init_012_50", "../init_016_50", "../init_020_20",
        "../init_024_20", "../init_001_50", "../init_005_50",
        "../init_009_50", "../init_013_50", "../init_017_50",
        "../init_021_20", "../init_025_20", "../init_002_50",
        "../init_006_50", "../init_010_50", "../init_014_50",
        "../init_018_50", "../init_022_20", "../init_026_20",
        "../init_003_50", "../init_007_50", "../init_011_50",
        "../init_015_30", "../init_019_50", "../init_023_20",
        "../init_027_20"
    ]
}
```

For details about the output directory generated after testing, see test_result details.

5.2.4 infer a Single Structure

The `infer` command accepts a MatPL `nep_model.ckpt` file, a GPUMD `nep4.txt` file, or a `nep5.txt` file shared by LAMMPS and GPUMD.

```
MatPL infer nep_model.ckpt atom.config pwmat/config
MatPL infer gpumd_nep.txt 0.lammpstrj lammps/dump Hf O
# Hf and O are the element names in the lammps/dump structure: Hf is
 ↪  atom type 1 and O is atom type 2
```

After successful inference, the terminal displays the total energy, per-atom energies, per-atom forces, and virial.

5.2.5 totxt: Convert a Trained Checkpoint to nep5.txt

This command converts a `nep_model.ckpt` trained by MatPL to a text-format `nep5.txt` file for molecular-dynamics simulations in GPUMD or lammps-MatPL.

```
MatPL totxt nep_model.ckpt
```

When successful, it creates `nep5.txt` in the current directory.

5.2.6 lammps MD

The HfO2 example provides separate directories for a single model, NEP Kokkos, and multi-model deviation calculations. The workflow below proceeds through preparing the force field, choosing an example, editing the input, and launching the simulation.

5.2.6.1 1. Prepare the Force-Field File After normal training, `nep5.txt` is generated in `model_record` and can be used directly in LAMMPS. If only `nep_model.ckpt` is available, convert it with the `totxt` command described above:

```
MatPL totxt nep_model.ckpt
```

Copy the generated `nep5.txt` to the selected example directory and name it `nep_to_lmps.txt` as referenced by the input script. lammps-MatPL also supports GPUMD NEP4 and NEP5 force-field files.

5.2.6.2 2. Select a LAMMPS Example

Example directory	Purpose	Main input files
`nep_lmps`	CPU and NEP Kokkos GPU simulations	`in.lmp-cpu-25`, `in.lmp-kk`
`nep_kokkos_lmps`	NEP Kokkos GPU simulation	`kkin.lmp`
`nep_lmps_deviation`	Four-model deviation calculation	`in.lmp-kk`, `nep0.txt`–`nep3.txt`

Each directory provides both `*_mcloud.job` and `*_station.job`. Select the appropriate script and update module names, software paths, partitions, and GPU resource parameters for your environment.

5.2.6.3 3. Configure the LAMMPS Input File The NEP Kokkos GPU interface uses a half neighbor list with Newton communication enabled. The essential settings for the single-model HfO2 example are:

```
package kokkos neigh half comm device
newton on

pair_style   matpl/nep/kk nep_to_lmps.txt
pair_coeff   * * 72 8
```

List the elements after `pair_coeff * *` in the atom-type order used by the data file. Element symbols or atomic numbers are both accepted. In this example, type 1 is Hf (72) and type 2 is O (8).

In the multi-model example, the first model drives MD while the others participate in the deviation calculation:

```
pair_style    matpl/nep/kk nep0.txt nep1.txt nep2.txt nep3.txt \
              out_freq ${DUMP_FREQ} out_file model_devi.out
pair_coeff    * * 72 8
```

`out_freq` sets the deviation-output frequency, and `out_file` sets the output filename. For complete NPT control parameters, refer directly to `in.lmp-kk`, `in.lmp-cpu-25`, or `kkin.lmp` in the examples.

5.2.6.4 4. Start the LAMMPS Simulation

From the example directory, submit the Slurm script appropriate for the runtime environment, for example:

```
sbatch rungpu_mcloud.job
# or
sbatch rungpu_station.job
```

You can also run directly after loading the LAMMPS environment:

```
# One GPU: nep_kokkos_lmps/kkin.lmp
mpirun -np 1 --bind-to numa lmp -k on g 1 -sf kk -pk kokkos -in
 ↪  kkin.lmp

# Four GPUs on one node: nep_lmps/in.lmp-kk
mpirun -np 4 --bind-to numa lmp -k on g 4 -sf kk -pk kokkos -in
 ↪  in.lmp-kk

# Two nodes with four GPUs per node
mpirun -np 8 --bind-to numa --map-by ppr:4:node \
    lmp -k on g 4 -sf kk -pk kokkos -in in.lmp-kk
```

When using the closed-source MatPL Pro version, set `NEP_GPU_LIB_PATH` and `NEP_LICENSE_PATH` before running. For multi-node jobs, every node must be able to access the shared library, license, input files, and force-field files.

5.2.6.5 MatPL Heat-Flux Calculation

> **Interface distinction:** The open-source version uses `matpl/heatflux`; the closed-source MatPL Pro version uses the Kokkos GPU heat-flux interface `matpl/heatflux/kk`. Select the command that matches the installed lammps-MatPL interface.

For the open-source version, configure the heat-flux compute as follows:

```
compute       flux all matpl/heatflux
```

In the closed-source MatPL Pro version, `matpl/heatflux/kk` calculates MatPL heat flux directly on the GPU and does not require the conventional `ke/atom + pe/atom + centroid/stress/atom + heat/flux` post-processing chain:

```
package kokkos neigh half comm device
newton on

pair_style    matpl/nep/kk nep.txt
pair_coeff    * * C

compute       flux all matpl/heatflux/kk
fix           fluxout all matpl/heatflux/ave/kk 10 100 1000 flux file
 ↪  compute_HeatFlux.out
```

`compute matpl/heatflux/kk` outputs a global vector with six components:

1. `Jx`: total heat flux in the x direction.
2. `Jy`: total heat flux in the y direction.
3. `Jz`: total heat flux in the z direction.
4. `Jconv,x`: convective heat flux in the x direction.
5. `Jconv,y`: convective heat flux in the y direction.
6. `Jconv,z`: convective heat flux in the z direction.

The virial contribution to heat flux is therefore the total heat flux minus the convective heat flux: `(1-4, 2-5, 3-6)`. `fix matpl/heatflux/ave/kk` time-averages the heat-flux vector and writes the results to `compute_HeatFlux.out`.

5.2.6.6 Heat-Flux Examples

- Open-source Heat_flux example
- Closed-source MatPL Pro Heat_flux example

5.2.7 ASE Interface

The NEP model provides an ASE interface. See the example scripts on Gitee or GitHub.

```
from src.ase.calculate import MatPL_calculator
calc = MatPL(model_file='nep_model.ckpt or nep.txt')
atoms = ..... # Create ase.atoms.Atoms
atoms.calc = calc # Or use atoms.set_calculator(calc)
energy = atoms.get_potential_energy()
```

```
forces = atoms.get_forces()
stress = atoms.get_stress()
```

Before using the ASE interface, make sure the MatPL environment variables have been loaded.

5.3 DP Model

Tutorial

5.3.1 DP Model Overview

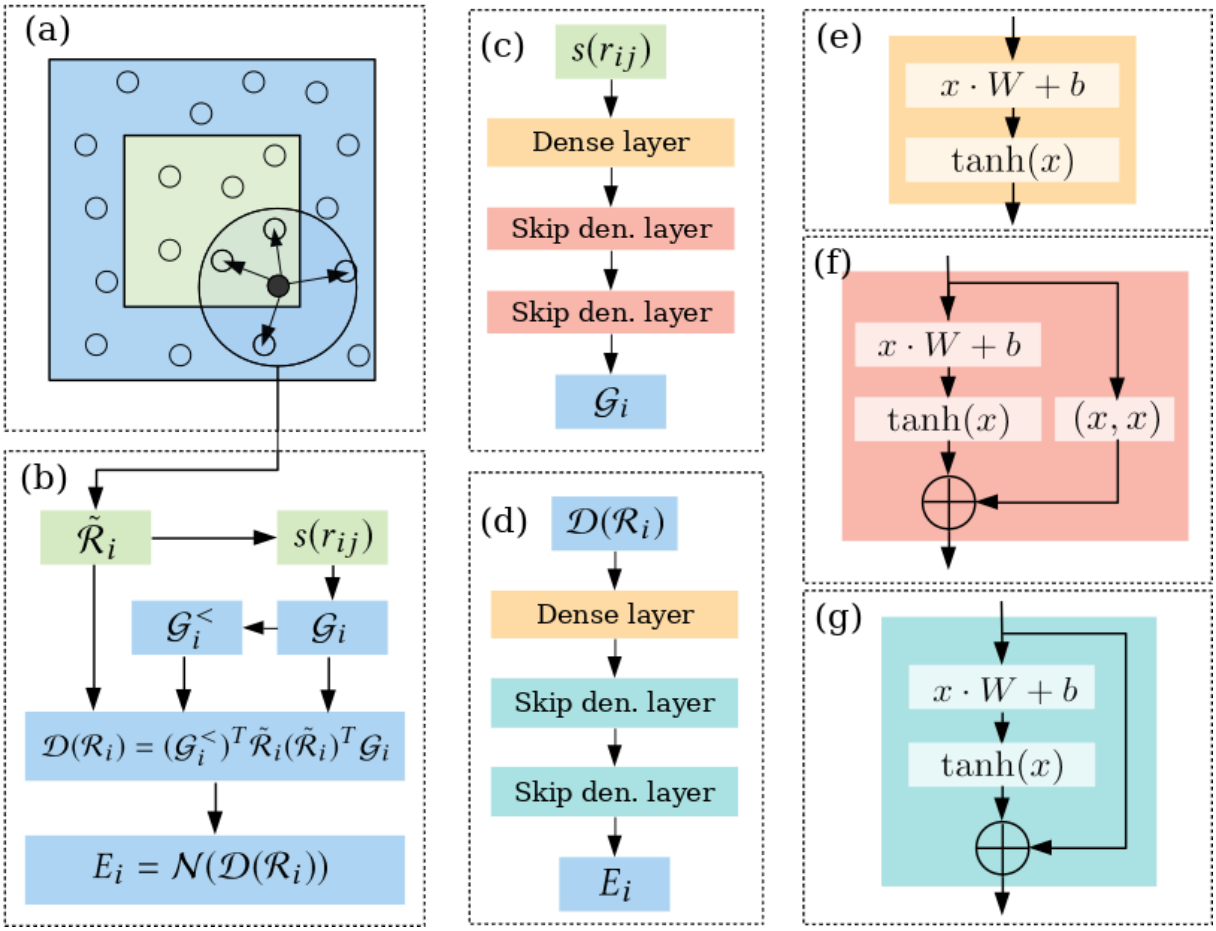


Figure 6: DP network architecture

For the DP model, see the following references:

- [SC′ 20] Weile Jia, Han Wang, Mohan Chen, Denghui Lu, Lin Lin, Roberto Car, E Weinan, Linfeng Zhang*, "Pushing the Limit of Molecular Dynamics with Ab Initio Accuracy to 100 Million Atoms with Machine Learning," SC20: International Conference for

High Performance Computing, Networking, Storage and Analysis, 2020, pp. 1-14, doi: 10.1109/SC41405.2020.00009.(CCF-A)(Gordon Bell Prize)

- Han Wang, Linfeng Zhang, Jiequn Han, and Weinan E. "DeePMD-kit: A deep learning package for many-body potential energy representation and molecular dynamics." Computer Physics Communications 228 (2018): 178-184. doi:10.1016/j.cpc.2018.03.016
- Zhang L, Han J, Wang H, et al. End-to-end symmetry preserving inter-atomic potential energy model for finite and extended systems[J]. Advances in neural information processing systems, 2018, 31.
- Lu D, Jiang W, Chen Y, et al. DP compress: A model compression scheme for generating efficient deep potential models[J]. Journal of chemical theory and computation, 2022, 18(9): 5559-5567.

5.3.2 type embedding

A DP model contains N^2 Embedding Nets, where N is the number of element types. When a system contains many element types, this design limits both training and inference speed and restricts the potential of DP as a general-purpose large model. Because the N^2 Embedding Nets implicitly encode element types, we modify S_{ij} by concatenating it with the physical properties of the corresponding element type. This enables a single Embedding Net to achieve results comparable to those of N^2 networks.

For S_{ij}, i is the central atom. The physical properties of the element type corresponding to j are concatenated with S_{ij} to form a vector of length $1+$ the number of physical properties, which is then passed to the Embedding Net. Tests on our five-component alloy (Ru, Rh, Ir, Pd, and Ni) dataset and quaternary LiGePS dataset (1200 K) show that a DP model using this type-embedding method matches or exceeds the prediction accuracy of a standard DP model while reducing training time by 27%. See the performance benchmarks for details.

Usage

To enable type embedding with the default physical properties, add the $type_embedding$ parameter to the JSON training configuration. See **example/LiGePS/ligeps.json** in the project examples.

```
{
  "type_embedding": true
}
```

You can also specify the required physical properties in the model parameters of the JSON file.

5.3.3 Performance Benchmark: Accuracy

Comparison of validation-set prediction accuracy between the type-embedding method and a standard DP model on the mixed five-component alloy dataset (9,486 configurations):

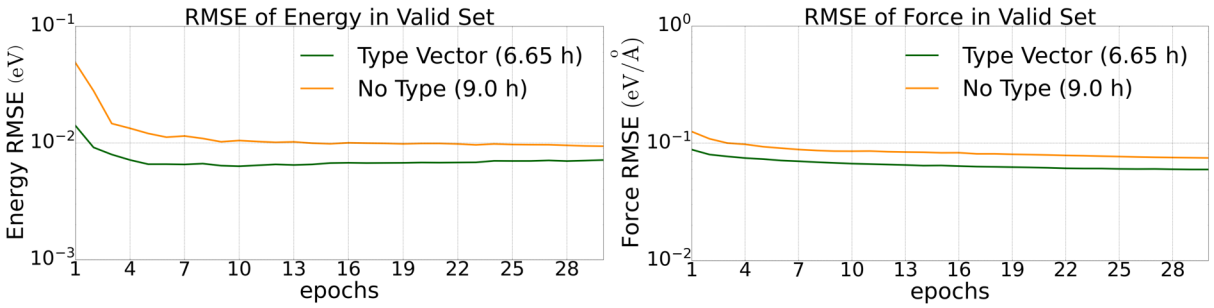


Figure 7: Reduction in energy and force errors on the validation set for the five-component alloy system

Comparison of validation-set prediction accuracy between the type-embedding method and a standard DP model on the quaternary LiGePS dataset (10,000 configurations at 1200 K):

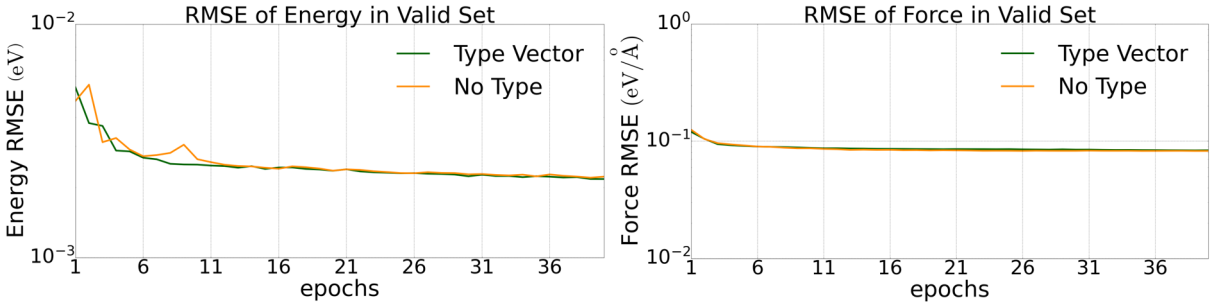


Figure 8: Reduction in energy and force errors on the validation set for the quaternary LiGePS system

5.3.4 Performance Benchmark: Training Time

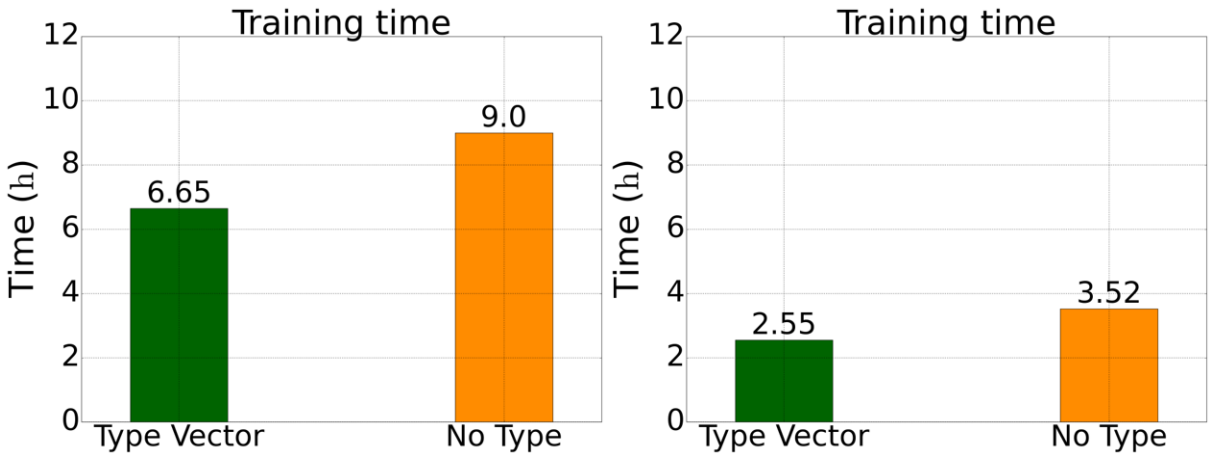


Figure 9: Training-time comparison for the alloy and LiGePS systems

The force field is invoked in LAMMPS in the same way as the standard DP model described above.

5.3.5 Polynomial Model Compression

A DP model contains N^2 Embedding Nets, where N is the number of atom types. As the number of types increases, the number of Embedding Nets grows rapidly, enlarging the computational graph used for backpropagation and becoming an inference bottleneck. The timing breakdown below for DP inference on a five-component alloy shows that Embedding Net evaluation and gradient computation account for more than 90% of the total time, leaving substantial room for optimization. An Embedding Net takes a scalar S_{ij} as input and outputs m values, where m is the number of neurons in its final layer. It can therefore be replaced by m scalar functions.

MatPL implements the fifth-order polynomial compression method described in Lu D, Jiang W, Chen Y, et al., *DP compress: A model compression scheme for generating efficient deep potential models*. It also provides a third-order polynomial method based on Hermite interpolation.

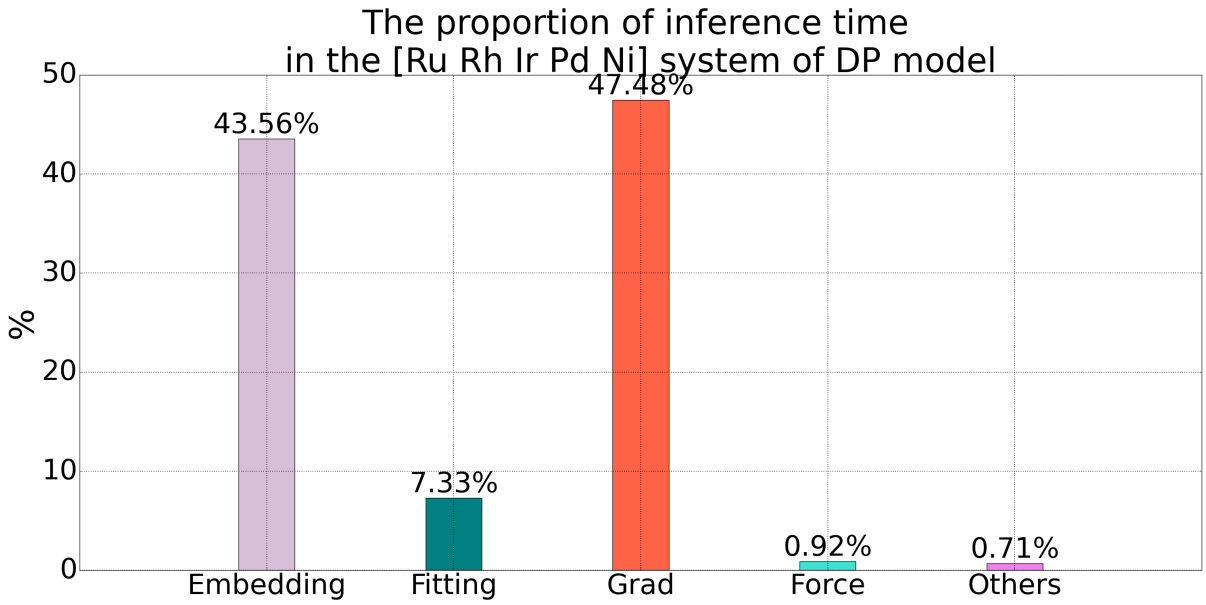


Figure 10: proportion_time

Usage

To compress a trained DP model, use the following command:

```
MatPL compress dp_model.ckpt -d 0.01 -o 3 -s cmp_dp_model
```

- `compress` is the compression command.
- `dp_model.ckpt` is the required model file to compress.
- `-d` sets the grid spacing for S_{ij}; the default is `0.01`.
- `-o` sets the compression order: `3` for third order or `5` for fifth order. The default is `3`.
- `-s` sets the compressed model name. The default is `cmp_dp_model`.

After compression, the model is used for molecular-dynamics simulations in LAMMPS in the same way as a standard DP model.

Model compression accuracy

We compressed DP models for bulk copper and a five-component alloy and evaluated them on their respective test sets. The results are shown below. For copper, second-order interpolation is included for comparison; unlike the third- and fifth-order methods, it does not provide sufficient accuracy.

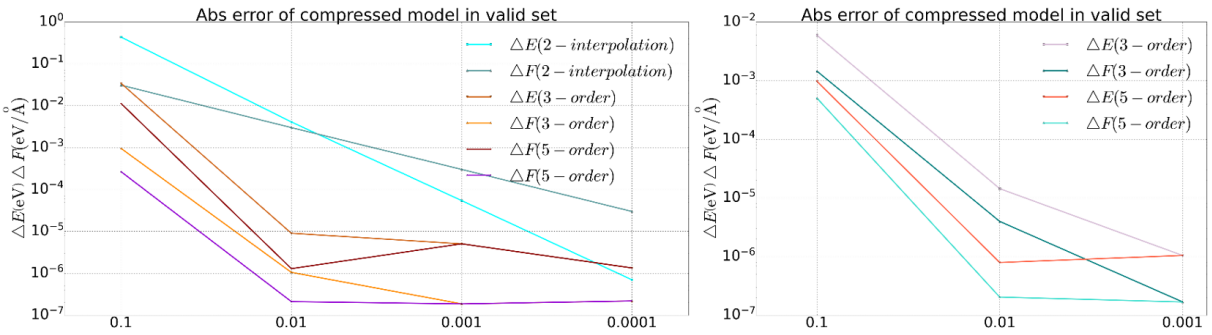


图1. Bulk铜体系DP模型二阶、三阶与五阶多项式压缩对比;图2. 五元合金体系DP模型三阶与五阶多项式压缩对比

Figure 11: Accuracy comparison for different model-compression orders

5.3.6 Polynomial Model Compression Procedure

5.3.6.1 Grid Construction We scan the entire training set to obtain the maximum value of s_{ij}. Because s_{ij} is a function of the three-dimensional distance r_{ij} between atoms i and j, it reaches its minimum when $r_{ij} = r_{\text{cut}}$. The range of s_{ij} is divided into L equal intervals of width dx, yielding $L + 1$ interpolation points denoted by $x_1, x_2, \cdots, x_{L+1}$. In practice, an incomplete training set may not cover every s_{ij} encountered during inference. The grid is therefore extended from the original upper limit to $10 \times s_{ij}$ using a spacing of $10 \times dx$.

5.3.6.2 Third-Order Polynomial For each interval $[x_l, x_{l+1})$, the Embedding Net is replaced by the following third-order polynomial:

$$g_m^l(x) = a_m^l x^3 + b_m^l x^2 + c_m^l x + d_m^l$$

Here, m is the number of neurons in the final layer of the Embedding Net—that is, the number of output values—and the polynomial variable x is $s_{ij} - x_l$. The following two conditions must hold at every grid point:

1. The polynomial value equals the Embedding Net output:

$$y_l = \mathcal{G}_m(x_l)$$

2. The first derivative of the polynomial equals the first derivative of the Embedding Net with respect to s_{ij}:

$$y_l' = \mathcal{G}_m'(x_l)$$

The resulting coefficients are

$$a_m^l = \frac{1}{\Delta t^3}\left[(y_{l+1}' + y_l')\Delta t - 2h\right]$$

$$b_m^l = \frac{1}{\Delta t^2}\left[-(y'_{l+1} + 2y'_l)\Delta t + 3h\right]$$

$$c_m^l = y'_l$$

$$d_m^l = y_l$$

where $h = y_{l+1} - y_l$ and $\Delta t = x_{l+1} - x_l$.

5.3.6.3 Fifth-Order Polynomial We also implement the fifth-order polynomial compression method from DP Compress.

For the fifth-order polynomial, s_{ij} is partitioned in the same way as for the third-order method, and the Embedding Net is replaced by the following polynomial:

$$g_m^l(x) = a_m^l x^5 + b_m^l x^4 + c_m^l x^3 + d_m^l x^2 + e_m^l x + f_m^l$$

Note that the polynomial variable x is $s_{ij} - x_l$. The following three conditions must hold at every grid point:

1. The polynomial value equals the Embedding Net output:

$$y_l = \mathcal{G}_m(x_l)$$

2. The first derivative of the polynomial equals the first derivative of the Embedding Net with respect to s_{ij}:

$$y'_l = \mathcal{G}'_m(x_l)$$

3. The second derivative of the polynomial equals the second derivative of the Embedding Net with respect to s_{ij}:

$$y''_l = \mathcal{G}''_m(x_l)$$

The six coefficients are therefore:

$$a_m^l = \frac{1}{2\Delta t^5}\left[12h - 6(y'_{l+1} + y'_l)\Delta t + (y''_{l+1} - y''_l)\Delta t^2\right]$$

$$b_m^l = \frac{1}{2\Delta t^4}\left[-30h + (14y'_{l+1} + 16y'_l)\Delta t + (-2y''_{l+1} + 3y''_l)\Delta t^2\right]$$

$$c_m^l = \frac{1}{2\Delta t^3}\left[20h - (8y'_{l+1} + 12y'_l)\Delta t + (y''_{l+1} - 3y''_l)\Delta t^2\right]$$

$$d_m^l = \frac{1}{2} y_l''$$

$$e_m^l = y_l'$$

$$f_m^l = y_l$$

where $h = y_{l+1} - y_l$ and $\Delta t = x_{l+1} - x_l$.

5.3.6.4 Verification of the Model Compression Formula The model-compression scheme divides the range of s_{ij} into L equal intervals, giving $L + 1$ interpolation points denoted by $x_1, x_2, \cdots, x_{L+1}$. For each interval $[x_l, x_{l+1})$, the embedding network is replaced by the following fifth-order polynomial:

$$g_m^l(x) = a_m^l x^5 + b_m^l x^4 + c_m^l x^3 + d_m^l x^2 + e_m^l x + f_m^l$$

Note that the polynomial variable x is $s_{ij} - x_l$. The following three boundary conditions must hold at every grid point:

1. The function values are equal:
$$y_l = \mathcal{G}_m(x_l)$$
2. The first derivatives are equal:
$$y_l' = \mathcal{G}_m'(x_l)$$
3. The second derivatives are equal:
$$y_l'' = \mathcal{G}_m''(x_l)$$

The six coefficients are therefore:

$$a_m^l = \frac{1}{2\Delta t^5} \left[12h - 6(y_{l+1}' + y_l')\Delta t + (y_{l+1}'' - y_l'')\Delta t^2\right]$$

$$b_m^l = \frac{1}{2\Delta t^4} \left[-30h + (14y_{l+1}' + 16y_l')\Delta t + (-2y_{l+1}'' + 3y_l'')\Delta t^2\right]$$

$$c_m^l = \frac{1}{2\Delta t^3} \left[20h - (8y_{l+1}' + 12y_l')\Delta t + (y_{l+1}'' - 3y_l'')\Delta t^2\right]$$

$$d_m^l = \frac{1}{2} y_l''$$

$$e_m^l = y_l'$$

$$f_m^l = y_l$$

where $h = y_{l+1} - y_l$ and $\Delta t = x_{l+1} - x_l$.

5.4 DP Tutorial

This tutorial uses the MatPL [source root/example/HfO2/dp_demo] example (source of the HfO2 training dataset) to demonstrate training, testing, LAMMPS simulation, and other operations for a DP model. The example directory is organized as follows.

```
HfO2/
├── atom.config
├── pwdata/
└── dp_demo/
    ├── dp_test.json
    ├── dp_train.json
    ├── train.job
    └── dp_lmps/
        ├── in.lammps
        ├── lmp.config
        ├── jit_dp.pt
        ├── runcpu.job
        └── rungpu.job
```

- pwdata is the training-data directory.
- dp_train.json is the input file for training the DP force field.
- dp_train.json is the input file for testing the DP force field.
- train.job is an example Slurm job script for training.
- The dp_lmps directory contains a LAMMPS MD example for the DP force field:
  - Force-field file: jit_dp.pt
  - Initial structure: lmp.config
  - Control file: in.lammps
  - Example Slurm scripts: runcpu.job and rungpu.job

5.4.1 train

Run the following command in the dp_demo directory to start training:

```
MatPL train dp_train.json
# Alternatively, update the environment variables and submit the
 ↪  training job through Slurm: sbatch train.job
```

Input file description

The contents of `dp_train.json` are shown below. For descriptions of the DP parameters, see the DP Parameter Reference:

```
{
    "model_type": "DP",
    "atom_type": [
        8, 72
    ],
    "format": "pwmlff/npy",
    "train_data": [
        "../pwdata/init_000_50/", "../pwdata/init_002_50/",
        "../pwdata/init_004_50/", "../pwdata/init_006_50/",
        "../pwdata/init_008_50/", "../pwdata/init_010_50/",
        "../pwdata/init_012_50/", "../pwdata/init_014_50/",
        "../pwdata/init_016_50/", "../pwdata/init_018_50/",
        "../pwdata/init_020_20/", "../pwdata/init_022_20/",
        "../pwdata/init_024_20/", "../pwdata/init_026_20/",
        "../pwdata/init_001_50/", "../pwdata/init_003_50/",
        "../pwdata/init_005_50/", "../pwdata/init_007_50/",
        "../pwdata/init_009_50/", "../pwdata/init_011_50/",
        "../pwdata/init_013_50/", "../pwdata/init_015_30/",
        "../pwdata/init_017_50/", "../pwdata/init_019_50/",
        "../pwdata/init_021_20/", "../pwdata/init_023_20/",
        "../pwdata/init_025_20/", "../pwdata/init_027_20/"
    ],
    "valid_data":[
        "../pwdata/init_000_50/", "../pwdata/init_004_50/",
        "../pwdata/init_008_50/"
    ]
}
```

For details about the force-field directory generated after training, see model_record details.

5.4.2 test

```
MatPL test dp_test.json
```

The contents of `test.json` are shown below. For parameter descriptions, see the Parameter Reference.

```
{
    "model_type": "DP",
    "format": "pwmlff/npy",
    "model_load_file": "./model_record/dp_model.ckpt",
    "test_data": [
        "../init_000_50", "../init_004_50", "../init_008_50",
        "../init_012_50", "../init_016_50", "../init_020_20",
        "../init_024_20", "../init_001_50", "../init_005_50",
        "../init_009_50", "../init_013_50", "../init_017_50",
        "../init_021_20", "../init_025_20", "../init_002_50",
        "../init_006_50", "../init_010_50", "../init_014_50",
        "../init_018_50", "../init_022_20", "../init_026_20",
        "../init_003_50", "../init_007_50", "../init_011_50",
        "../init_015_30", "../init_019_50", "../init_023_20",
        "../init_027_20"
    ]
}
```

For details about the force-field directory generated after testing, see test_result details.

5.4.3 infer a Single Structure

```
MatPL infer dp_model.ckpt atom.config pwmat/config
MatPL infer dp_model.ckpt 0.lammpstrj lammps/dump Hf O
# Hf and O are the element names in the lammps/dump structure: Hf is
 ↪  atom type 1 and O is atom type 2
```

After successful inference, the terminal displays the total energy, per-atom energies, per-atom forces, and virial.

5.4.4 compress a Model

To compress a trained DP force field, use the following command:

```
MatPL compress dp_model.ckpt -d 0.01 -o 3 -s cmp_dp_model
```

- `compress` is the compression command.
- `dp_model.ckpt` is the name of the model file to compress and is required.

- `-d` sets the grid spacing for `S_ij`; the default is `0.01`.
- `-o` sets the compression order: `3` selects third-order compression and `5` selects fifth-order compression. The default is `3`.
- `-s` sets the name of the compressed model. The default is `cmp_dp_model`.

After compression, a force-field file named `cmp_dp_model.ckpt` is created in the current directory.

5.4.5 script: Convert to an MD Force Field

This command converts `dp_model.ckpt` to the LibTorch format recognized by LAMMPS.

```
MatPL script dp_model.ckpt
# Or convert a compressed force field
MatPL script cmp_dp_model.ckpt
```

The conversion creates a `jit_dp.pt` file in the current directory for subsequent LAMMPS MD simulations.

5.4.6 lammps MD

Step 1. Prepare the force-field file

Before using the trained `dp_model.ckpt` file in a LAMMPS simulation, extract the deployable force field with the following command:

```
MatPL script dp_model.ckpt
```

After successful conversion, you will obtain the `jit_dp.pt` force-field file.

Step 2. Prepare the input control file

Configure the force field in the LAMMPS input control file as shown below. This example uses HfO2 (`Hf02/dp_demo/dp_lmps`).

```
pair_style   matpl/dp   jit_dp.pt
pair_coeff   * *     8 72
```

Here: - `pair_style` sets the path to the force-field file. `matpl/dp` is the required style name for a MatPL force field, and `jit_dp.pt` is the force-field path.

The interface also supports model-deviation output from multiple models, a feature commonly used in active learning. You can specify several models: the first is used for MD, while the others participate in the deviation calculation. In that case, configure `pair_style` as follows: `txt pair_style matpl/dp 0_jit_dp.pt 1_jit_dp.pt 2_jit_dp.pt 3_jit_dp.pt out_freq ${DUMP_FREQ} out_file model_devi.out pair_coeff * * 8 72`

- `pair_coeff` maps the atom types in the simulated structure to atomic numbers. For example, if type 1 is O and type 2 is Hf, use `pair_coeff * * 8 72`.

Step 3. Start the LAMMPS simulation

```
# Load the LAMMPS env.sh environment file; after installation, it is
 ↪  located in the LAMMPS source root
source /the/path/of/lammps/env.sh
# Run LAMMPS
mpirun -np N lmp -in in.lammps
```

Here, N is the number of CPU cores used for MD. If the system has M available GPUs, the N LAMMPS processes are distributed evenly across those GPUs. We recommend using the same number of CPU cores and GPUs, because multiple processes sharing one GPU compete for resources and reduce performance.

The LAMMPS interface also supports multi-node parallelism, including GPUs across nodes; specify the required numbers of nodes and GPUs in your job configuration.

5.4.7 ASE Interface

The DP model provides an ASE interface. See the example scripts on Gitee or GitHub.

```
from src.ase.calculate import MatPL_calculator
calc = MatPL(model_file='dp_model.ckpt')
atoms = ..... # Create ase.atoms.Atoms
atoms.calc = calc # Or use atoms.set_calculator(calc)
energy = atoms.get_potential_energy()
forces = atoms.get_forces()
stress = atoms.get_stress()
```

Before using the ASE interface, make sure the MatPL environment variables have been loaded.

5.5 NN Model

Tutorial

5.5.1 Neural Network (NN) Model Overview

The Neural Network (NN) model implements the following 8 feature types, all invariant under translation, rotation, and permutation:

1. 2-body(2b)
2. 3-body(3b)
3. 2-body Gaussian(2b gauss)
4. 3-body Cosine(3b cos)
5. Moment Tensor Potential(MTP)
6. Spectral Neighbor Analysis Potential(SNAP)
7. DP-Chebyshev(dp1)
8. DP-Gaussian(dp2)

Features (or descriptors) characterize the local atomic environment and must preserve translational, rotational, and permutational symmetry. They are commonly used as inputs to regressors such as linear models and neural networks, which predict atomic energies and forces. Because the features are differentiable functions of spatial coordinates, forces can be calculated as

$$\mathbf{F}_i = -\frac{dE_{\text{tot}}}{d\mathbf{R}_i} = -\sum_{j,\alpha} \frac{\partial E_j}{\partial G_{j,\alpha}} \frac{\partial G_{j,\alpha}}{\partial \mathbf{R}_i}$$

where j indexes neighboring atoms within the cutoff radius and α indexes the features.

5.5.2 2-b and 3-b features with piecewise cosine functions (feature 1 & 2)

For a given central atom, piecewise cosine functions are used to describe its local environment. The following diagram illustrates the basic idea.

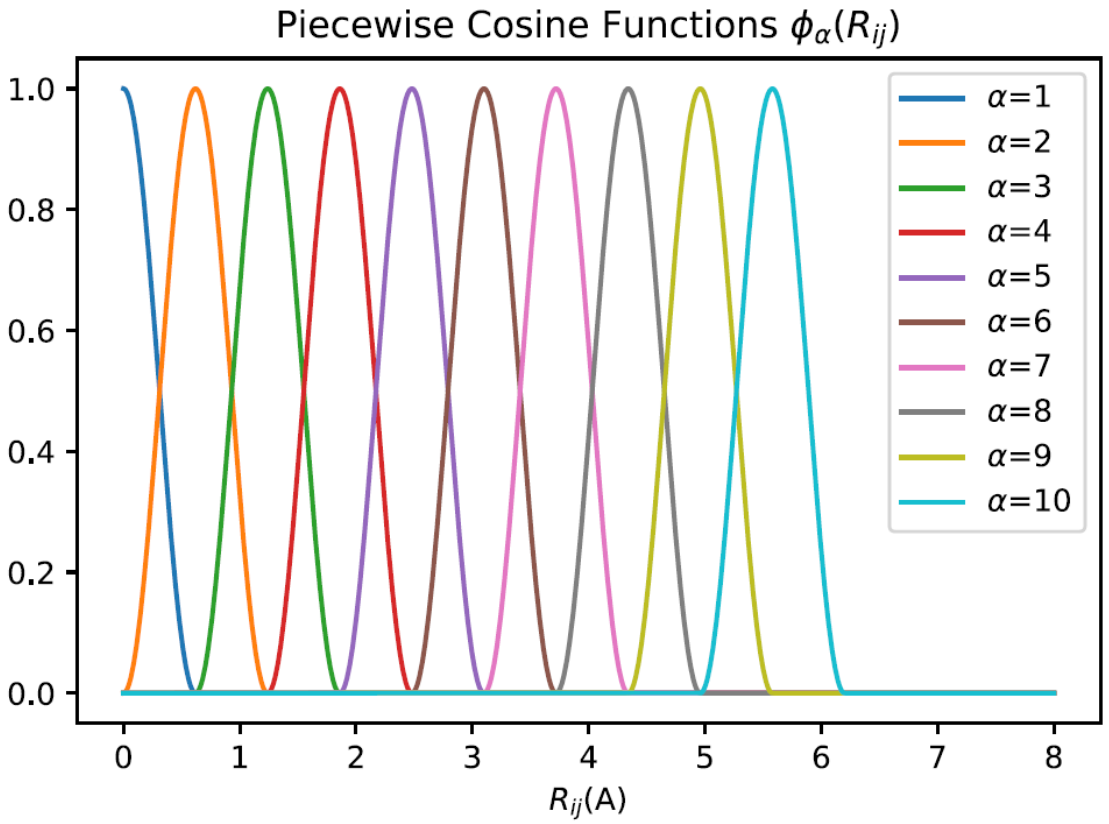


Figure 12: features

We first define piecewise cosine functions for the two-body and three-body features. Given inner and outer cutoffs R_{inner} and R_{outer}, basis order M, segment width h, and interatomic distance R_{ij} between central atom i and neighboring atom j, the basis function is

$$\phi_\alpha(R_{ij}) = \begin{cases} \frac{1}{2}\cos\left(\frac{R_{ij}-R_\alpha}{h}\pi\right)+\frac{1}{2} & ,|R_{ij}-R_\alpha|<h \\ 0 & ,\text{otherwise} \end{cases}$$

where

$$R_\alpha = R_{\text{inner}} + (\alpha-1)h,\ \alpha = 1,2,...,M$$

The **two-body feature** of central atom i is

$$G_{\alpha,i} = \sum_m \phi_\alpha(R_{ij})$$

and the **three-body feature** is

$$G_{\alpha\beta\gamma,i} = \sum_{j,k} \phi_\alpha(R_{ij})\phi_\beta(R_{ik})\phi_\gamma(R_{jk})$$

where $\sum_m$ and $\sum_{m,n}$ denote sums over all atoms within the cutoff radius R_{outer} of central atom i, respectively.

These two features are usually used together.

Reference:

Huang, Y., Kang, J., Goddard, W. A. & Wang, L.-W. Density functional theory based neural network force fields from energy decompositions. Phys. Rev. B 99, 064103 (2019)

5.5.3 2-b and 3-b Gaussian feature (feature 3 & 4)

These features were first used in the Behler–Parrinello neural network. Given cutoff radius R_c and interatomic distance R_{ij} between central atom i and neighboring atom j, the cutoff function f_c is defined as

$$f_c(R_{ij}) = \begin{cases} \frac{1}{2}\cos\left(\frac{\pi R_{ij}}{R_c}\right) + \frac{1}{2} & , R_{ij} < R_c \\ 0 & , \text{otherwise} \end{cases}$$

The **two-body Gaussian** feature of central atom i is defined as

$$G_i = \sum_{j \neq i} e^{(-\eta(R_{ij}-R_s)^2)} f_c(R_{ij})$$

where η and R_s are user-defined parameters.

The **three-body Gaussian** feature of central atom i is defined as

$$G_i = 2^{1-\zeta} \sum_{j,k \neq i} (1 + \lambda \cos\theta_{ijk})^{\zeta} e^{-\eta(R_{ij}^2 + R_{ik}^2 + R_{jk}^2)} f_c(R_{ij}) f_c(R_{ik}) f_c(R_{jk})$$

where

$$\cos\theta_{ijk} = \frac{\mathbf{R}_{ij} \cdot \mathbf{R}_{ik}}{|\mathbf{R}_{ij}||\mathbf{R}_{ik}|}$$

η, ζ, and $\lambda = \pm 1$ are user-defined parameters.

These two features are usually used together.

Reference:

J. Behler and M. Parrinello, Generalized Neural-Network Representation of High Dimensional Potential-Energy Surfaces. Phys. Rev. Lett. 98, 146401 (2007)

5.5.4 Moment Tensor Potential (feature 5)

In MTP, the local environment of central atom i is defined by

$$\mathbf{n}_i = (z_i, z_j, \mathbf{r}_{ij})$$

where z_i is the atom type of the central atom, z_j is the atom type of a neighbor, and $\mathbf{r}_{ij}$ is the relative coordinate of that neighbor. The energy contribution of each atom is then expanded as

$$E_i(\mathbf{n}_i) = \sum_{\alpha} c_\alpha B_\alpha(\mathbf{n}_i)$$

where B_α is a user-selected basis function and c_α is a parameter to be fitted.

To construct the basis functions, we introduce the moment tensor $M_{\mu\nu}$:

$$M_{\mu\nu}(\mathbf{n}_i) = \sum_j f_\mu(|\mathbf{r}_{ij}|, z_i, z_j) \bigotimes_\nu \mathbf{r}_{ij}$$

These moment tensors contain radial and angular components. The radial component can be expanded as

$$f_\mu(|\mathbf{r}_{ij}|, z_i, z_j) = \sum_\beta c^{(\beta)}_{\mu,z_i,z_j} Q^{(\beta)}(|\mathbf{r}_{ij}|)$$

where $Q^{(\beta)}(|\mathbf{r}_{ij}|)$ is a radial basis function. Specifically,

$$Q^{(\beta)}(|\mathbf{r}_{ij}|) = \begin{cases} \phi^{(\beta)}(|\mathbf{r}_{ij}|)(R_{\text{cut}} - |\mathbf{r}_{ij}|)^2 & , |\mathbf{r}_{ij}| < R_{\text{cut}} \\ 0 & , \text{otherwise} \end{cases}$$

where $\phi^{(\beta)}$ is a polynomial, such as a Chebyshev polynomial, defined on $[R_{\text{min}}, R_{\text{cut}}]$.

The angular component is given by $\bigotimes_\nu \mathbf{r}_{ij}$, the ν-fold tensor product of $\mathbf{r}_{ij}$, and contains the angular information of neighborhood $\mathbf{n}_i$. The value of ν determines the rank of the moment tensor: $\nu = 0$ gives a constant scalar, $\nu = 1$ a vector (rank-1 tensor), $\nu = 2$ a matrix (rank-2 tensor), and so on.

Finally, the level of a moment tensor is defined as

$$\text{lev}(M_{\mu\nu}) = 2 + 4\mu + \nu$$

This is an empirical formula.

Reference:

I.S. Novikov, etal, The MLIP package: moment tensor potential with MPI and active learning. Mach. Learn.: Sci. Technol, 2, 025002 (2021)

5.5.5 Spectral Neighbor Analysis Potential (feature 6)

SNAP does not use Gaussian basis functions, so it does not calculate distances or kernel functions between two atomic-environment maps. Instead, it first defines a local atomic environment and expands it in spherical harmonics—or on a 4D hypersphere using rotation matrices. A bispectrum is then used to enforce rotational invariance. In this sense, SNAP resembles MTP, but uses a specialized contraction of directional indices to achieve rotational invariance. It is commonly paired with linear regression.

First, the local environment around the neighbors of central atom i at $\mathbf{r}$ is defined as a sum of δ functions in three-dimensional space:

$$\rho(\mathbf{r}) = \delta(\mathbf{r}) + \sum_{\mathbf{r}_{ki}<R_C} f_C(\mathbf{r}_{ki})\omega_k\delta(\mathbf{r}-\mathbf{r}_{ki})$$

where $\mathbf{r}_{ki}$ is the position of the k-th neighbor of atom i, ω_k is its weight, and radial function $f_C(\mathbf{r}_{ki})$ ensures that each neighbor′ s contribution smoothly approaches zero near cutoff radius R_C:

$$f_C(\mathbf{r}) = 0.5\left[\cos\left(\frac{\pi r}{R_C}\right)+1\right]$$

The angular component of this local-environment function can be expanded in spherical harmonics defined for $l = 0, 1, 2, ...$ and $m = -l, -l+1, ..., l-1, l$. A radial distribution is normally represented by a set of radial basis functions. Here, however, radial information $\mathbf{r}$ is mapped to the 4D hyperspherical function $U^j_{mm'}(\theta_0, \theta, \phi)$, where all points (neighboring atoms) lie on a 3D sphere embedded in 4D space and orientation is represented by three angles:

$$\mathbf{r} \equiv \begin{pmatrix} x \\ y \\ z \end{pmatrix} \begin{matrix} \\ \rightarrow \\ \\ \end{matrix} \begin{matrix} \phi = \arctan(y/x) \\ \theta = \arccos(z/r) \\ \theta_0 = \frac{3}{4}\pi r/r_c \end{matrix}$$

The local-environment function can therefore be expanded in these 4D hyperspherical functions $U^j_{mm'}(\theta_0, \theta, \phi)$ with coefficients $u^j_{mm'}$:

$$\rho(\mathbf{r}) = \sum_{j=0,\frac{1}{2},1,\ldots}^{\infty} \sum_{m=-j,-j+1}^{j} \sum_{m'=-j,-j+1}^{j} u^j_{mm'} U^j_{mm'}(\theta_0, \theta, \phi)$$

Using the local-environment function above, $u^j_{mm'}$ is calculated as

$$u^j_{mm'} = U^j_{mm'}(0,0,0) + \sum_{\mathbf{r}_{ki}<R_C} f_C(\mathbf{r}_{ki}) \omega_k U^j_{mm'}(\theta_0(k), \theta(k), \phi(k))$$

where k indexes neighboring atoms and $\theta_0(k), \theta(k), \phi(k)$ are the three angles of the position vector of atom i' s k-th neighbor. Because of the indices m and m', $u^j_{mm'}$ is direction dependent. Three such terms can be contracted using

$$F(j_1, j_2, j) = \sum_{m_1, m_1'=-j_1}^{j} \sum_{m_2, m_2'=-j_2}^{j} \sum_{m, m'=-j}^{j} (u^j_{mm'})^* u^{j_1}_{m_1 m_1'} u^{j_2}_{m_2 m_2'} \times C^{jm}_{j_1 m_1 j_2 m_2} C^{jm'}_{j_1 m_1' j_2 m_2'}$$

Here, $C^{jm}_{j_1 m_1 j_2 m_2} C^{jm}_{j_1 m_1' j_2 m_2'}$ are Clebsch–Gordan coefficients, and the final scalar feature is $F(j_1, j_2, j)$. Different choices of j_1, j_2, j produce different features. These features have no radial-function index; instead, they contain three angular-momentum indices because radial-distance information has been transformed into the third angular coordinate on the 3D sphere.

5.5.6 DP-Chebyshev (feature 7)

This feature resembles the DP embedding network and uses Chebyshev polynomials as its basis.

First, $S(\mathbf{r}_{ij})$ is defined as a weighted inverse-distance function:

$$S(\mathbf{r}) = \frac{f_C(\mathbf{r})}{r}$$

$$f_C(\mathbf{r}) = \begin{cases} 1 & , r < R_{C_2} \\ \frac{1}{2}\cos\left(\pi \frac{r - R_{C_2}}{R_c - R_{C_2}}\right) + \frac{1}{2} & , R_{C_2} \le r < R_C \\ 0 & , r > R_C \end{cases}$$

Here, R_{C_2} is a smooth cutoff parameter that allows the components of $\mathbf{r}_i$ to approach zero smoothly at the boundary of the local region defined by R_C. This smoothing function is more elaborate than the one used previously. $S(\mathbf{r}_{ji})$ reduces the weight of atoms farther from

central atom i. We then define radial function $g_M(s)$ using Chebyshev polynomial C_M for the *Deep Potential Chebyshev feature*:

$$g_M(s) = C_M(2R_{\min}S - 1)$$

Here, $R_{\min}$ is the minimum input value of r.

To construct this feature, we first calculate a four-component vector:

$$T_M(k) = \sum_{\mathbf{r}_{ji}<R_C} \hat{X}_{ji}(k)S(\mathbf{r}_{ji})g_M(S(\mathbf{r}_{ji}))$$

Here, $k = 0, 1, 2, 3$ indexes the four components: the usual x, y, z components plus an S component:

$$\{x_{ji}, y_{ji}, z_{ji}\} \to \{S(\mathbf{r}_{ji}), \hat{x}_{ji}, \hat{y}_{ji}, \hat{z}_{ji}\}$$

Here, $\hat{x}_{ji} = \frac{x_{ji}}{r_{ji}}, \hat{y}_{ji} = \frac{y_{ji}}{r_{ji}}, \hat{z}_{ji} = \frac{z_{ji}}{r_{ji}}$ are the components of the unit vector along $\mathbf{r}_{ji}$.
The component indices of these 4D vectors can then be contracted to obtain a scalar feature:

$$F(M_1, M_2) = \sum_{k=0}^{3} T_{M_1}(k)T_{M_2}(k)$$

In addition to the Chebyshev order, M_1 also encodes the atom type. Therefore, if the maximum Chebyshev order is M, the number of features is $M \cdot n_{\text{type}} \cdot (M \cdot n_{\text{type}} + 1)/2$. Different values of M produce different feature sets.

5.5.7 DP-Gaussian (feature 8)

This feature is similar to DP-Chebyshev, but replaces the Chebyshev polynomials with Gaussian functions whose position and width parameters are specified by the user.

As in DP-Chebyshev, the 4D vector is constructed as follows:

$$T_M(k) = \sum_{\mathbf{r}_{ji}<R_C} \hat{X}_{ji}(k)g_M(\mathbf{r}_{ji})$$

$$\hat{X}(0) = S(\mathbf{r}'), \quad \hat{X}(1) = \frac{x}{r}, \quad \hat{X}(2) = \frac{y}{r}, \quad \hat{X}(3) = \frac{z}{r}$$

$$g_M(\mathbf{r}) = f_C(\mathbf{r}) \cdot \exp\left(-\frac{(r - r_M)}{\omega_M}\right)$$

$$f_C(\mathbf{r}) = \frac{1}{2}\cos\left(\frac{\pi r}{R_C}\right) + \frac{1}{2}$$

The contraction is

$$F(M_1, M_2) = \sum_{k=0}^{3} T_{M_1}(k) T_{M_2}(k)$$

In addition to the Gaussian-function index, M_1 also encodes the atom type. Therefore, if the maximum basis order is M, the number of features is $M \cdot n_{\text{type}} \cdot (M \cdot n_{\text{type}} + 1)/2$. Different values of M produce different feature sets.

5.6 NN Tutorial

This tutorial uses the MatPL [source root/example/Cu/nn_demo] example to demonstrate training, testing, LAMMPS simulation, and other operations for an NN model. The example directory is organized as follows.

```
Cu/nn_demo
    ├── nn_test.json
    ├── nn_train.json
    ├── train.job
    └── nn_lmps/
        ├── in.lammps
        ├── lmp.config
        ├── forcefield.ff
        └── runcpu.job
```

- `nn_train.json` is the input file for training the NN force field.
- `nn_test.json` is the input file for testing the NN force field.
- `train.job` is an example Slurm job script for training.
- The `nn_lmps` directory contains a LAMMPS MD example for the NN force field:
  - Force-field file: `forcefield.ff`
  - Initial structure: `lmp.config`
  - Control file: `in.lammps`
  - Example Slurm script: `runcpu.job`

5.6.1 train

Run the following command in the `nn_demo` directory to start training:

```
MatPL train nn_train.json
# Alternatively, update the environment variables and submit the
 ↪  training job through Slurm: sbatch train.job
```

Input file description The contents of `nn_train.json` are shown below. For descriptions of the NN parameters, see the NN Parameter Reference:

```
{
    "format":"pwmat/movement",
    "train_data":["0_300_MOVEMENT", "1_500_MOVEMENT"],
    "valid_data":["valid_movement"],
    "model_type": "NN",
    "atom_type":[29]
}
```

For details about the force-field directory generated after training, see model_record details.

5.6.2 test

```
MatPL test nn_test.json
```

The contents of `test.json` are shown below. For parameter descriptions, see the Parameter Reference.

```
{
    "format":"pwmat/movement",
    "test_data":["0_300_MOVEMENT", "1_500_MOVEMENT"],
    "model_type": "NN",
    "model_load_file":"./model_record/nn_model.ckpt"
}
```

For details about the force-field directory generated after testing, see test_result details.

5.6.3 extract_ff

Extract the `nn_model.ckpt` file from the `model_record` directory after training:

```
# Extract the NN force-field model
MatPL extract_ff nn_model.ckpt
```

After extraction, a `forcefield` directory is created with the structure shown below. The other files in this directory contain descriptor-related information.

```
forcefield
  ├──fread_dfeat/
```

```
├──input/
├──output/
└──forcefield.ff
```

5.6.4 lammps MD

For installation and usage of the LAMMPS interface for NN models, see lammps-fortran.

Use the trained `*.ff` force-field file for the LAMMPS simulation. Add the following settings to the LAMMPS input file:

```
pair_style      matpl
pair_coeff      * * 3 1 forcefield.ff 29
```

Here, `3` selects a force field generated by the NN model, `1` specifies that one force-field file is read, `forcefield.ff` is the file generated by MatPL, and `29` is the atomic number of copper.

5.7 LINEAR Model

Tutorial

5.7.1 Model Overview

The LINEAR model uses the same descriptors as the NN model. The difference is that the NN model fits the force field with a neural network, whereas the LINEAR model performs a linear fit.

5.8 LINEAR Tutorial

This tutorial uses the MatPL [source root/example/SiC] example to demonstrate training, testing, LAMMPS simulation, and other operations for a Linear model. The example directory is organized as follows.

```
SiC
  ├── 1_300_MOVEMENT
  ├── 2_300_MOVEMENT
  ├── atom.config
  ├── linear_test.json
  ├── linear_train.json
  ├── train.job
  ├── linear_lmps
```

```
└── MD
    └── MOVEMENT
```

- `linear_train.json` is the input file for training the Linear force field.
- `linear_test.json` is the input file for testing the Linear force field.
- `train.job` is an example Slurm job script for training.
- The `linear_lmps` directory contains a LAMMPS MD example for the Linear force field:
  - Force-field file: `forcefield.ff`
  - Initial structure: `lmp.config`
  - Control file: `in.lammps`
  - Example Slurm script: `runcpu.job`

5.8.1 train

Run the following command in the `SiC` directory to start training: Example **etot.input** input file:

```
MatPL train linear_train.json
# Alternatively, update the environment variables and submit the
 ↪  training job through Slurm: sbatch train.job
```

Input file description

The contents of `linear_train.json` are shown below. For descriptions of the Linear parameters, see the Parameter Reference:

```
{
    "train_data":["./1_300_MOVEMENT", "./2_300_MOVEMENT"],
    "valid_data":["./1_300_MOVEMENT"],
    "format":"pwmat/movement",
    "model_type": "Linear",
    "atom_type":[14, 6]
}
```

After the program runs, it creates a `forcefield` directory in the working directory:

```
forcefield
├── forcefield.ff
├── fread_dfeat
│   ├── energyL*
│   ├── forceL*
│   ├── linear*
│   ├── weight_feat.*
│   ├── energyL*
```

```
│   ├── ...
│   ├── feat*
│   ├── weight_feat.*
│   └── linear_fitB.ntype
├── input
│   └── *feature.in
└── (output)
    └── grid*   # Used for features 1 and 2
```

5.8.2 test

```
MatPL test linear_test.json
```

The contents of `test.json` are shown below. For parameter descriptions, see the Parameter Reference.

```
{
    "test_movement_file":["./MD/MOVEMENT"],
    "format": "pwmat/movement",
    "model_type": "Linear",
    "atom_type":[14, 6]
}
```

Note that the MOVEMENT file must be placed in a directory named MD; otherwise, it cannot be detected. After testing, the `test_result` force-field directory contains the following files:

5.8.3 lammps MD

For installation and usage of the LAMMPS interface for Linear models, see lammps-fortran.

Use the trained `*.ff` force-field file for the LAMMPS simulation. Add the following settings to the LAMMPS input file:

```
pair_style      matpl
pair_coeff      * * 1 1 forcefield.ff 14 6
```

The first 1 selects a force field generated by the Linear model, and the second 1 specifies that one force-field file is read. `forcefield.ff` is the force-field file generated by MatPL, while 14 and 6 are the atomic numbers of Si and C, respectively.

6 PWact Active Learning

6.1 PWact Platform Overview

Machine-learning force fields can predict material properties and reaction mechanisms faster and more accurately than conventional approaches; state-of-the-art deep-learning molecular dynamics can now simulate systems containing tens of billions of atoms. However, because machine-learning models are fundamentally interpolative, an MLFF may be inaccurate in regions of phase space not represented by its training set. Training data are normally generated with expensive first-principles calculations, making large *ab initio* datasets difficult to obtain. Creating sufficiently representative data without relying on excessive first-principles calculations is therefore essential for improving extrapolation. PWact (Active Learning Based on PWmat Machine Learning Force Fields) is an open-source, MatPL-based automated active-learning platform for efficient data sampling.

PWact consists of a primary workflow and a task dispatcher.

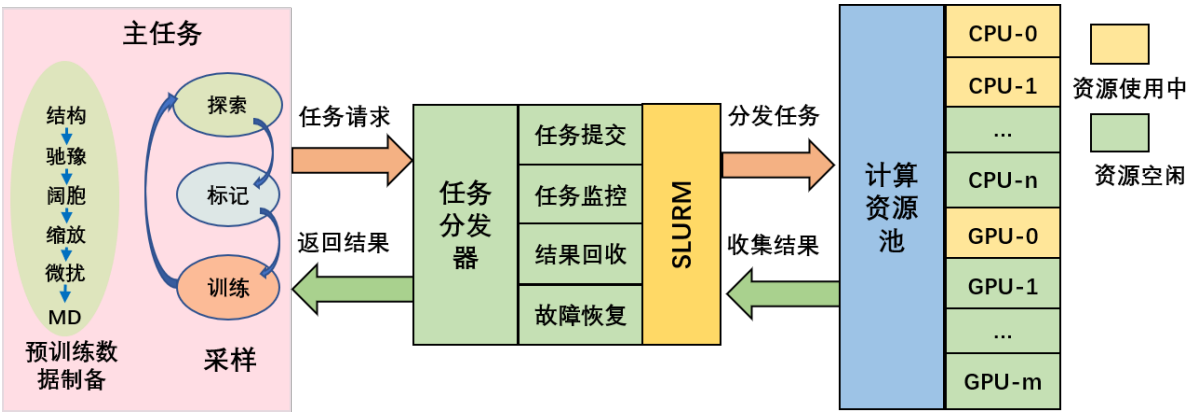


Figure 13: PWact platform architecture

The primary workflow contains `pretraining-data preparation` and `active learning`. It generates computational tasks and collects their results. When the dispatcher receives a scheduling request, it assigns the task to a compute node according to current resource usage and the resources requested. After execution, it collects the results and returns them to the primary workflow.

6.1.1 Pretraining-Data Preparation

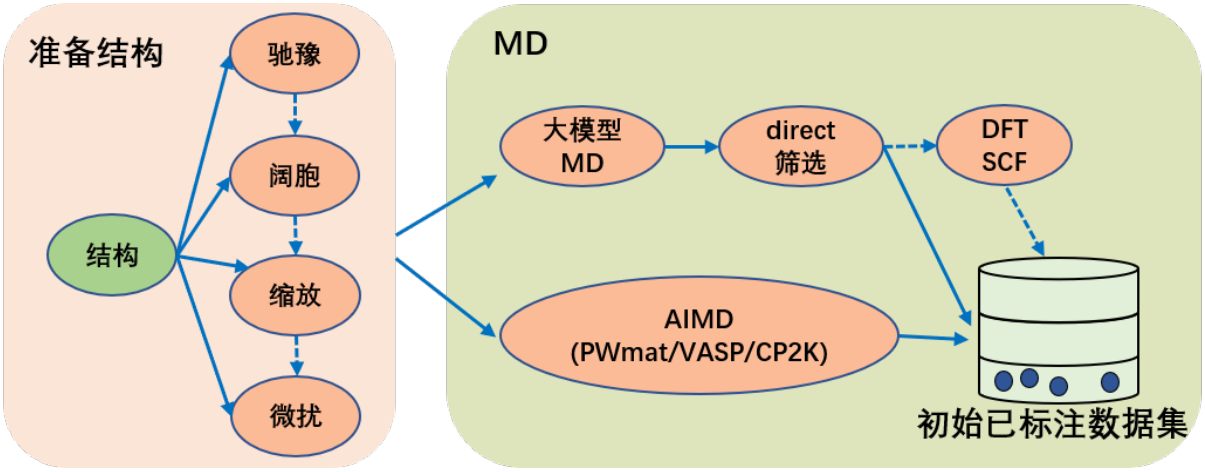


Figure 14: Pretraining-data preparation workflow

The preparation workflow supports relaxation (`PWmat`, `VASP`, and `CP2K`), supercell construction, lattice scaling, perturbation, and MD, and these operations can be combined. AIMD trajectories from PWmat, VASP, or CP2K can serve as pretraining data. PWact can also run MD with a foundation model and use DIRECT sampling to remove similar configurations. The filtered structures can then be labeled with PWmat, VASP, or CP2K through self-consistent calculations of energies and forces.

- DIRECT sampling: Qi J, Ko T W, Wood B C, et al. Robust training of machine learning interatomic potentials with dimensionality reduction and stratified sampling[J]. npj Computational Materials, 2024, 10(1): 43.

6.1.2 Active Learning

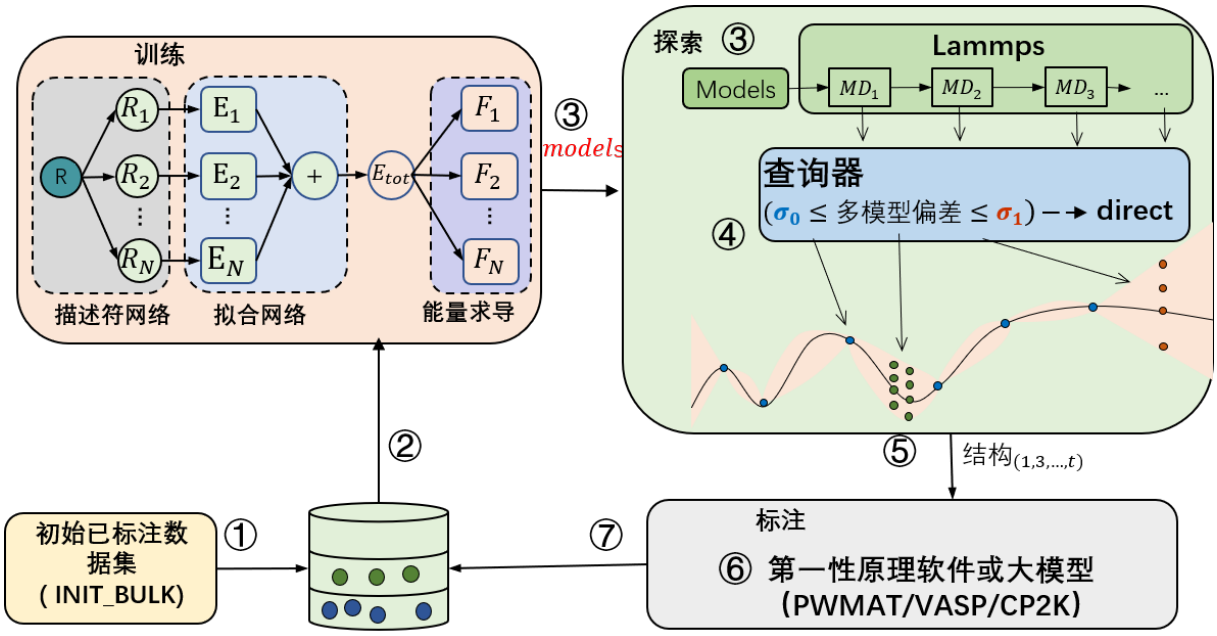


Figure 15: Active-learning workflow

The active-learning workflow contains `training`, `configuration exploration`, and `labeling`. First, the training module trains a model and passes it to exploration. Exploration runs molecular dynamics with the force field and sends the resulting trajectories to a query engine for uncertainty estimation. Selected configurations are passed to labeling, which performs self-consistent calculations to obtain energy and force labels and adds the labeled structures to the database. This cycle repeats until convergence.

- Model training supports MatPL DP, compressed DP, type-embedding DP, and NEP models.
- For uncertainty estimation, PWact provides the widely used multi-model Committee Query method and our single-model Kalman Prediction Uncertainty (KPU) method. KPU approaches Committee Query accuracy while reducing model-training cost to $1/N$, where N is the number of committee models. KPU currently applies only to DP models.
- **Optional:** Use DIRECT sampling to reduce the number of similar structures in the candidate set.
- Labeling supports PWmat, VASP, CP2K, or foundation-model inference.

6.1.3 Dependencies

- PWact uses Slurm for cluster and job scheduling, so Slurm must be installed on the compute cluster.
- DFT calculations support PWmat, VASP, and CP2K. Install at least one of them on the cluster.
- Force-field models are trained with MatPL. See the MatPL installation guide.
- LAMMPS molecular dynamics uses lammps-MatPL. See the MatPL installation guide.
- DIRECT sampling and foundation-model sampling are preinstalled on Lonxun Supercomputing Cloud (`Mcloud`), with support for DIRECT and the EquiformerV2 model. In other environments, install the required software and provide the processing scripts.

6.1.4 Installation

PWact can be installed with `pip` or from source.

6.1.4.1 Install with pip The package is available on PyPI and can be installed directly with `pip`.

```
pip install pwact

# Install PWact, or upgrade an existing installation
pip install pwact --upgrade

# List available versions
pip index versions pwact
# Example output:
# pwact (0.4.8)
# Available versions: 0.4.8, 0.4.7, 0.4.6, 0.4.5, 0.4.4, 0.4.2,
 ↪  0.3.4, 0.3.3, 0.2.4, 0.1.28, 0.1.10

# Install a specific version
pip install pwact==n.m.o
```

6.1.4.2 Install from GitHub Source Install from source only when you need to modify the code; otherwise, use `pip`. Download the source:

```
git clone https://github.com/LonxunQuantum/PWact.git
or
git clone https://gitee.com/pfsuo/pwact.git
Gitee may update later than GitHub, so GitHub is recommended
```

After downloading, enter the source root containing `setup.py` and run:

```
pip install .
# Or use editable mode so changes to the source take effect
 ↪  immediately
# pip install -e .
# After installing from source, add PWact to PYTHONPATH if needed
# export PYTHONPATH=/the/path/pwact:$PYTHONPATH
```

PWact is written in Python and supports Python 3.9 or later. We recommend using MatPL's Python environment.

To create a separate virtual environment for PWact, install the following packages in a Python 3.9-or-later environment:

```
pip install numpy pandas tqdm pwdata
```

6.1.5 Command List

PWact commands begin with `pwact`. Subcommands are case-insensitive, so `INIT_BULK`, `Init_BUlk`, and `init_bulk` are equivalent.

6.1.5.1 List Available Commands

```
pwact  [ -h / --help / help ]

# List options for a specific cmd_name
pwact cmd_name -h
```

6.1.5.2 Prepare an Initial Training Set

```
pwact init_bulk param.json resource.json
```

6.1.5.3 Active Learning

```
pwact run param.json resource.json
```

You may rename the JSON files used by these commands, but the argument order of `param.json` and `resource.json` must remain unchanged.

6.1.5.4 gather_pwdata Command This command searches the active-learning directory for all explored structures and saves them under `final_pwdata` in the configured `data_format`. `final_pwdata_list.txt` lists the data directory for each iteration.

```
final_pwdata/
final_pwdata_list.txt  iter.0000  iter.0001  iter.0002 ...
```

```
pwact gather_pwdata -i .
```

Here, `-i` specifies the active-learning root, corresponding to the `run_iter` directory in the examples.

6.1.5.5 kill Command

Stop running `init_bulk` tasks such as relaxation and AIMD:

```
# Enter the directory from which pwact init_bulk was started
pwact kill init_bulk
```

Stop running `run` tasks, including training, exploration (MD), and labeling:

```
# Enter the directory from which pwact run was started
pwact kill run
```

You can perform the same operation manually by first terminating the primary `pwact init_bulk` or `pwact run` process and then canceling the associated Slurm jobs.

Because manual termination may affect unrelated processes, the `kill` command is recommended.

After stopping the workflow, inspect the command output and use Slurm commands to check for remaining jobs.

6.1.5.6 filter Command

Test structure selection with specified lower and upper thresholds:

```
pwact filter -i iter.0000/explore/md -l 0.01 -u 0.02 -s
```

This command evaluates all trajectories under `iter.0000/explore/md` using a lower threshold of `0.01` and an upper threshold of `0.02`, as shown below. The optional `-s` flag saves detailed selection information.

```
Image select result (lower 0.01 upper 0.02):
 Total structures 972    accurate 20 rate 2.06%    selected 44 rate
 ↪  4.53%    error 908 rate 93.42%

Select by model deviation force:
Accurate configurations: 20, details in file accurate.csv
Candidate configurations: 44
        Select details in file candidate.csv
Error configurations: 908, details in file fail.csv
```

6.1.6 Input Files

PWact uses `param.json` and `resource.json` for initial-dataset preparation and active learning. JSON keys in both files are case-insensitive.

6.1.6.1 param.json Initial training-set preparation: init_param.json

Configures relaxation, supercell construction, scaling, perturbation, and AIMD (PWmat, VASP, or CP2K) for VASP- or PWmat-format structures.

Active learning: run_param.json

Configures training (network architecture and optimizer), exploration (LAMMPS and selection strategy), and labeling (VASP/PWmat self-consistent calculations).

6.1.6.2 resource.json resource.json

Configures cluster resources for training, molecular dynamics, and DFT calculations (SCF, relaxation, and AIMD), including compute nodes, CPUs, GPUs, and software such as LAMMPS, VASP, PWmat, and MatPL.

6.1.7 Active-Learning Examples

- Active learning for silicon
- Active learning for silicon (foundation model + DIRECT)
- Active learning for an Au–Ag alloy

6.2 init_bulk param.json Configuration

`init_bulk` prepares an initial training set by applying `relaxation`, `supercell construction`, `scaling`, `perturbation`, and `AIMD` to structures in formats such as VASP and PWmat. AIMD supports DFTB, PWmat, and VASP. The parameters are described below.

6.2.1 data_format

Sets the output data format after `init_bulk` completes. The default is extended XYZ (`extxyz`).

6.2.2 reserve_work

Whether to retain temporary working directories. The default is `false`.

6.2.3 interval

Sets the interval for extracting structures from trajectories. A value of N selects every N-th structure. The default is 1.

6.2.4 sys_config_prefix

Optional path prefix for initial configurations, used together with `sys_configs/config`. It may be absolute or relative to the current directory.

Example: with `"sys_config_prefix":"/data/structure"` and `"config":"atom.config"`, the resolved path is `/data/structure/atom.config`.

6.2.5 sys_configs

Configures structure paths, relaxation, supercell construction, lattice scaling, atomic-position perturbation, and AIMD. A complete example appears below.

6.2.5.1 config Sets the structure-file path. If `sys_config_prefix` is set, the prefix and `config` are joined; otherwise, the value of `config` is used directly.

6.2.5.2 format Sets the structure format. Supported values are VASP POSCAR (`"vasp/poscar"`) and PWmat `atom.config` (`"pwmat/config"`, the default).

6.2.5.3 relax Whether to relax the configuration. The default is `true`.

6.2.5.4 relax_input_idx Selects the relaxation control file from relax_input. The default is `0`, selecting the first file. For example, an index may select `relax_etot1.input` as the PWmat control file.

6.2.5.5 super_cell Optional supercell setting. If omitted, no supercell is created. Accepted list forms include `[1, 1, 2]`, `[[1,0,0],[0,2,0],[0,0,1]]`, and `[1,0,0,0,2,0,0,0,1]`.

6.2.5.6 scale Optional lattice-scaling list. For example, `[0.9, 0.95, 0.96, 0.97]` produces four structures by multiplying the lattice by each factor. If omitted, no scaling is applied.

6.2.5.7 perturb Sets the number of perturbed structures generated using `cell_pert_fraction` and `atom_pert_distance`. If omitted, no perturbation is applied.

6.2.5.8 cell_pert_fraction Perturbs each of the nine lattice components by a value sampled uniformly from `[-cell_pert_fraction, cell_pert_fraction]`. The default is `0.03`.

6.2.5.9 atom_pert_distance Perturbs atomic coordinates in ångströms. Each coordinate is shifted by a value sampled uniformly from `[-atom_pert_distance, atom_pert_distance]`. The default is `0.01`.

6.2.5.10 aimd Whether to run molecular dynamics on the structure. The default is `true`.

6.2.5.11 aimd_input_idx Selects the AIMD control file from aimd_input. The default is `0`, selecting the first file. For example, an index may select `aimd_etot.input` as the PWmat control file.

6.2.5.12 bigmodel Whether to run MD with a foundation model. The default is `false`.

6.2.5.13 bigmodel_input_idx Selects the foundation-model script from bigmodel_input.

6.2.5.14 direct Whether to select structures with DIRECT. The default is `false`. When `direct` is `true`, direct_input must specify the DIRECT script.

6.2.6 sys_config Example

```
"sys_config_prefix": "../../si_example/init_bulk",
"sys_configs": [{"config":"./structures/49.config",
                 "relax":true,
                 "_relax_input_idx":0,
                 "super_cell":[1, 1, 2],
                 "scale":[0.9,0.95],
                 "perturb":3,
                 "cell_pert_fraction":0.03,
                 "atom_pert_distance":0.01,
                 "aimd":true,
                 "_aimd_input_idx":0
```

```
                    },
                    {"config":"./structures/44_POSCAR",
                    "format":"vasp/poscar",
                    "relax":false,
                    "super_cell":[[1,0,0],[0, 2, 0],[0,0,1]],
                    "perturb":2,
                    "aimd":true,
                    "aimd_input_idx": 1
                    }
        ]
```

This example configures `49.config` in the default `pwmat/config` format and `44_POSCAR` in `vasp/poscar` format. - For `49.config`: Step 1 relaxes the structure with the first file in `relax_input`. Step 2 scales the relaxed lattice by 0.9 and 0.95; see lattice scaling. Step 3 applies lattice and coordinate perturbations to each scaled structure, generating three variants from each; see perturbation. Step 4 runs AIMD on the resulting six structures using the first file in `aimd_input`. The workflow produces six AIMD trajectories. - For `44_POSCAR`: Step 1 creates a supercell with `[[1,0,0],[0,2,0],[0,0,1]]`; see supercell construction. Step 2 perturbs the supercell. Step 3 runs AIMD on the two perturbed structures using the second file in `aimd_input`. The workflow produces two AIMD trajectories.

After `init_bulk` completes, trajectories are automatically extracted into the format specified by `data_format`.

6.2.7 dft_style

Sets the DFT software used for `relax` and `AIMD`. The default is `pwmat`; use `vasp` for VASP.

6.2.8 pseudo

List of PWmat or VASP pseudopotential files. Paths may be absolute or relative to the current directory.

6.2.9 gaussian_param

Gaussian-basis parameters for CP2K or PWmat. `basis_set_file` and `potential_file` specify basis-set and potential files. `atom_list`, `basis_set_list`, and `potential_list` map elements to basis sets and potentials. `kspacing` configures k-points in the same way as the PWmat kspacing setting.

```
"gaussian_param": {
    "basis_set_file":"./init_bulk/BASIS_MOLOPT_1",
    "potential_file":"./init_bulk/POTENTIAL_1",
    "atom_list":["Si"],
    "kspacing" :0.4,
    "basis_set_list":["SZV-MOLOPT-SR-GTH"],
    "potential_list":["GTH-PBE-q4"]
}
```

6.2.10 relax_input

Sets relaxation control files. Use a list when multiple files are provided. See the examples below.

6.2.11 aimd_input

Sets AIMD control files. Use a list for multiple files; a dictionary is also accepted for a single file.

6.2.11.1 input Sets the input-control-file path, either absolute or relative to the current directory.

6.2.11.2 kspacing Optional PWmat k-point setting. Use it only when `MP_N123` is absent from `etot.input`; configuring both causes an error.

If neither `MP_N123` nor `kspacing` is set, `kspacing` defaults to `0.5`.

Do not set `MP_N123` and `kspacing` simultaneously.

6.2.11.3 flag_symm Optional PWmat symmetry setting. The default is `0` for relaxation or SCF and 3 for AIMD.

6.2.12 bigmodel_input

Sets foundation-model scripts. Use a list for multiple scripts.

For interface configuration, see the si_direct_bigmodel example.

6.2.13 direct_input

Sets the DIRECT sampling script as a single file path.

For interface configuration, see the si_direct_bigmodel example.

6.2.14 Complete Example

```
{
    "reserve_work": true,
    "sys_configs":[{
        "config":"atom.config",
        "relax":true,
        "relax_input_idx":1,
        "super_cell":[1, 1, 2],
        "scale":[0.9,0.95],
        "perturb":20,
        "cell_pert_fraction":0.03,
        "atom_pert_distance":0.01,
        "aimd":true,
        "aimd_input_idx":0
    }],

"dft_style":"PWmat",
"pseudo" : ["/path/Si.SG15.PBE.UPF"],

"relax_input":[{
        "input":"relax_etot.input",
        "kspacing":0.5,
        "flag_symm":"0"
    },{
        "input":"relax_etot1.input",
        "kspacing":0.3,
        "flag_symm":"0"
    },{
        "input":"relax_etot2.input",
        "kspacing":0.4,
        "flag_symm":"0",
        "_flag":"One integer: 0 for relax or SCF, 3 for AIMD, and
         ↪  2 for magnetic systems"
    }],

"aimd_input":[{
        "input":"aimd_etot.input",
```

```
            "kspacing":0.5,
            "flag_symm":"3"
        },{
            "input":"aimd_etot1.input",
            "kspacing":0.5,
            "flag_symm":"3"
        }]
    }
```

The configuration above: - Relaxes `atom.config`. - Creates a `(1,1,2)` supercell. - Scales the supercell lattice by 0.9 and 0.95. - Perturbs atomic positions in each scaled structure, generating 20 structures from each. - Runs AIMD on the 40 perturbed structures. - Extracts the AIMD trajectories automatically into `PWdata` as pretraining data.

6.2.14.1 relax_input and aimd_input kspacing and flag_symm configure PWmat k-points when `MP_N123` is absent from `etot.input`. If `MP_N123` is already present, relax_input and `aimd_input` can be abbreviated as follows:

```
    "relax_input":[
        "relax_etot.input",
        "relax_etot1.input",
        "relax_etot2.input"
    ],

    "aimd_input":[
        "aimd_etot.input"
        "aimd_etot1.input"
    ]
```

If every structure in sys_configs uses the same relaxation or AIMD input, abbreviate the parameters further:

```
    "relax_input":"relax_etot.input",
    "aimd_input": "aimd_etot.input"
```

In this case, omit relax_input_idx and aimd_input_idx from sys_configs:

```
    "sys_configs":[{
        "config":"atom.config",
        "relax":true,
        "super_cell":[1, 1, 2],
        "scale":[0.9,0.95],
        "perturb":20,
        "cell_pert_fraction":0.03,
        "atom_pert_distance":0.01,
```

```
    "aimd":true
}]
```

6.2.14.2 PWmat Examples

Set PWmat k-points with MP_N123 in the control file:

```
"dft_style":"PWmat",
"relax_input":["relax-agau-etot.input", "relax_ag-etot.input",
 ↪  "relax_au-etot.input"],
"aimd_input": ["aimd_agau-etot.input",   "aimd_ag-etot.input",
 ↪  "aimd_au-etot.input" ],
"pseudo" : ["../Ag.SG15.PBE.UPF", "../Au.SG15.PBE.UPF"]
```

Or set PWmat k-points with kspacing:

```
"dft_style":"PWmat",
"relax_input":[{
        "input":"relax_etot0.input",
        "kspacing":0.5,
        "flag_symm":"0"
    },{
        "input":"relax_etot1.input",
        "kspacing":0.3,
        "flag_symm":"0"
    },{
        "input":"relax_etot2.input",
        "kspacing":0.4,
        "flag_symm":"0",
        "_flag":"One integer: 0 for relax or SCF, 3 for AIMD, and
         ↪  2 for magnetic systems"
    }],

"aimd_input":[{
        "input":"aimd_etot0.input",
        "kspacing":0.5,
        "flag_symm":"3"
    },{
        "input":"aimd_etot1.input",
        "kspacing":0.5,
        "flag_symm":"3"
    }],
"pseudo" : ["../Ag.SG15.PBE.UPF", "../Au.SG15.PBE.UPF"]
```

PWmat Gaussian-basis example:

```
"dft_style":"PWmat",
```

```
"relax_input":["relax_etot.input",
 ↪  "relax_etot1.input","relax_etot2.input"],
"aimd_input": ["aimd_etot1.input", "aimd_etot2.input"],
"gaussian_param": {
    "basis_set_file":"./BASIS_MOLOPT_1",
    "potential_file":"./POTENTIAL_1",
    "atom_list":["Si"],
    "basis_set_list":["SZV-MOLOPT-SR-GTH"],
    "potential_list":["GTH-PBE-q4"]
}
```

6.2.14.3 VASP Example

```
"dft_style":"vasp",
"relax_input":["INCAR_relax_AgAu", "INCAR_relax_Ag",
 ↪  "INCAR_relax_Au"],
"aimd_input": ["INCAR_md_AgAu",   "INCAR_md_Ag", "INCAR_md_Au" ],
"pseudo" : ["../Ag_POTCAR", "../Au_POTCAR"]
```

6.2.14.4 CP2K Example

```
"dft_style":"cp2k",
"gaussian_param": {
    "basis_set_file":"../BASIS_MOLOPT_1",
    "potential_file":"../POTENTIAL_1",
    "atom_list":["Ag", "Au"],
    "basis_set_list":["SZV-MOLOPT-SR-GTH-q11",
     ↪  "SZV-MOLOPT-SR-GTH-q11"],
    "potential_list":["GTH-PBE", "GTH-PBE"],
    "kspacing":0.5
},
"aimd_input":["aimd_cp2k_AgAu.inp", "aimd_cp2k_Ag.inp",
 ↪  "aimd_cp2k_Au.inp"],
"relax_input":["relax_cp2k_AgAu.inp", "relax_cp2k_Ag.inp",
 ↪  "relax_cp2k_Au.inp"]
```

6.3 run param.json Configuration

The active-learning workflow includes training settings (network and optimizer), exploration settings (LAMMPS and selection strategy), and labeling settings (VASP/PWmat self-consistent calculations). The parameters are described below.

6.3.1 reserve_work

Whether to retain temporary working directories. The default is `false`, so they are deleted after each iteration.

6.3.2 reserve_md_traj

Whether to retain MD trajectories. The default is `false`, so trajectories are deleted after each iteration.

6.3.3 reserve_scf_files

Whether to retain all SCF outputs. The default is `false`. After each iteration, PWmat calculations retain only `REPORT`, `etot.input`, `OUT.MLMD`, and `atom.config`; VASP calculations retain only `OUTCAR`, `POSCAR`, and `INCAR`.

6.3.4 data_format

Sets the format of initial and collected datasets. The default is extended XYZ (`extxyz`).

6.3.5 init_data

List of initial training-set directories, as absolute paths or paths relative to the current directory.

6.3.6 valid_data

List of validation-set directories. Paths may be absolute or relative. If omitted, no validation results are generated during active-learning training.

6.3.7 init_model_list

Sets initial exploration models. To begin exploration from existing MatPL DP or NEP force fields, list their paths in `init_model_list`.

The number of force fields must match strategy/model_num, and their model type must match `train/model_type`. Training parameters are extracted from the force fields, so network and descriptor settings in `train_input_file` or the `train` dictionary are ignored.

6.3.8 use_pre_model

Whether the current exploration step uses force fields trained in the previous step. The default is `true`.

6.3.9 train

Model-training settings for the network and optimizer. See the `MatPL Training Parameters`. Configure them inline as below or specify a separate JSON file with `train_input_file`.

6.3.9.1 train_input_file Optional path to a standalone MatPL input file. If omitted, configure the parameters inline. See the MatPL Parameter Reference.

6.3.9.2 train Example

```
    "train": {
        "model_type": "DP",
        "atom_type": [
            14
        ],
        "seed": 2023,
        "model": {
            "descriptor": {
                "Rmax": 6.0,
                "Rmin": 0.5,
                "M2": 16,
                "network_size": [25, 25, 25]
            },
            "fitting_net": {
                "network_size": [50, 50, 50, 1]
            }
        },
        "optimizer": {
            "optimizer": "ADAM",
            "epochs": 30,
            "batch_size": 1,
            "print_freq": 10,
            "train_energy": true,
            "train_force": true,
            "train_virial": false
        }
```

```
    }
```

You can also place the training settings in a separate file such as `nep.json`:

```
"train": {
    "model_type": "NEP",
    "atom_type": [14],
    "train_input_file":"nep.json"
}
```

Because the MatPL defaults cover most training needs, the configuration can be abbreviated as follows to train a standard NEP model with ADAM.

```
"train": {
      "model_type": "NEP",
      "atom_type": [14]
}
```

PWact supports both MatPL DP and NEP force fields.

6.3.10 strategy

Configures uncertainty estimation and optional model compression.

6.3.10.1 uncertainty Sets the uncertainty strategy. The multi-model Committee Query method (`committee`) is supported.

6.3.10.2 lmps_tolerance Whether to continue active learning when some LAMMPS trajectories fail because of lost atoms, excessively short distances, or similar problems. The default is `true`, meaning the workflow continues.

6.3.10.3 lower_model_devi_f Sets the lower deviation threshold. Structures below it are considered accurate and are not labeled. The default is `0.05`.

6.3.10.4 upper_model_devi_f Sets the upper deviation threshold. Structures above it are considered unphysical and are not labeled. The default is `0.15`.

The maximum deviation is calculated as

$$\varepsilon_t = \max_i \left(\sqrt{\frac{\sum_1^w \left\| F_{w,i}(R_t) - \hat{F}_i \right\|^2}{W}} \right), \quad \hat{F}_i = \frac{\sum_1^W F_{w,i}}{W}$$

where W is the number of models and i indexes atoms.

6.3.10.5 model_num Sets the number of committee models. The default is 4, and the value must be at least 3.

6.3.10.6 max_select Sets the maximum number of configurations labeled for each initial exploration structure when select_sys is not set. If more candidates are available, max_select are selected randomly. By default, no limit is applied.

For an MD setting with two structures in sys_idx and no select_sys, each structure contributes at most max_select candidates, for a total of $2 \times max_select$.

6.3.10.7 direct Whether to apply DIRECT sampling to the structures selected by model deviation and remove similar configurations. The default is false.

6.3.10.8 direct_script Sets the DIRECT processing script. It is required when direct is true.

For interface setup, see the si_direct_bigmodel example.

6.3.10.9 compress Whether to compress the model. Compression may slightly reduce accuracy but can approximately double simulation speed. The default is false.

6.3.10.10 compress_order Sets the polynomial compression order. The default is 3; 5 provides higher accuracy but is slightly slower.

6.3.10.11 Compress_dx Sets the compression-grid spacing. The default is 0.01.

6.3.10.12 Example Committee Query selection:

```
"strategy": {
    "uncertainty":"committee",
    "lower_model_devi_f": 0.1,
    "upper_model_devi_f": 0.2,
    "model_num": 4,
    "max_select": 50,
    "compress": false
}
```

To enable model compression:

```
    "strategy": {
        "uncertainty":"committee",
        "lower_model_devi_f": 0.1,
        "upper_model_devi_f": 0.2,
        "model_num": 4,
        "max_select": 50,
        "compress": true,
        "compress_order":3,
        "Compress_dx":0.01
    }
```

6.3.11 explore

Configures molecular dynamics during exploration in each iteration.

6.3.11.1 sys_config_prefix Optional prefix for exploration-structure paths, used with `sys_configs`. It may be absolute or relative to the current directory.

6.3.11.2 sys_configs Sets exploration-structure paths. When `sys_config_prefix` is set, it is joined with each path; otherwise, the values are used directly.

This is a list. PWmat structures can be listed by path; VASP structures also require a `format`, as shown below.

```
        "sys_config_prefix": "../../structures",
        "sys_configs": [
                        {"config":"POSCAR", "format":"vasp/poscar"},
                        "49.config",
                        "45.config",
                        "41.config",
                        "37.config",
                        "33.config",
                        "29.config",
                        "25.config",
                        "21.config",
                        "17.config",
                        "1.config"
            ]
```

For example, the resolved path of `49.config` is `../../structures/49.config`.

6.3.11.3 lmps_prefix Optional path prefix for LAMMPS exploration input files, used with `lmps_in`. It may be absolute or relative.

Example: with `"lmps_prefix":"/data/in_lmps_files"` and `"lmps_in":"in0.lmps"`, the resolved path is `/data/in_lmps_files/in0.lmps`.

6.3.11.4 lmps_in Optional LAMMPS exploration input-file paths. When `lmps_prefix` is set, it is joined with each path; otherwise, `lmps_in` values are used directly.

This parameter is a list, as shown below.

```
"lmps_prefix": "../../in_lmps_files",
"lmps_in": [
                "in0.lmps",
                "in1.lmps",
                "in2.lmps",
                "in3.lmps",
                "in4.lmps",
                "in5.lmps"
    ]
```

6.3.11.5 md_jobs List of MD settings for each active-learning iteration. Entry `i` configures iteration `i` and may be a dictionary or a list of dictionaries. All time units below assume LAMMPS `units metal`.

PWact supports two forms of LAMMPS control. The first uses `param.json` fields for the step count, temperature, pressure, and ensemble; see the example. The second uses a user-provided `lammps.in` file through `lmps_prefix` and `lmps_in`; see the Au–Ag example.

`lmps_in` is supported in `pwact >= 0.4`.

When a user-provided `lammps.in` runs MD with Kokkos, it must include:

```
package kokkos neigh half comm device
newton on
```

At runtime, PWact manages the following fields in a user-provided `lammps.in`:

- `dump_freq`, controlled by trj_freq, sets the sampling interval.
- `units`, `boundary`, and `atom_style` are fixed because MatPL supports only periodic systems:

```
units           metal
boundary        p p p
atom_style      atomic
```

- `restart` is automatically set to save state every 10,000 steps.
- `read_data` is automatically updated to the required initial-structure path.
- `mass`, `pair_style`, and `pair_coeff` configure the machine-learning force field. For silicon, PWact generates:

```
mass    1    28.086
pair_style   matpl/dp  0_torch_script_module.pt
↪  1_torch_script_module.pt 2_torch_script_module.pt
↪  3_torch_script_module.pt  out_freq ${DUMP_FREQ}  out_file
↪  model_devi.out
pair_coeff   * * 14
```

- dump configures trajectory output. PWact inserts the following before the first `run` command, with DUMP_FREQ taken from trj_freq:

```
dump 1 all custom ${DUMP_FREQ} traj/*.lammpstrj id type x y z
↪  fx fy fz
```

- For `dump_modify`, PWact replaces the user-provided dump ID with 1 to match dump, leaving all other options unchanged.

The following `lmp.in` is an example input for silicon:

```
variable        NSTEPS          equal 400
variable        THERMO_FREQ     equal 5
variable        DUMP_FREQ       equal 5
variable        restart         equal 0
variable        TEMP            equal 500.000000
variable        PRESS           equal 100.000000
variable        TAU_T           equal 0.100000
variable        TAU_P           equal 0.500000

units           metal
boundary        p p p
atom_style      atomic

neighbor        1.0 bin
neigh_modify    delay 10

box              tilt large
if "${restart} > 0" then "read_restart lmps.restart.*" else
 ↪  "read_data lmp.config"
change_box       all triclinic

thermo_style    custom step temp pe ke etotal press vol lx ly lz xy xz
 ↪  yz
```

```
thermo          ${THERMO_FREQ}
restart         10000 lmps.restart

if "${restart} == 0" then "velocity        all create ${TEMP} 76752"
fix             1 all npt temp ${TEMP} ${TEMP} ${TAU_T} iso ${PRESS}
 ↪  ${PRESS} ${TAU_P}

timestep        0.001000
run             ${NSTEPS} upto
```

At runtime, it is transformed into:

```
variable        DUMP_FREQ       equal 5
variable        restart         equal 0
units           metal
boundary        p p p
atom_style      atomic

if "${restart} > 0" then "read_restart lmps.restart.*" else
 ↪  "read_data lmp.config"

mass   1    28.086
pair_style   matpl/dp  0_torch_script_module.pt
 ↪  1_torch_script_module.pt 2_torch_script_module.pt
 ↪  3_torch_script_module.pt  out_freq ${DUMP_FREQ} out_file
 ↪  model_devi.out
pair_coeff       * * 14

variable        NSTEPS          equal 400
variable        THERMO_FREQ     equal 5
variable        TEMP            equal 500.000000
variable        PRESS           equal 100.000000
variable        TAU_T           equal 0.100000
variable        TAU_P           equal 0.500000

neighbor        1.0 bin
neigh_modify    delay 10

box              tilt large
change_box       all triclinic


thermo_style    custom step temp pe ke etotal press vol lx ly lz xy xz
 ↪  yz
```

```
thermo          ${THERMO_FREQ}

fix             1 all npt temp ${TEMP} ${TEMP} ${TAU_T} iso ${PRESS}
 ↪  ${PRESS} ${TAU_P}

timestep        0.001000
dump            1 all custom ${DUMP_FREQ} traj/*.lammpstrj id type x y
 ↪  z fx fy fz
restart         10000 lmps.restart

if "${restart} == 0" then "velocity        all create ${TEMP} 75740"
run             ${NSTEPS} upto
```

6.3.11.5.1 sys_idx List of indices in `sys_configs` identifying initial MD structures. Multiple structures may be selected.

6.3.11.5.2 select_sys With `sys_idx`, limits the number of labeled configurations from each initial structure. If omitted, `max_select` applies; if neither is set, the default is `100`. For example:

```
  "sys_idx": [0, 1],
  "select_sys":[20, 30],
```

Here, `sys_idx` selects structure 0 (POSCAR) and structure 1 (`49.config`). At most 20 structures are labeled from the first trajectory set and 30 from the second. Without `select_sys`, each is limited by `max_select`.

`"select_sys":20` is equivalent to `"select_sys":[20, 20]`.

6.3.11.5.3 trj_freq Sets the trajectory sampling frequency (`thermo`). The default is every `10` steps.

6.3.11.5.4 lmps_in_idx With `lmps_in`, assigns LAMMPS input files to the initial structures in `sys_idx`, as shown below.

```
        "md_jobs": [
            [{
                "sys_idx": [ 1,3,4],
                "select_sys":[10,15,20],
                "lmps_in_idx":[0, 1, 2],
                "trj_freq": 5
            },{
```

```
            "sys_idx": [0, 1],
            "lmps_in_idx":3
        }
        ]]
```

Here, `"sys_idx": [1,3,4]` selects `49.config`, `41.config`, and `37.config` from the previous example, assigns `in0.lmps`, `in1.lmps`, and `in2.lmps`, and samples every five steps.

For `"sys_idx": [0,1]`, `in3.lmps` controls both selected structures and the default ten-step sampling interval is used.

When `lmps_in_idx` is present, `ensemble`, `nsteps`, `md_dt`, `press`, `taup`, `temps`, and `taut` are ignored.

6.3.11.5.5 ensemble For keyword-based LAMMPS control, sets the ensemble. The default is `nve`; supported values are:

`npt`, `npt-i`, or `npt-iso`:

```
fix  1 all npt temp ${TEMP} ${TEMP} ${TAU_T} iso ${PRESS} ${PRESS}
 ↪  ${TAU_P}
```

`npt-a` or `npt-aniso`:

```
fix  1 all npt temp ${TEMP} ${TEMP} ${TAU_T} aniso ${PRESS} ${PRESS}
 ↪  ${TAU_P}
```

`npt-t` or `npt-tri`:

```
fix  1 all npt temp ${TEMP} ${TEMP} ${TAU_T} tri ${PRESS} ${PRESS}
 ↪  ${TAU_P}
```

`nvt`:

```
fix  1 all nvt temp ${TEMP} ${TEMP} ${TAU_T}
```

`nve`:

```
fix  1 all nve
```

6.3.11.5.6 nsteps For keyword-based control, sets the required total MD step count. ##### md_dt

For keyword-based control, sets `timestep`. The default is `0.001`, or 1 fs.

6.3.11.5.7 press For keyword-based control, sets exploration pressures as a list.

6.3.11.5.8 taup For keyword-based control, sets the barostat coupling time in ps. The default is `0.5`.

6.3.11.5.9 temps For keyword-based control, sets exploration temperatures as a list.

6.3.11.5.10 taut For keyword-based control, sets the thermostat coupling time in ps. The default is `0.1`.

6.3.11.5.11 boundary For keyword-based control, sets periodic boundaries. MatPL supports only periodic simulations, so this is fixed to `true` (p p p) and need not be configured.

6.3.11.5.12 Example

```
"explore": {
    "sys_config_prefix": "./init_bulk/collection/init_config_0",
    "sys_configs": [
        {"config":"0.95_scale.poscar", "format":"vasp/poscar"},
        {"config":"0_pertub.poscar", "format":"vasp/poscar"},
        {"config":"0_pertub.poscar", "format":"vasp/poscar"}
    ],
    "md_jobs": [
        [{
            "ensemble": "npt",
            "nsteps": 1000,
            "md_dt": 0.002,
            "trj_freq": 10,
            "sys_idx": [0, 1],
            "temps": [500, 800],
            "taut":0.1,
            "press" :[ 100,200],
            "taup": 0.5,
            "boundary":true
            },{
            "ensemble": "nvt",
            "nsteps": 1000,
            "md_dt": 0.002,
            "trj_freq": 10,
            "sys_idx": [2],
            "temps": [400],
            "taut":0.1,
```

```
            "boundary":true
        }]
    ]
}
```

The example configures one active-learning iteration with two LAMMPS simulation dictionaries in `md_jobs`.

The first dictionary uses NPT for two structures, `0.95_scale.poscar` and `0_pertub.poscar`, at every combination of temperatures `[500,800]` and pressures `[100,200]`. Each run has 1,000 steps, output every 10 steps, and a 2 fs timestep, producing eight trajectories.

The second dictionary uses NVT for `0_pertub.poscar` at 400 K and produces one trajectory.

The following `lammps.in` is generated for `"ensemble":"npt"`, `"nsteps":1000`, `"md_dt":0.002`, `"trj_freq":10`, `"sys_idx":0`, `"temps":500`, `"taut":0.1`, `"press":100`, `"taup":0.5`, and `"boundary":true`:

```
variable        NSTEPS          equal 400
variable        THERMO_FREQ     equal 5
variable        DUMP_FREQ       equal 5
variable        restart         equal 0
variable        TEMP            equal 500.000000
variable        PRESS           equal 100.000000
variable        TAU_T           equal 0.100000
variable        TAU_P           equal 0.500000

units           metal
boundary        p p p
atom_style      atomic

neighbor        1.0 bin
neigh_modify    delay 10

box              tilt large
if "${restart} > 0" then "read_restart lmps.restart.*" else
 ↪  "read_data lmp.config"
change_box       all triclinic

mass   1    28.086
pair_style   matpl/dp  0_torch_script_module.pt
 ↪  1_torch_script_module.pt 2_torch_script_module.pt
 ↪  3_torch_script_module.pt  out_freq ${DUMP_FREQ} out_file
 ↪  model_devi.out
pair_coeff       * * 14
```

```
thermo_style    custom step temp pe ke etotal press vol lx ly lz xy xz
 ↪  yz
thermo          ${THERMO_FREQ}
dump            1 all custom ${DUMP_FREQ} traj/*.lammpstrj id type x y
 ↪  z fx fy fz
restart         10000 lmps.restart

if "${restart} == 0" then "velocity        all create ${TEMP} 76752"
fix             1 all npt temp ${TEMP} ${TEMP} ${TAU_T} iso ${PRESS}
 ↪  ${PRESS} ${TAU_P}

timestep        0.001000
run             ${NSTEPS} upto
```

The velocity seed 76752 is generated randomly. ### DFT

Configures self-consistent labeling calculations as a dictionary.

6.3.11.6 dft_style Selects the labeling software. The default is pwmat; vasp, cp2k, and bigmodel are also supported. For bigmodel, specify a script with bigmodel_script.

6.3.11.7 bigmodel_script Sets the foundation-model labeling script used to infer energies and forces.

For interface setup, see the si_direct_bigmodel example.

6.3.11.8 input Sets the input control file as an absolute path or a path relative to the current directory.

6.3.11.9 kspacing Optional PWmat k-point setting used when MP_N123 is absent from etot.input. Do not configure both.

If neither is set, kspacing defaults to 0.5.

6.3.11.10 flag_symm Optional PWmat symmetry setting. The default is 0 for relaxation or SCF and 3 for AIMD.

6.3.11.11 pseudo List of PWmat or VASP pseudopotential files. Paths may be absolute or relative.

6.3.11.12 gaussian_param Gaussian-basis settings for CP2K or PWmat. `basis_set_file` and `potential_file` specify basis and potential files. `atom_list`, `basis_set_list`, and `potential_list` map elements to their basis sets and potentials. `kspacing` configures k-points as described above.

```
"gaussian_param": {
    "basis_set_file":"./init_bulk/BASIS_MOLOPT_1",
    "potential_file":"./init_bulk/POTENTIAL_1",
    "atom_list":["Si"],
    "kspacing" :0.4,
    "basis_set_list":["SZV-MOLOPT-SR-GTH"],
    "potential_list":["GTH-PBE-q4"]
}
```

6.3.11.13 DFT Examples Because all SCF labeling tasks share one control file, only a single input is needed. Configure each DFT package as follows.

6.3.11.13.1 PWmat PWmat configuration resembles init_bulk. If `MP_N123` is absent from `scf_etot.input`, set `kspacing` and `flag_symm`.

```
  "DFT": {
          "dft_style": "pwmat",
          "input": "scf_etot.input",
          "kspacing":0.5,
          "flag_symm":0,
          "pseudo" : ["../Ag.SG15.PBE.UPF","../Au.SG15.PBE.UPF"],
          "_flag":"One integer: 0 for SCF, 3 for AIMD, and 2 for
           ↪  magnetic systems"
  }
```

6.3.11.13.2 VASP

```
  "DFT": {
          "dft_style":"vasp",
          "input":"INCAR_scf",
          "pseudo" : ["../Ag_POTCAR", "../Au_POTCAR"]
  }
```

6.3.11.13.3 CP2K

```
  "DFT": {
          "dft_style":"cp2k",
          "input":"scf.inp",
```

```
        "gaussian_param": {
            "basis_set_file":"../BASIS_MOLOPT_1",
            "potential_file":"../POTENTIAL_1",
            "atom_list":["Ag", "Au"],
            "basis_set_list":["SZV-MOLOPT-SR-GTH-q11",
             ↪  "SZV-MOLOPT-SR-GTH-q11"],
            "potential_list":["GTH-PBE", "GTH-PBE"],
            "_kspacing":0.5
        }
  }
```

Set `IGNORE_CONVERGENCE_FAILURE` in `scf_cp2k.inp` so CP2K exits normally when convergence fails and PWact can continue. Otherwise, PWact treats the exit as abnormal, resubmits the task up to three times, and stops after the third failure.

6.3.12 run param.json Example

The following standard workflow runs two active-learning iterations with a multi-model committee. For more examples, see the source repository's `examples`.

```
{
  "reserve_work": false,
  "reserve_md_traj": false,
  "reserve_scf_files": false,

  "init_data": ["/path/init_data"],

  "train": {
    "_train_input_file": "std_si.json",

    "model_type": "DP",
    "atom_type": [14],
    "seed": 2023,
    "recover_train": true,
    "model": {
      "descriptor": {
        "Rmax": 6.0,
        "Rmin": 0.5,
        "M2": 16,
        "network_size": [25, 25, 25]
      },
      "fitting_net": {
        "network_size": [50, 50, 50, 1]
      }
```

```
    },
    "optimizer": {
      "optimizer": "ADAM",
      "epochs": 30,
      "batch_size": 1,
      "print_freq": 10,
      "train_energy": true,
      "train_force": true,
      "train_virial": false
    }
  },

  "strategy": {
    "uncertainty": "committee",
    "lower_model_devi_f": 0.05,
    "upper_model_devi_f": 0.15,
    "model_num": 4,
    "max_select": 10
  },

  "explore": {
    "sys_config_prefix": "/path/structures",
    "sys_configs": [
      { "config": "POSCAR", "format": "vasp/poscar" },
      "atom1.config",
      "atom2.config",
      "atom3.config",
      "atom4.config"
    ],
    "md_jobs": [
      [
        {
          "ensemble": "nvt",
          "nsteps": 1000,
          "md_dt": 0.002,
          "trj_freq": 10,
          "taup": 0.5,
          "sys_idx": [0, 1],
          "temps": [500, 700],
          "taut": 0.1,
          "boundary": true
        },
        {
```

```
          "ensemble": "npt",
          "nsteps": 1000,
          "md_dt": 0.002,
          "trj_freq": 10,
          "press": [100.0, 200.0],
          "taup": 0.5,
          "sys_idx": [0, 3],
          "temps": [500, 700],
          "taut": 0.1,
          "boundary": true
        }
      ],
      {
        "ensemble": "nvt",
        "nsteps": 4000,
        "md_dt": 0.002,
        "trj_freq": 10,
        "press": [100.0, 200.0],
        "taup": 0.5,
        "sys_idx": [0, 1],
        "temps": [500, 700],
        "taut": 0.1,
        "boundary": true
      }
    ]
  },
  "DFT": {
    "dft_style": "pwmat",
    "input": "scf_etot.input",
    "kspacing": 0.5,
    "flag_symm": 0,
    "pseudo": ["path/Si.SG15.PBE.UPF"]
  }
}
```

6.4 resource Configuration

6.4.1 resource.json File

This file configures cluster resources for training, molecular dynamics, and DFT calculations (SCF, relaxation, and AIMD), including compute nodes, CPUs, GPUs, and software such as LAMMPS, VASP, PWmat, and MatPL.

Parameters are grouped into `train`, `explore`, DFT, and `direct` modules. The resource fields have the same meaning in every module.

For initial training-set preparation (`init_bulk`), configure `explore`, DFT, and `direct`. In the Mcloud example below, `explore` defines the foundation-model MD environment, `direct` defines the DIRECT sampling environment, and DFT defines the environment for SCF or AIMD calculations.

```
{
    "DFT": {
        "command":"mpirun -np 4 PWmat",
        "task_run_num":1,
        "number_node": 1,
        "cpu_per_node": 4,
        "gpu_per_node": 4,
        "group_size": 1,
        "queue_name": "3080ti,3090",
        "custom_flags": [
        ],
        "_custom_flags": [
        ],
        "module_list": [
            "compiler/2022.0.2",
            "mkl/2022.0.2",
            "mpi/2021.5.1",
            "cuda/11.6",
            "pwmat"
        ]
    },

    "explore": {
        "command": "python sevennet_md.py",
        "group_size": 1,
        "number_node": 1,
        "gpu_per_node": 1,
        "cpu_per_node": 1,
        "queue_name": "3080ti",
        "custom_flags": [],
        "source_list": [
            "/share/app/anaconda3/envs/SevenNet/env.sh"
        ],
        "module_list": [
        ],
        "env_list": [
```

```
        ]
    },

    "direct": {
        "command": "python direct.py",
        "group_size": 1,
        "number_node": 1,
        "gpu_per_node": 1,
        "cpu_per_node": 1,
        "queue_name": "3080ti",
        "custom_flags": [],
        "source_list": [
            "/share/app/anaconda3/envs/m3gnet/env.sh"
        ],
        "module_list": [
        ],
        "env_list": [
        ]
    }
}
```

For active learning (`run`), configure all four modules. In the Mcloud example below, `train` defines the training environment, `explore` defines the LAMMPS MD environment, `direct` defines the DIRECT sampling environment, and DFT defines the environment for SCF calculations or foundation-model labeling.

```
{
  "train": {
    "command": "MatPL",
    "group_size": 1,
    "number_node": 1,
    "gpu_per_node": 1,
    "cpu_per_node": 1,
    "queue_name": "3080ti,3090",
    "custom_flags": [
    ],
    "source_list": [
      "/share/app/MATPL/MatPL-2025.3/env.sh"
    ],
    "module_list": [
    ],
    "env_list":[
    ]
  },
```

```
  "explore": {
    "command": "mpirun -np 1 lmp_mpi -in in.lammps",
    "group_size": 1,
    "number_node": 1,
    "gpu_per_node": 1,
    "cpu_per_node": 1,
    "queue_name": "3080ti,3090",
    "custom_flags": [],
    "source_list": [],
    "module_list": [
      "lammps4matpl/2025.3"
    ],
    "env_list":[]
  },
  "DFT": {
    "command": "mpirun -np 4 PWmat",
    "number_node": 1,
    "cpu_per_node": 4,
    "gpu_per_node": 4,
    "group_size": 1,
    "queue_name": "3080ti,3090",
    "custom_flags": [],
    "source_list": [],
    "module_list": [],
    "env_list":[
      "compiler/2022.0.2",
      "mkl/2022.0.2",
      "mpi/2021.5.1",
      "cuda/11.6",
      "pwmat"
    ]
  }
}
```

6.4.2 Parameter Details

The parameters fall into three categories: `command`, which defines the executable command;

resource fields for each computational task: `number_node`、`cpu_per_node`、`gpu_per_node`、`group_size`、`queue_name`、`custom_flags`;

and `custom_flags`, `source_list`, `module_list`, and `env_list`, which load software and environment variables.

The fields are described below. ### command

Required. Sets the command for the module.

Examples for different tasks:

For DFT calculations:

```
PWmat:
"command":"mpirun -np 4 PWmat"
VASP:
"command":"vasp_std"
CP2K:
"command":"mpirun -np $SLURM_NTASKS cp2k.popt"
```

For LAMMPS:

```
"command":"mpirun -np 10 lmp_mpi"
```

The value after -np is the CPU process count and must match cpu_per_node. When gpu_per_node is set, these processes are distributed evenly across the configured GPUs.

For model training with MatPL:

```
"command":"MatPL"
```

6.4.3 group_size

Groups multiple submitted calculations. Tasks within a group run sequentially, while groups run in parallel.

For example, with 34 SCF calculations and "group_size": 5, PWact creates 7 groups (six groups of five and one group of four), represented by seven Slurm jobs. The Slurm jobs are submitted in parallel, while the calculations inside each job run sequentially.

- For training or exploration, the default is 1, so each task forms its own group and all tasks are submitted together.
- For AIMD or SCF tasks under DFT, the default is -1, placing all tasks in one group for sequential execution. Set this value according to the workload to avoid submitting enough DFT jobs to consume all cluster resources.

6.4.4 number_node

Sets the number of compute nodes per Slurm job. The default is 1.

In the train module, this value is automatically set to 1.

6.4.5 gpu_per_node

Sets the number of GPUs per node. The default is `0`. For PWmat DFT calculations (SCF, relaxation, or AIMD), it must match the GPU count specified by `command`.

In the `train` module, this value is automatically set to `1`.

6.4.6 cpu_per_node

Sets the number of CPUs per node. The default is `1`, and it must be greater than or equal to `gpu_per_node`.

In the `train` module, this value is automatically set to `1`.

6.4.7 queue_name

Required. Sets the cluster partitions as a comma-separated string, for example `"queue_name":"cpu, 3080ti, 3090"`.

6.4.8 custom_flags

Optional list of additional #SBATCH options for the Slurm script. For example:

```
"custom_flags": [
  "#SBATCH -x gn43,gn66"
]
```

PWact adds `"#SBATCH -x gn43,gn66"` to the Slurm script. The #SBATCH prefix may be omitted, leaving only `-x gn43,gn66`.

For example, with the following settings:

```
"number_node": 1,
"cpu_per_node": 4,
"gpu_per_node": 4,
"queue_name": "3080ti,3090",
"custom_flags": [
  "#SBATCH -x gn43,gn66"
]
```

the generated Slurm header is:

```
#SBATCH --nodes=1
#SBATCH --gres=gpu:4
#SBATCH --ntasks-per-node=4
```

```
#SBATCH --partition=3080ti,3090
#SBATCH -x gn43,gn66
```

Because some systems do not support `--gpus-per-task`, PWact does not generate it automatically. Add it through `custom_flags` when needed.

6.4.9 source_list

Optional list of environment scripts to source at runtime. For example:

```
"source_list": [
    "~/anaconda3/etc/profile.d/conda.sh"
]
```

PWact prefixes each string with `source` and writes it to the Slurm script.

6.4.10 module_list

Optional list of software modules to load in the Slurm script.

For example:

```
"module_list": [
    "cuda/11.8",
    "intel/2020"
]
```

PWact writes `module load cuda/11.8` and `module load intel/2020` to the Slurm script.

6.4.11 env_list

Optional list of arbitrary environment commands written verbatim to the Slurm script.

For example:

```
"env_list": [
    "source the/path/MatPL-2025.3/env.sh"
]
```

The string is copied directly into the Slurm script.

With the queue_name, custom_flags, source_list, module_list, and env_list settings above, the generated Slurm script contains:

```
#SBATCH --partition=3080ti,3090
#SBATCH -x gn43,gn66

source /opt/rh/devtoolset-8/enable
module load cuda/11.8
module load intel/2020
source the/path/MatPL-2025.3/env.sh
```

6.4.12 Configuration Example: train Module

The `train` module requires the MatPL Python environment. To use the MatPL installation on Mcloud, configure:

```
"train": {
  "command": "MatPL",
  "group_size": 1,
  "number_node": 1,
  "gpu_per_node": 1,
  "cpu_per_node": 1,
  "queue_name": "3080ti,3090",
  "custom_flags": [
  ],
  "source_list": [
  "/share/app/MATPL/MatPL-2025.3/env.sh"
  ],
  "env_list": [
  ],
  "module_list": [
  ]
}
```

Each training task uses one node, one GPU, and one CPU from either the `3080ti` or `3090` partition.

For a MatPL installation built from source, use:

```
"train":{
  "command": "MatPL",
  "group_size": 1,
  "number_node": 1,
  "gpu_per_node": 1,
  "cpu_per_node": 1,
  "queue_name": "3080ti,3090",
  "custom_flags": [
```

```
    ],
    "source_list": [
        "~/anaconda3/etc/profile.d/conda.sh",
        "the/path/MatPL-2025.3/env.sh"
    ],
    "env_list": [
        "conda activate matpl-2025.3"
    ],
    "module_list": [
        "cuda/11.8-share",
        "intel/2020"
    ]
}
```

Here, `"~/anaconda3/etc/profile.d/conda.sh"` is the Conda initialization script, `matpl-2025.3` is the MatPL Python environment, and `the/path/MatPL-2025.3` is the source path.

6.4.13 Configuration Example: explore Module

For the `explore` module with the LAMMPS installation on Mcloud, load `lammps4matpl` as follows:

```
    "explore": {
      "command": "mpirun -np 1 lmp_mpi",
      "group_size": 2,
      "number_node": 1,
      "gpu_per_node": 1,
      "cpu_per_node": 1,
      "queue_name": "3080ti,3090",
      "custom_flags": [
      ],
      "source_list": [

      ],
      "module_list": [
          "lammps4matpl/2025.3"
      ],
      "env_list":[

      ]
    }
```

Each LAMMPS task uses one node, one GPU, and one CPU, with two LAMMPS tasks per

group.

For LAMMPS built from source, use:

```
  "explore": {
    "command": "mpirun -np 1 lmp_mpi",
    "group_size": 2,
    "number_node": 1,
    "gpu_per_node": 1,
    "cpu_per_node": 1,
    "queue_name": "3080ti,3090",
    "custom_flags": [
    ],
    "source_list": [
        "the/path/of/lammps/env.sh"
    ],
    "module_list": [
        "cuda/11.8-share",
        "intel/2020"
    ],
    "env_list":[
    ]
    }
```

Here, `the/path/of/lammps/env.sh` is the environment file in the LAMMPS source tree.

6.4.14 Configuration Example: DFT Module

The following example configures PWmat in the DFT module:

```
  "DFT": {
      "command":"PWmat",
      "number_node": 1,
      "cpu_per_node": 4,
      "gpu_per_node": 4,
      "group_size": 5,
      "queue_name": "3080ti,1080ti,3090",
      "custom_flags": [
      "#SBATCH -x gn18,gn17"
      ],
      "module_list": [
          "compiler/2022.0.2",
          "mkl/2022.0.2",
          "mpi/2021.5.1",
```

```
            "cuda/11.6"
        ],
        "env_list":[
        ]
    }
```

Each PWmat task uses one node, four GPUs, and four CPUs, with five DFT tasks per group.

6.5 Active-Learning Example for Silicon

This example demonstrates active learning for silicon and is located in `pwact/example/si_pwmat/`. Mcloud users can find it under `/share/public/PWMLFF_test_data/pwact_examples/25-pwact-demo`.

The example first constructs an initial training set with `init_bulk`, trains models on that dataset, and uses structures generated by perturbation during `init_bulk` as initial configurations for active-learning sampling at `300 K`, `500 K`, and `900 K`.

The supplied DFT settings are intended only to test the workflow and do not guarantee calculation accuracy.

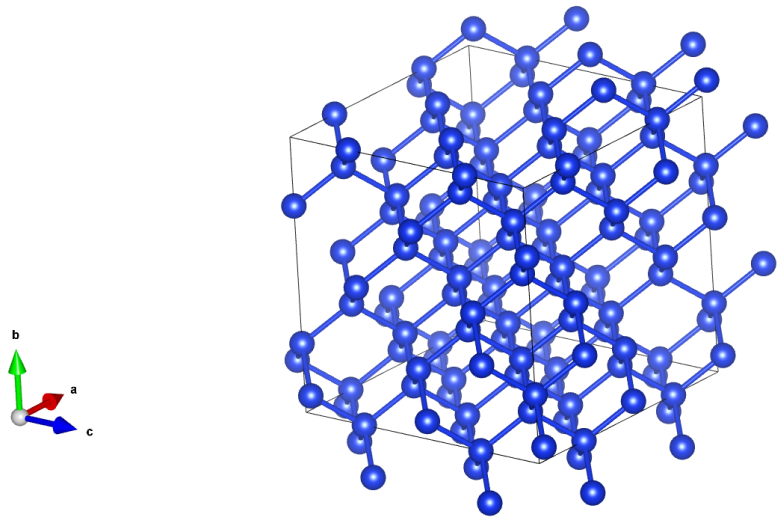


Figure 16: Bulk silicon

6.5.1 init_bulk Example

Command:

Enter `pwact/example/si_pwmat/init_bulk`:

```
pwact init_bulk init_param.json resource.json
```

6.5.2 init_bulk Directory Structure

The directory structure is shown below. `atom.config`, POSCAR, `resource.json`, `init_param.json`, `relax_etot.input`, `relax_etot1.input`, `aimd_etot1.input`, and `aimd_etot2.input` are input files; `collection` contains the results.

6.5.3 init_bulk Directory Structure: collection

`datapath.txt` records the locations of the pretraining datasets.

`init_config_0` collects the results of relaxation, supercell construction, scaling, perturbation, and AIMD applied to `atom.config`.

`relax.config` is the relaxed structure; `super_cell.config` is the expanded structure; and `0.9_scale.config` and `0.95_scale.config` are lattice-scaled variants of `super_cell.config`.

`0.9_scale_pertub` contains 30 structures generated by perturbing the lattice and atomic positions of `0.9_scale.config`.

After AIMD is run on the perturbed structures with `aimd_etot1.input`, the trajectories are extracted to `train.xyz` in extxyz format. For `pwmlff/npy`, the output is a `PWdata` directory such as the one below. `atom_type.npy` stores atom types, `position.npy` stores atomic positions, and `energies.npy`, `forces.npy`, `ei.npy`, and `virials.npy` store total energies, Cartesian forces, atomic energies, and virials. `ei.npy` and `virials.npy` are optional and are omitted when the trajectory lacks those labels.

```
./init_config_0/PWdata/Si128
├── atom_type.npy
├── energies.npy
├── ei.npy
├── forces.npy
├── image_type.npy
├── lattice.npy
├── position.npy
└── virials.npy
```

6.5.4 init_bulk Directory Structure: Tree

```
example/init_bulk
├──atom.config
├──POSCAR
├──resource.json
├──init_param.json
```

```
├──relax_etot.input
├──relax_etot1.input
├──relax_etot2.input
├──aimd_etot1.input
├──aimd_etot2.input
└──collection
    ├──datapath.txt
    ├──init_config_0
    │   ├──super_cell.config
    │   ├──0.9_scale.config
    │   ├──0.9_scale_pertub
    │   │       ├──0.9_scale.config
    │   │       ├──0_pertub.config
    │   │       ├──1_pertub.config
    │   │       ├──2_pertub.config
    │   │       ...
    │   │       └──30_pertub.config
    │   ├──0.95_scale.config
    │   ├──0.95_scale_pertub
    │   │       ├──0.95_scale.config
    │   │       ├──0_pertub.config
    │   │       ├──1_pertub.config
    │   │       ├──2_pertub.config
    │   │       ...
    │   │       └──30_pertub.config
    │   └──train.xyz
    ├──init_config_1
    ...
```

6.5.5 run Active-Learning Example

The workflow uses the pretraining data and perturbed structures from `init_bulk` for active learning at 500 K, 800 K, and 1100 K.

Command: After `init_bulk` completes, enter `pwact/example/si_pwmat/`:

```
pwact run param.json resource.json
```

6.5.6 run Active-Learning Directory

The active-learning directory structure is shown below.

`param.json` and `resource.json` are active-learning control files; `scf_etot.input` controls self-consistent labeling calculations.

`si.al` records the completed active-learning stages.

`iter_result.txt` records exploration and label selection for each iteration. In the example below, the first line summarizes selection: `Total structures = accurate + selected + error`. The next lines summarize MD trajectories, including successful and failed runs; failed trajectories are listed in `error_traj.log` and usually indicate that an inaccurate force field caused MD instability. Subsequent lines report converged and unconverged labeling calculations and list the paths of failures. Unconverged structures are not added to the training set.

```
iter.0000  Total structures 1296    accurate 6 rate 0.46%    selected
 ↪  23 rate 1.77%    error 1267 rate 97.76%
A total of 16 MD trajectories were run. with 16 trajectories
 ↪  correctly executed and 0 trajectories normally completed.
For detailed information, refer to File error_traj.log.

Number of converged files: 194, number of non-converged files: 3
List of non-converged files:
/share/public/PWMLFF_test_data/pwact_examples/25-pwact-
 ↪  demo/auag_vasp/run_iter_lmps/iter.0000/temp_run_iter_work/02.label/scf/md.00
 ↪  scf/OUTCAR
...

iter.0001  Total structures 12816    accurate 598 rate 4.67%
 ↪  selected 2926 rate 22.83%    error 9292 rate 72.50%
A total of 16 MD trajectories were run. with 16 trajectories
 ↪  correctly executed and 0 trajectories normally completed.
For detailed information, refer to File error_traj.log.

Number of converged files: 194, number of non-converged files: 3
List of non-converged files:
/share/public/PWMLFF_test_data/pwact_examples/25-pwact-
 ↪  demo/auag_vasp/run_iter_lmps/iter.0001/temp_run_iter_work/02.label/scf/md.00
 ↪  scf/OUTCAR
...
```

`iter.0000` is the first active-learning iteration, `iter.0001` is the second, and so on.

`00.train`, `01.explore`, and `02.label` contain the training, exploration, and labeling stages of an iteration.

6.5.7 run 00.train Directory

This example uses a four-model committee. The models differ only in their initial network parameters. `0-train.job` through `3-train.job` are the four Slurm training scripts. Successful training produces four marker files (`0-tag.train.success` through `3-tag.train.success`) and four model directories (`train.000` through `train.003`).

In `train.000`, `train.json` is the MatPL training input, `std_input.json` summarizes the resolved training settings, and `model_record` stores the model. `dp_model.ckpt` is the checkpoint, `epoch_train.dat` records average training error for each epoch, and `epoch_valid.dat` records average validation error after each epoch.

`torch_script_module.pt` is produced by compiling `dp_model.ckpt` with TorchScript and serves as the force field for subsequent LAMMPS simulations.

6.5.8 run 01.explore Directory

`01.explore` contains `md` and `select`.

`md` contains the input files and trajectories for MD exploration of the initial structures under different temperatures, pressures, and other conditions.

The resulting trajectories are filtered using the Committee Query thresholds, and selection results are saved under `select`.

6.5.9 run 01.explore Directory: md

MD directories follow the pattern `md.***.sys.***/md.***.sys.***.t.***`, such as `md.000.sys.000/md.000.sys.000.t.000`. In `md.000.sys.000`, `md.000` refers to the first `md_jobs` entry, and `sys.000` refers to the structure at index `0` in `sys_index`. Subdirectories such as `md.000.sys.000.t.000` and `md.000.sys.000.t.001` represent simulations at temperature indices `0` and `1` in `temps`.

Each `md.*.sys.*` directory contains `model_devi_distribution.png`, which plots the deviation distribution across its trajectories.

The deepest subdirectories contain the MD settings and trajectory files:

```
0_torch_script_module.pt, 1_torch_script_module.pt,
 ↪ 2_torch_script_module.pt, 3_torch_script_module.pt,
 ↪ atom_type.txt, in.lammps, lmp.config, log.lammps, md.log,
 ↪ model_devi.out, tag.md.success
```

```
model_devi.out format:
#    step         avg_devi_f         min_devi_f         max_devi_f
 ↪  avg_devi_e         min_devi_e         max_devi_e
       0      0.008682422      0.000574555      0.014187865
 ↪  0.030333196      0.011702489      0.043806718
      10      0.016611307      0.000627126      0.027185410
 ↪  0.031661788      0.005340899      0.048241921
      20      0.028689107      0.000811486      0.040269587
 ↪  0.036761117      0.000214928      0.059791462
      30      0.046457606      0.001416745      0.063635973
 ↪  0.049553956      0.000165127      0.081012839
      40      0.065051169      0.000681319      0.089395358
 ↪  0.065790218      0.000036582      0.103522832
```

In `model_devi.out`, columns 1–4 are the step, mean force deviation, minimum force deviation, and maximum force deviation. The maximum force deviation is

$$\varepsilon_t = \max_i \sqrt{\left\langle \sum_{w=1}^{W} \| F_{w,i}(R_t) - \hat{F}_i \|^2 \right\rangle}, \quad \hat{F}_i = \frac{1}{W} \sum_{w=1}^{W} F_{w,i}$$

where W is the number of models, i indexes atoms, and $\langle \cdot \rangle$ denotes an average.

The final three columns are the mean, minimum, and maximum atomic-energy deviations. PWact uses the `maximum force deviation in column 4` for active learning.

6.5.10 run 01.explore Directory: select

The `.csv` files below contain `devi_force`, `config_index`, and `file_path` columns.

`accurate.csv` lists structures below the configured lower force-deviation threshold.

`fail.csv` lists structures above the configured upper force-deviation threshold.

If the number of candidates between the lower and upper thresholds exceeds `max_select`, PWact randomly selects `max_select` structures for `candidate.csv` and writes the remainder to `candidate_delete.csv`.

`model_devi_distribution-md.*.sys.*.png` files link to deviation-distribution plots for explored structures.

`error_traj.log` lists interrupted MD trajectories, usually caused by force-field inaccuracies that lead to lost atoms, unphysically short distances, or similar failures.

`select_summary.txt` summarizes the number of selected structures:

```
Total structures 862   accurate 0 rate 0.00%   selected 616 rate 71.46%   err

Select by model deviation force:
Accurate configurations: 0, details in file accurate.csv
Candidate configurations: 616, randomly select 27, delete 589
        Select details in file candidate.csv
        Delete details in file candidate_delete.csv.
Error configurations: 246, details in file fail.csv

A total of 10 MD trajectories were run. with 8 trajectories correctly execute
For detailed information, refer to File error_traj.log.
```

6.5.11 run 02.label Directory

`02.label` contains `scf` and `result`. `scf` holds self-consistent calculations for structures selected during exploration. After labeling, structures and their energies, forces, atomic energies, and virials are extracted to PWdata under `result` for training in later iterations.

6.5.12 run 02.label Directory: scf

The first- and second-level `md.*.sys.*/md.*.sys.*.t.*` directories under `scf` have the same naming scheme as the md directory. Their children contain individual SCF calculations. For example, `scf/md.000.sys.000/md.000.sys.000.t.000/820-scf` labels the structure at step 820 of that trajectory. `820.config` is the input structure, `etot.input` is the control file, and `REPORT` and `OUT.MLMD` are PWmat outputs. `OUT.MLMD` contains atomic positions, energy, forces, and related data.

6.5.13 run 02.label Directory: result

`result` collects the labeled dataset. With `data_format: extxyz`, it contains `train.xyz`; with `pwmlff/npy`, it contains a `PWdata` directory.

6.5.14 run Active-Learning Directory Tree

```
example
├──param.json
├──resource.json
├──scf_etot.input
├──si.al
```

```
├──iter_result.txt
├──iter.0000
│      ├──00.train
│      │    ├──0-train.job
│      │    ├──1-train.job
│      │    ├──3-train.job
│      │    ├──2-train.job
│      │    ├──train.000
│      │    │     ├──train.json
│      │    │     ├──std_input.json
│      │    │     ├──model_record
│      │    │     │     ├──dp_model.ckpt
│      │    │     │     ├──epoch_train.dat
│      │    │     │     └──epoch_valid.dat
│      │    │     ├──torch_script_module.pt
│      │    │     └──tag.train.success
│      │    ├──train.001
│      │    │     └──...
│      │    ├──train.002
│      │    │     └──...
│      │    ├──train.003
│      │    │     └──...
│      │    ├──0-tag.train.success
│      │    ├──1-tag.train.success
│      │    ├──2-tag.train.success
│      │    └──3-tag.train.success
│      ├──01.explore
│      │    ├──md
│      │    │     ├──md.000.sys.000
│      │    │     │     ├──md.000.sys.000.t.000.p.000
│      │    │     │     ├──...
│      │    │     │     └──model_devi_distribution.png
│      │    │     ├──md.000.sys.001
│      │    │     ├──md.001.sys.000
│      │    │     └──md.001.sys.003
│      │    └──select
│      │          ├──accurate.csv
│      │          ├──candidate.csv
│      │          ├──candidate_delete.csv
│      │          ├──fail.csv
```

```
│   │   ├── error_traj.log
│   │   ├── select_summary.txt
│   │   ├── model_devi_distribution-md.000.sys.000.png
│   │   └── ...
│   └── 02.label
│       ├── scf
│       │   ├── md.000.sys.000
│       │   │   ├── md.000.sys.000.t.001
│       │   │   │   ├── 820-scf
│       │   │   │   │   ├── 820.config
│       │   │   │   │   ├── etot.input
│       │   │   │   │   ├── REPORT
│       │   │   │   │   └── OUT.MLMD
│       │   │   │   ├── 200-scf
│       │   │   │   └── ...
│       │   │   └── md.000.sys.000.t.000
│       │   │       └── ...
│       │   ├── md.000.sys.001
│       │   ├── ...
│       │   ├── md.001.sys.000
│       └── result
│           └── train.xyz
├── iter.0001
│   └── ...
├── ...
```

6.6 Silicon Example (Foundation-Model Sampling)

This example demonstrates active learning for silicon using a foundation model (EquiformerV2) and DIRECT sampling. `PWact/examples/` provides several combinations; see `PWact/examples/README.md` for details. Mcloud users can find the examples under `/share/public/PWMLFF_test_data/pwact_examples/25-pwact-demo`. The [`PWact/examples/si_pwmatgaussion_bigmodel_direct`] example illustrates the parameters and scripts for foundation-model and DIRECT workflows. For constructing an initial training set with `init_bulk`, see `examples/si_pwmatgaussion_bigmodel_direct/init_bulk_bigmodel`:

- Step 1. Relax the structure with PWmat using a Gaussian basis.
- Step 2. Run molecular dynamics with a foundation model (SevenNet).

- Step 3. Apply DIRECT sampling to the MD trajectories, remove similar structures, and use the selected structures for subsequent active learning.

The Gaussian-basis PWmat settings in this directory are intended only for quick workflow testing. The generated data are not suitable for scientific use.

6.6.1 Prepare Pretraining Data with init_bulk

Command: Enter `examples/si_pwmatgaussion_bigmodel_direct/init_bulk_bigmodel`. The directory provides an Mcloud job script, or you can run:

```
pwact init_bulk init_param.json resource.json
```

6.6.2 init_bulk Directory Structure

The `init_bulk` directory resembles that of the si_pwmat example, with an additional `bigmodel` directory:

```
.
├── datapath.txt
├── init_config_0/
├── init_config_1/
└── bigmodel/
    ├── 0-bigmodel.job
    ├── 0-tag.bigmodel.success
    ├── 1-bigmodel.job
    ├── 1-tag.bigmodel.success
    ├── ...
    ├── direct/
    │   ├── 0-direct.job
    │   ├── 0-tag.direct.success
    │   ├── candidate.json
    │   ├── candidate.xyz
    │   ├── Cov_score.png
    │   ├── direct.py
    │   ├── PCA_direct.png
    │   ├── PCA_variance.png
    │   ├── PWdata
    │   ├── select_idx.dat
    │   ├── select.xyz
    │   └── tag.direct.success
    │
```

```
├── init_config_0/
│   ├── 0.95_scale/
│   │   ├── 3_bigmodel/
│   │   │   ├── npt.log
│   │   │   ├── POSCAR
│   │   │   ├── sevennet_md.py
│   │   │   ├── tag.bigmodel.success
│   │   │   ├── tmp.traj
│   │   │   └── traj.xyz
│   │   ├── 4_bigmodel/
│   │   └── 5_bigmodel/
│   ├── 0.9_scale/
│   └── 1.0_scale/
└── init_config_*/
```

- `*-bigmodel.job` and `*-tag.bigmodel.success` are Slurm scripts for foundation-model MD and success-marker files.
- `init_config_0`, `init_config_1`, and so on are working directories for foundation-model MD.
- `direct` is the working directory for structure selection with DIRECT.

6.6.3 Integrate Foundation-Model MD into init_bulk

Foundation-model MD uses a user-defined interface and requires the corresponding runtime environment. See sevennet_md.py in the example.

As illustrated by `bigmodel/init_config_0/0.95_scale/3_bigmodel/`, when PWact detects a bigmodel setting during `init_bulk`, it creates a working directory for each structure, writes the structure as a VASP/POSCAR file, copies the user-provided MD interface script such as `sevennet_md.py`, generates a Slurm script from the `explore` settings in `resource.json`, and submits the job.

The interface script must read POSCAR, run MD, and convert the trajectory to an `extxyz` file named `traj.xyz`. PWact detects this file and uses it in subsequent processing.

6.6.4 Integrate DIRECT Sampling into init_bulk

DIRECT sampling uses a user-defined interface and requires a configured DIRECT environment. See direct.py in the example.

After foundation-model MD produces multiple trajectories, PWact detects and merges them into `bigmodel/direct/candidate.xyz`. It copies the DIRECT interface script speci-

fied by direct_input, such as `direct.py`, into that directory, generates a Slurm script from the `direct` settings in `resource.json`, and submits the job.

The script must read `candidate.xyz` and generate `select_idx.dat` and `select.xyz`. `select_idx.dat` stores the indices of selected structures in `candidate.xyz`, while `select.xyz` stores the selected structures themselves. PWact reads both files for subsequent steps.

6.6.5 Active Learning with run

This example uses the pretraining data and perturbed structures from `init_bulk` for active learning at 500 K, 800 K, and 1100 K.

Command: After `init_bulk` completes, enter `examples/si_pwmatgaussion_bigmodel_direct`

```
pwact run param.json resource.json
```

6.6.6 Active-Learning Directory

The directory structure resembles that of the si_pwmat example.

6.6.7 train Directory

The `train` directory is identical to that in the si_pwmat example.

6.6.8 explore Directory

In addition to md and `select`, `explore` contains a `bigmodel` subdirectory whose contents match those described for DIRECT sampling in init_bulk.

The md and `select` subdirectories are identical to those in the si_pwmat example.

6.6.9 label Directory

When `bigmodel_script` is configured for foundation-model labeling, a `bigmodel` subdirectory is added under `label`:

```
bigmodel/
├── 0-bigmodel.job
├── eqv2_label.py
├── select.xyz
└── train.xyz
```

- `eqv2_label.py` is the user-specified `bigmodel_script`; PWact copies it into `bigmodel`.
- `select.xyz` is the extxyz input containing structures to label.
- `train.xyz` is the labeled extxyz output. It preserves the structure order of `select.xyz` and adds energies and forces.
- `0-bigmodel.job` is the Slurm script generated from the DFT settings in `resource.json`.

6.6.10 Integrate Foundation-Model Labeling into run

Foundation-model labeling uses a user-defined interface and requires the corresponding runtime environment. See eqv2_label.py in the example.

After multi-model deviation selection—or additional DIRECT selection—during `explore`, PWact detects the selected structures, merges them into `bigmodel/select.xyz`, copies the interface script specified by bigmodel_script, such as `eqv2_label.py`, generates a Slurm script from the DFT settings in `resource.json`, and submits the job.

The script must read `select.xyz` and generate `train.xyz`, adding energy and force information. PWact reads this file for subsequent steps.

6.6.11 result

`result` collects the labeled dataset. When `data_format` is `extxyz`, the result is `train.xyz`; when it is `pwmlff/npy`, the result is a `PWdata` directory containing the dataset files.

6.7 Au–Ag Alloy Example

This example demonstrates active learning for an Au–Ag alloy and is located in `pwact/example/auag_pw`. It first uses `init_bulk` to construct the initial training set and trains a model on that dataset. Structures generated by perturbation during `init_bulk` are then used as initial configurations, and active-learning sampling is controlled by a `user-provided lammp.in` file. :::tip Active-learning sampling with a `user-provided lammp.in` control file is supported in `pwact >= 0.4`. :::

The example below uses PWmat for DFT calculations. Corresponding configurations are also provided for VASP in `pwact/example/auag_vasp/` and CP2K in `pwact/example/auag_cp2k/`. Mcloud users can find the examples under `/share/public/PWMLFF_test_data/pwact_examples/25-pwact-demo`.

Note that the DFT settings supplied with the example are intended only to test the workflow and do not guarantee calculation accuracy.

6.7.1 Prepare Pretraining Data with init_bulk

Command: Enter the pwact/example/auag_pwmat/init_bulk directory:

```
pwact init_bulk init_param.json resource.json
```

6.7.2 init_bulk Directory Structure

The init_bulk directory has the same structure as in the si_pwmat example.

6.7.3 Active Learning with run

Command: After init_bulk completes, enter the examples/auag_pwmat/run_iter_lmps directory:

```
pwact run param.json resource.json
```

6.7.4 Active-Learning Directory

The active-learning directory has the same structure as in the si_pwmat example. The only difference is the MD subdirectory name: for example, md.000.sys.000/md.000.sys.000.t.000.p becomes md.000.sys.000/md.000.sys.000.lmps.000, where lmps.000 identifies the selected lammps.in file.

```
example
├──param.json
├──resource.json
├──scf_etot.input
├──iter.0000
│     ├──00.train
│     │    ├──...
...
│     ├──01.explore
│     │    ├──md
│     │    │     ├──md.000.sys.000
│     │    │     │      ├──md.000.sys.000.lmps.000
│     │    │     │      ├──...
│     │    │     │      └──model_devi_distribution.png
│     │    │     ├──md.000.sys.001
│     │    │     ├──md.001.sys.000
│     │    │     └──md.001.sys.003
│     │    └──select
│     │          ├──accurate.csv
```

```
│   │   ├──candidate.csv
│   │   ├──candidate_delete.csv
│   │   ├──fail.csv
│   │   ├──error_traj.log
│   │   ├──select_summary.txt
│   │   ├──model_devi_distribution-md.000.sys.000.png
│   │   └──...
│   └──02.label
│   │   ├──scf
│   │   │   ├──md.000.sys.000
│   │   │   │   ├──md.000.sys.000.t.001
│   │   │   │   │   ├──820-scf
│   │   │   │   │   │   ├──820.config
│   │   │   │   │   │   ├──etot.input
│   │   │   │   │   │   ├──REPORT
│   │   │   │   │   │   └──OUT.MLMD
│   │   │   │   │   ├──200-scf
│   │   │   │   │   └──...
│   │   │   │   └──md.000.sys.000.t.000
│   │   │   │       └──...
│   │   │   ├──md.000.sys.001
│   │   │   ├──...
│   │   │   ├──md.001.sys.000
│   │   └──result
│   │       └──train.xyz

├──iter.0001
│   └──...
├──...
```

7 pwdata Data Conversion

7.1 Overview

`This page applies to pwdata >= 0.5.10.`

pwdata is MatPL's data-preprocessing toolkit. It can:

- Convert among `atom.config` (PWmat), POSCAR (VASP), `lmp.init` (LAMMPS), and `cp2k.init` (CP2K).
- Build supercells, scale lattices, and perturb lattices or atomic positions.
- Extract `MOVEMENT` (PWmat), `OUTCAR` (VASP), LAMMPS dump, and CP2K MD trajectories, or read `pwmlff/npy`, `extxyz`, `deepmd/npy`, `deepmd/raw`, and the `Meta OMAT24 dataset`, then write `pwmlff/npy` or `extxyz`. For Meta datasets, CPU-parallel queries make it practical to search databases containing over 100 million structures.

7.1.1 Supported Data Types

Software	file	multi-Image	label	format
PWmat	MOVEMENT	True	True	'pwmat/movement'
PWmat	OUT.MLMD	False	True	'pwmat/movement'
PWmat	atom.config	False	False	'pwmat/config'
VASP	OUTCAR	True	True	'vasp/outcar'
VASP	poscar	False	False	'vasp/poscar'
LAMMPS	lmp.init	False	False	'lammps/lmp'
LAMMPS	dump	True	False	'lammps/dump'
CP2K	stdout, xyz, pdb	True	True	'cp2k/md'
CP2K	stdout	False	True	'cp2k/scf'
MatPL	*.npy	True	True	'pwmlff/npy'
DeepMD (read)	*.npy, *.raw	True	True	'deepmd/npy' , 'deepmd/raw'
* (extended xyz)	*.xyz	True	True	'extxyz'
Meta (read)	*aselmdb	True	True	'meta'

7.1.2 Installation

Install with pip:

```
pip install pwdata

# Install or upgrade pwdata
pip install pwdata --upgrade

# List available versions
pip index versions pwdata
# Example output:
# pwdata (0.5.10)
# Available versions: 0.5.10, 0.5.9, 0.5.8, 0.5.7, 0.5.6, 0.5.5,
 ↪  0.5.4, 0.5.3, 0.4.7, 0.4.0, 0.3.2, 0.2.16

# Install a specific version
pip install pwdata==n.m.o
```

7.2 Command-Line Usage

pwdata supports both `command-line use` and `Python API integration`.

Command list: Use -h for descriptions, options, and examples. `pwdata -h` lists all commands.

```
pwdata -h
pwdata convert_config or cvt_config -h
pwdata convert_configs or cvt_configs -h
pwdata scale_cell or scale -h
pwdata super_cell or super -h
pwdata perturb -h
pwdata count -h
```

7.2.1 pwdata convert_config: Convert a Structure

Converts between structure formats. Use `convert_config` or `cvt_config`. Options:

```
pwdata convert_config [-h] -i INPUT -f INPUT_FORMAT -o OUTPUT_FORMAT
 ↪  [-s SAVENAME] [-c] [-t ATOM_TYPES [ATOM_TYPES ...]]
```

7.2.1.1 -h Displays all options and their descriptions.

7.2.1.2 -i Required input path, absolute or relative.

7.2.1.3 -f Optional input format. If omitted, it is inferred. Supported values: pwmat/config, vasp/poscar, lammps/lmp, and cp2k/scf.

7.2.1.4 -o Required output format: pwmat/config, vasp/poscar, or lammps/lmp.

7.2.1.5 -s Output filename used with -o. Defaults are atom.config, POSCAR, and lammps.lmp for the respective formats.

7.2.1.6 -c Use -c to write Cartesian coordinates; otherwise fractional coordinates are used. pwmat/config always uses fractional coordinates.

7.2.1.7 -t For lammps/lmp or lammps/dump, specifies atom types as symbols or atomic numbers in input-file order. Ignored for other formats.

See the repository' s examples:

```
# convert_config example: atom.config to POSCAR
# Creates examples/test_workdir/cvtcnf_atom.POSCAR
pwdata cvt_config -i examples/pwmat_data/LiGePS_atom.config -s
 ↪  examples/test_workdir/cvtcnf_atom.POSCAR -o vasp/poscar
```

7.2.2 pwdata convert_configs: Convert Training Data

Extracts supported trajectories and datasets—including MOVEMENT, OUTCAR, LAMMPS dump, CP2K MD, pwmlff/npy, extxyz, deepmd/npy, deepmd/raw, and Meta OMAT24—to pwmlff/npy or extxyz. Use convert_configs or cvt_configs.

Options:

```
pwdata [-h] -i INPUT [INPUT ...] [-f INPUT_FORMAT] [-s SAVEPATH] [-o
 ↪  OUTPUT_FORMAT] [-r]
              [-m MERGE] [-g GAP] [-q QUERY] [-n CPU_NUMS] [-t
               ↪  ATOM_TYPES [ATOM_TYPES ...]]
```

7.2.2.1 -h Displays all options and descriptions.

7.2.2.2 -i Required list of input files or directories, absolute or relative. For `pwmlff/npy`, `extxyz`, `deepmd/npy`, `deepmd/raw`, and Meta OMAT24, pwdata searches directories recursively, so only the data root is required.

For example, to read all dataset subdirectories under examples/pwmlff_data/LiSiC, use `-i examples/pwmlff_data`.

For many files or directories, such as a Meta database, list the paths in JSON and pass that file with `-i`.

```
{
    "datapath" : [
        "/share/public/PWMLFF_test_data/eqv2-
         ↪ models/datasets/decompress/Omat24/train/rattled-1000-
         ↪ subsampled",
        "/share/public/PWMLFF_test_data/eqv2-
         ↪ models/datasets/decompress/Omat24/train/rattled-300",
        "/share/public/PWMLFF_test_data/eqv2-
         ↪ models/datasets/decompress/Omat24/train/rattled-300-
         ↪ subsampled"
    ]
}
```

7.2.2.3 -f Optional input format; inferred by default. Supported values are `pwmat/movement`, `vasp/outcar`, `lammps/dump`, `cp2k/md`, `pwmlff/npy`, `deepmd/npy`, `deepmd/raw`, `extxyz`, and `meta`.

7.2.2.4 -o Optional output format: `pwmlff/npy` (default) or `extxyz`.

7.2.2.5 -s Optional output directory; defaults to the current directory.

7.2.2.6 -m Optional integer for extxyz output. `-m 1` (default) writes all structures to one XYZ file; `-m 0` separates files by element combination.

7.2.2.7 -g Optional integer trajectory interval. The default is `1`.

7.2.2.8 -q Optional query string for `meta` input. Quote expressions such as `H>4` so the shell does not interpret > as redirection. See the Meta query examples.

7.2.2.9 -n Optional CPU count for parallel Meta queries. The default is one core.

7.2.2.10 `-t` For LAMMPS input, specifies atom types as symbols or atomic numbers in input order.

For Meta input, selects structures containing `only these element types`.

7.2.3 pwdata convert_configs Examples

See the repository' s examples:

Example 1. Extract every fifth frame from two PWmat trajectories to `examples/test_workdir/3_1_co`

```
pwdata convert_configs -i examples/pwmat_data/50_LiGePS_movement
 ↪  examples/pwmat_data/lisi_50_movement -s
 ↪  examples/test_workdir/3_1_configs_extxyz -o extxyz -g 5
```

Example 2. Convert all `deepmd/npy` data under `examples/deepmd_data/alloy` to `pwmlff/npy`.

```
pwdata convert_configs -i examples/deepmd_data/alloy -s
 ↪  ./test_workdir/7_0_configs_PWdata
```

Example 3. Convert all XYZ files under `examples/xyz_data` to `pwmlff/npy` in `examples/test_workdir/5_0_configs_PWdata`.

```
pwdata convert_configs -i examples/xyz_data -s
 ↪  examples/test_workdir/5_0_configs_PWdata -g 1
```

Example 4. Query all `.aselmdb` databases under `examples/meta_data/alex_val` for structures containing only Pt and Ge, then save them under `./test_workdir/10_1_configs_ext`

```
pwdata convert_configs -i examples/meta_data/alex_val -s
 ↪  ./test_workdir/10_1_configs_extxyz -t Pt Ge
```

Example 5. Query the `.aselmdb` databases listed in `meta_data['data_path']` for Pt–Ge structures and save them under `./test_workdir/10_1_configs_extxyz`.

```
pwdata convert_configs -i examples/meta_data.json -s
 ↪  ./test_workdir/10_1_configs_extxyz -o extxyz -t Pt Ge
```

7.2.4 Meta Query Examples for convert_configs

Example 1. Query structures containing `only` Pt and Ge and write XYZ.

```
pwdata convert_configs -i
 ↪  examples/meta_data/alex_val/alex_go_aao_001.aselmdb
 ↪  examples/meta_data/alex_val/alex_go_aao_002.aselmdb -s
 ↪  examples/test_workdir/10_1_configs_extxyz -o extxyz -t Pt Ge
```

Example 2. Use `-q` to find all structures containing Cu.

```
pwdata convert_configs -i
 ↪  examples/meta_data/alex_val/alex_go_aao_001.aselmdb
 ↪  examples/meta_data/alex_val/alex_go_aao_002.aselmdb -s
 ↪  examples/test_workdir/10_1_configs_extxyz -o extxyz -q 'Cu'
```

Example 3. Find structures with fewer than three H atoms.

```
pwdata convert_configs -i
 ↪  examples/meta_data/alex_val/alex_go_aao_001.aselmdb
 ↪  examples/meta_data/alex_val/alex_go_aao_002.aselmdb -s
 ↪  examples/test_workdir/10_1_configs_extxyz -o extxyz -q 'H<3'
```

Example 4. Find structures containing Cu and fewer than three H atoms.

```
pwdata convert_configs -i
 ↪  examples/meta_data/alex_val/alex_go_aao_001.aselmdb
 ↪  examples/meta_data/alex_val/alex_go_aao_002.aselmdb -s
 ↪  examples/test_workdir/10_1_configs_extxyz -o extxyz -q 'Cu,H<3'
```

Example 5. Find structures with at least two H atoms and at least one O atom.

```
pwdata convert_configs -i
 ↪  examples/meta_data/alex_val/alex_go_aao_001.aselmdb
 ↪  examples/meta_data/alex_val/alex_go_aao_002.aselmdb -s
 ↪  examples/test_workdir/10_1_configs_extxyz -o extxyz -q 'H2O'
```

Additional query expressions:

```
| ---query string---| -------------------------Function
 ↪  Explanation---------------------------- |
| v3                | has  'v3'  key
 ↪  |
| abc=bla-bla       | has key  'abc'  with value  'bla-bla'
 ↪  |
| v3,abc=bla-bla    | both of the above
 ↪  |
| calculator=nwchem | calculations done with NWChem
 ↪  |
| 2.2<bandgap<3.0   |  'bandgap'  key has value between 2.2 and 3.0
 ↪  |
| natoms>=10        | 10 or more atoms
 ↪  |
| id=2345           | specific id
 ↪  |
| age<1h            | not older than 1 hour
 ↪  |
| age>1y            | older than 1 year
 ↪  |
```

```
| pbc=TTT           | Periodic boundary conditions along all three
 ↪  axes                          |
| pbc=TTF           | Periodic boundary conditions along the first two
 ↪  axes (F=False, T=True)    |
https://databases.fysik.dtu.dk/ase/ase/db/db.html#id7
```

pwdata infers formats and searches dataset directories automatically. For `pwmlff/npy`, `extxyz`, `deepmd/npy`, `deepmd/raw`, and Meta OMAT24, specify only the data root.

If no format is specified, every supported data source under the input directory is included.

7.2.5 pwdata scale_cell: Scale a Lattice

Use `scale_cell` or `scale`. Options:

```
pwdata scale_cell [-h] -r SCALE_FACTOR [SCALE_FACTOR ...] -i INPUT -f
 ↪  INPUT_FORMAT [-s SAVENAME] [-o OUTPUT_FORMAT] [-c] [-t ATOM_TYPES
 ↪  [ATOM_TYPES ...]]
```

7.2.5.1 -h Displays all options and descriptions.

7.2.5.2 -r Required space-separated scale factors in `(0,1)`, with $Lattice_{new} = factor \times Lattice_{old}$. For example, `-r 0.97 0.98 0.99` creates three scaled structures.

7.2.5.3 -i Required input path, absolute or relative.

7.2.5.4 -f Optional input format, inferred by default. Supported values: `pwmat/config`, `vasp/poscar`, `lammps/lmp`, and `cp2k/scf`.

7.2.5.5 -o Optional output format: `pwmat/config`, `vasp/poscar`, or `lammps/lmp`. Defaults to the input format, except `cp2k/scf` defaults to `pwmat/config`.

7.2.5.6 -s Optional base output filename used with `-o`; the scale factor is prefixed. Defaults are `atom.config`, POSCAR, and `lammps.lmp`. Example: `-o pwmat/config -s atom.config -r 0.99` writes `0.99_atom.config`.

7.2.5.7 -c Use `-c` for Cartesian coordinates; otherwise fractional coordinates are used. PWmat always uses fractional coordinates.

7.2.5.8 -t For LAMMPS input, specifies atom types as symbols or atomic numbers in input order.

See the repository' s examples:

```
# scale_cell example
pwdata scale_cell -r 0.98 0.99 0.97 0.95 -i
 ↪  examples/pwmat_data/lisi_atom.config -f pwmat/config -s
 ↪  examples/test_workdir/scale_atom.config -o pwmat/config

# or the abbreviated command
pwdata scale -r 0.98 0.99 0.97 0.95 -i
 ↪  examples/pwmat_data/lisi_atom.config -f pwmat/config -s
 ↪  examples/test_workdir/scale_atom.config -o pwmat/config

# Creates four scaled files under examples/test_workdir
```

7.2.6 pwdata super_cell: Build a Supercell

Use `super_cell` or `super`. Options:

```
pwdata super_cell [-h] -m SUPERCELL_MATRIX [SUPERCELL_MATRIX ...] -i
 ↪  INPUT -f INPUT_FORMAT [-s SAVENAME] [-o OUTPUT_FORMAT] [-c] [-p
 ↪  PERIODICITY [PERIODICITY ...]] [-l TOLERANCE] [-t ATOM_TYPES
 ↪  [ATOM_TYPES ...]]
```

7.2.6.1 -h Displays all options and descriptions.

7.2.6.2 -m Required 3×3 supercell matrix, supplied as three or nine values. `-m 2 0 0 0 2 0 0 0 2` and `-m 2 2 2` both define a 2×2×2 supercell.

7.2.6.3 -i Required input path.

7.2.6.4 -f Optional input format, inferred by default: `pwmat/config`, `vasp/poscar`, `lammps/lmp`, or `cp2k/scf`.

7.2.6.5 -o Optional output format: `pwmat/config`, `vasp/poscar`, or `lammps/lmp`. Defaults to input format, except CP2K defaults to PWmat.

7.2.6.6 -s Optional output filename used with `-o`. Defaults are `atom.config`, POSCAR, and `lammps.lmp`. Example: `-o pwmat/config -s super_atom.config`.

7.2.6.7 -c Use -c for Cartesian coordinates; otherwise fractional coordinates are used. PWmat always uses fractional coordinates.

7.2.6.8 -t For LAMMPS input, specifies atom types in input order.

7.2.6.9 -p Optional periodic boundaries. `[1,1,1]` (default) is periodic in x, y, and z.

7.2.6.10 -l Optional fractional-coordinate tolerance. The default is `1e-5` and prevents tiny negative coordinates from being mapped incorrectly.

See the repository's examples:

```
# super_cell example
pwdata super_cell -m 2 3 4 -i examples/pwmat_data/lisi_atom.config -s
 ↪  examples/test_workdir/super_atom.config -o pwmat/config

# or the abbreviated command
pwdata super -m 2 0 0 0 3 0 0 0 4 -i
 ↪  examples/pwmat_data/lisi_atom.config -s
 ↪  examples/test_workdir/super_atom.config -o pwmat/config

# Creates examples/test_workdir/super_atom.config with a 2×3×4
 ↪  supercell
```

7.2.7 pwdata perturb: Perturb a Lattice and Atomic Positions

Perturbs a structure's lattice or atomic positions. Options:

```
pwdata perturb [-h] [-d ATOM_PERT_DISTANCE] [-e CELL_PERT_FRACTION]
 ↪  -n PERT_NUM -i INPUT -f  INPUT_FORMAT [-s SAVENAME] [-o
 ↪  OUTPUT_FORMAT] [-c] [-t ATOM_TYPES [ATOM_TYPES ...]]
```

7.2.7.1 -h Displays all options and descriptions.

7.2.7.2 -d Optional atomic perturbation distance in ångströms. Each coordinate is shifted by a value sampled uniformly from `[-atom_pert_distance, atom_pert_distance]`. The default `0` disables atomic perturbation.

7.2.7.3 -e Optional cell-deformation fraction. Each of the nine lattice values is perturbed by a uniformly sampled value in `[-cell_pert_fraction, cell_pert_fraction]`. For example, `0.03` represents approximately 3% deformation. The default `0` disables lattice perturbation.

7.2.7.4 -n Required number of perturbed structures to generate.

7.2.7.5 -i Required input path.

7.2.7.6 -f Optional input format, inferred by default: `pwmat/config`, `vasp/poscar`, `lammps/lmp`, or `cp2k/scf`.

7.2.7.7 -o Optional output format: `pwmat/config`, `vasp/poscar`, or `lammps/lmp`. Defaults to input format, except CP2K defaults to PWmat.

7.2.7.8 -s Optional output filename used with `-o`. Defaults are `atom.config`, POSCAR, and `lammps.lmp`.

7.2.7.9 -c Use `-c` for Cartesian coordinates; otherwise fractional coordinates are used. PWmat always uses fractional coordinates.

7.2.7.10 -t For LAMMPS input, specifies atom types in input order.

See the repository's examples:

```
# perturb example
pwdata perturb -e 0.01 -d 0.04 -n 20 -i
 ↪  examples/pwmat_data/lisi_atom.config -f pwmat/config -s
 ↪  examples/test_workdir/perturb_atom -o pwmat/config
# Generates 20 perturbed structures under
 ↪  examples/test_workdir/pertub_atom
```

7.2.8 pwdata count: Count Structures

Counts structures in supported trajectories and datasets, including `MOVEMENT`, `OUTCAR`, `LAMMPS dump`, `CP2K MD`, `pwmlff/npy`, `extxyz`, `deepmd/npy`, `deepmd/raw`, and `Meta OMAT24`.

Options:

```
pwdata convert_configs [-h] [-h] -i INPUT [INPUT ...] [-f
 ↪  INPUT_FORMAT] [-q QUERY] [-n CPU_NUMS] [-t ATOM_TYPES [ATOM_TYPES
 ↪  ...]]
```

7.2.8.1 -h Displays all options and descriptions.

7.2.8.2 -i Required list of input files or directories. For pwmlff/npy, extxyz, deepmd/npy, deepmd/raw, and Meta OMAT24, specify only the data root; pwdata searches recursively.

For example, to count all datasets under examples/pwmlff_data/LiSiC, use -i examples/pwmlff_data.

For many inputs, list their paths in JSON and pass the file with -i.

```
{
    "datapath" : [
        "/share/public/PWMLFF_test_data/eqv2-
        ↪ models/datasets/decompress/Omat24/train/rattled-1000-
        ↪ subsampled",
        "/share/public/PWMLFF_test_data/eqv2-
        ↪ models/datasets/decompress/Omat24/train/rattled-300",
        "/share/public/PWMLFF_test_data/eqv2-
        ↪ models/datasets/decompress/Omat24/train/rattled-300-
        ↪ subsampled"
    ]
}
```

7.2.8.3 -f Optional input format, inferred by default. Supported values are pwmat/movement, vasp/outcar, lammps/dump, cp2k/md, pwmlff/npy, deepmd/npy, deepmd/raw, extxyz, and meta.

7.2.8.4 -q Optional query string for Meta input. See the Meta query examples.

7.2.8.5 -n Optional CPU count for parallel Meta queries. The default is one core.

7.2.8.6 -t For LAMMPS input, specifies atom types in input order.

For Meta input, selects structures containing only the specified element types.

7.3 Using pwdata as a Python Library

Use the pwdata API to generate datasets or convert data programmatically.

7.3.1 pwdata Config: Read Data

```
Config (self, format: str, data_path: str, pbc = None, atom_names =
 ↪ None, index = ':', **kwargs)
```

Parameters: - **format**: String. Input format: `pwmat/config`, `vasp/poscar`, `lammps/dump`, `lammps/lmp`, `pwmat/movement`, `vasp/outcar`, `cp2k/md`, `cp2k/scf`, `pwmlff/npy`, `deepmd/npy`, `deepmd/raw`, `extxyz`, or `meta`.

- `pwmat/config`: PWmat structure, e.g. `atom.config`.
- `pwmat/movement`: PWmat MD trajectory, e.g. `MOVEMENT`.
- `lammps/dump`: LAMMPS dump, e.g. `dump.lammptraj`.
- `lammps/lmp`: LAMMPS structure, e.g. `in.lmp`.
- `vasp/poscar`: VASP structure, e.g. POSCAR.
- `vasp/outcar`: VASP MD trajectory, e.g. OUTCAR.
- `cp2k/md`: CP2K output plus positions and forces, e.g. `cp2k.out`, `*pos-1.xyz`, `*pos-1.pdb`, or `*frac-1.xyz`.
- `cp2k/scf`: CP2K output, e.g. `cp2k.out`.
- `pwmlff/npy`: MatPL dataset, e.g. `energies.npy`.
- `deepmd/npy`: DeepMD dataset, e.g. `force.npy`.
- `deepmd/raw`: DeepMD dataset, e.g. `force.raw`.
- `extxyz`: Extended XYZ, e.g. `*.xyz`.
- `meta`: Open Meta data with `.aselmdb` extension.

Set `PRINT_LEVEL MEDIUM` in CP2K input so the standard output contains lattice information.

- **data_path**: String, **required**. Input path.
- **pbc**: Optional list of periodic boundaries. Default `None`; `[1,1,1]` is periodic in x, y, and z.
- **atom_names**: Optional list of atom names for `lammps/dump`, e.g. `['C','H','O']`. Default `None`.
- **index**: Optional integer, slice, or string selecting structures from a multi-structure file. Default `:` reads all.
  - `index=0`: first structure
  - `index=-2`: second-to-last structure
  - `index=':'` or `slice(None)`: all structures
  - `index='-3:'` or `slice(-3,None)`: last three structures
  - `index='::2'` or `slice(0,None,2)`: even indices
  - `index='1::2'` or `slice(1,None,2)`: odd indices
- **kwargs**: Optional keyword arguments for the reader.
- **unit**: Optional LAMMPS unit style. Default `metal`.
- **style**: Optional LAMMPS atom style. Default `atomic`. See LAMMPS atom_style.
- **sort_by_id**: Optional Boolean controlling atom sorting by ID for LAMMPS. Default `True`.

Returns: A list of `Image` objects, one per structure.

Example:

```
from pwdata import Config

data_file = "./cp2k.out"
format = "cp2k/scf"
config = Config(format, data_file)
```

7.3.2 pwdata Config.to(): Write Data

Writes loaded data in the requested format.

```
Config.to (self, data_path, format = None, **kwargs)
```

Parameters:

- **data_path**: String, **required**. Output path.
- **format**: String, **required**. Output format: `pwmat/config`, `pwmat/movement`, `vasp/poscar`, `lammps/lmp`, `extxyz`, or `pwmlff/npy`.
- **Kwargs**: Additional output keyword arguments:
- For `pwmat/config`, `vasp/poscar`, `lammps/lmp`, and `extxyz`:
  - **data_name**: String, **required**. Output filename.
  - **sort**: Optional Boolean controlling sorting by atomic number. Default `None`.
  - **wrap**: Optional Boolean controlling wrapping into the simulation cell. Default `False`.
  - **direct**: Boolean, **required**. Selects fractional versus Cartesian coordinates.
- For labeled `pwmlff/npy` output:
  - **data_name**: String, **required**. Dataset-directory name.
  - **random**: Optional Boolean controlling shuffling.
  - **seed**: Optional random seed. Default `2024`.
- For CP2K data, set `sort=False`; CP2K is already sorted and re-sorting corrupts ordering.
- `pwmlff/npy` stores labeled datasets for model training.
- PWmat structures always use fractional coordinates, so `direct` is ignored.
- LAMMPS structures always use Cartesian coordinates, so `direct` is ignored.

Example 1:

```
from pwdata import Config

data_file = "./POSCAR"
format = "vasp/poscar"
config = Config(format, data_file)
config.to(data_path = "./", format = "lammps/lmp", data_name =
 ↪  "lmp.init", direct = False, sort = True)
```

Multiple configurations with the same structure can be combined with `.append()` before calling `.to()`.

Example:

```
from pwdata import Config

raw_data = ["./OUTCAR0", "./OUTCAR1", "./OUTCAR2"]    # the same
 ↪  atoms...
format = "vasp/outcar"
multi_data = Config(format, raw_data[0])
for data in raw_data[1:]:
   image_data = Config(format, data)
   multi_data.append(image_data)
multi_data.to(data_path = "./PWdata", format='pwmlff/npy')
```

Example 2:

Convert `pwmat/movement`, `vasp/outcar`, `cp2k/md`, or `lammps/dump` trajectories to individual `pwmat/config`, `vasp/poscar`, or `lammps/lmp` structures.

```
from pwdata import Config
from pwdata.utils.constant import FORMAT

def trajs2config():
    atom_types = ["Hf", "O"] # for lammps
    input_file =
 ↪  "/data/home/wuxingxing/codespace/pwdata/examples/lmps_data/HfO2/30.lammpstrj
    input_format="lammps/dump"
    save_format = "pwmat/config"
    image = Config(data_path=input_file, format=input_format,
 ↪  atom_names=atom_types)
    tmp_image_data = image.images
    save_dir = "./tmp_test"
    for id, config in enumerate(tmp_image_data):
        savename = "{}_{}".format(id,
 ↪  FORMAT.get_filename_by_format(save_format))
        image.iamges = [config]
```

```
        image.to(data_path = save_dir,
            data_name = savename,
            format = save_format,
            sort = True)

if __name__=="__main__":
    trajs2config()
```

7.3.3 pwdata build.supercells: Supercell Construction

```
build.supercells.make_supercell (image_data, supercell_matrix: list,
 ↪  pbc: list = None, wrap=True, tol=1e-5)
```

Construct a supercell from the input structure and supercell matrix.

Parameters:

- **image_data**: **Required**. An Image object containing information about the original structure.
- **supercell_matrix**: List, **required**. The 3x3 supercell matrix. For example, `[[2, 0, 0], [0, 2, 0], [0, 0, 2]]` creates a 2x2x2 supercell.
- **pbc**: List, optional. Periodic boundary conditions. The default is None. For example, `[1, 1, 1]` makes the system periodic along the x, y, and z directions.
- **wrap**: Boolean, optional. Whether to wrap atoms into the simulation cell when periodic boundary conditions are used. The default is True.
- **tol**: Float, optional. Tolerance for fractional coordinates. The default is 1e-5. It prevents slightly negative coordinates from being wrapped into the simulation cell.

Returns: A new Image object (or list of Image objects) containing information about the supercell.

Example:

```
from pwdata import Config, make_supercell

data_file = "./atom.config"
config = Config('pwmat/config', data_file)
supercell_matrix = [[2, 0, 0], [0, 2, 0], [0, 0, 2]]
supercell = make_supercell(config, supercell_matrix, pbc=[1, 1, 1])
supercell.to(data_path = "./", data_name = "atom_2x2x2.config",
 ↪  format = "pwmat/config", sort = True)
```

7.3.4 pwdata pertub.perturbation: Structure Perturbation

Perturb a structure.

```
pertub.perturbation.perturb_structure (image_data, pert_num:int,
 ↪  cell_pert_fraction:float, atom_pert_distance:float)
```

Parameters:

- **image_data**: **Required**. The Image object to perturb, containing information about the original structure.
- **pert_num**: Integer, **required**. Number of perturbed structures to generate.
- **cell_pert_fraction**: Float, **required**. Magnitude of the cell deformation. For example, `0.03` means that the cell is deformed by 3% relative to the original cell.
- **atom_pert_distance**: Float, **required**. Atomic perturbation distance, in angstroms, that controls how far atoms move from their original positions. For example, `0.01` means that atoms are displaced by 0.01 angstrom.

Returns: A list of new Image objects, each containing information about a perturbed structure.

Example:

```
from pwdata import Config, perturb_structure

data_file = "./atom.config"
config = Config('pwmat/config', data_file)
pert_num = 50
cell_pert_fraction = 0.03
atom_pert_distance = 0.01
save_format = "pwmat/config"
perturbed_structs = perturb_structure(config, pert_num,
 ↪  cell_pert_fraction, atom_pert_distance)
perturbed_structs.to(data_path = "~/pwdata/test/pertubed/",
        data_name = "pertubed",
        format = save_format,
        direct = True,
        sort = True)
```

7.3.5 pwdata pertub.scale: Cell Scaling

```
pertub.scale.scale_cell (image_data, scale_factor:float)
```

Parameters:

- **image_data**: **Required**. The Image object to scale, containing information about the original structure.
- **scale_factor**: Float, **required**. Scaling factor for the cell.

Returns: A new Image object (or list of Image objects), each containing information about a scaled structure.

Example:

```
from pwdata import Config, scale_cell

data_file = "./atom.config"
config = Config('pwmat/config', data_file)
scale_factor = 0.95
scaled_structs = scale_cell(config, scale_factor)
scaled_structs.to(data_path = "~/test/scaled/",
        data_name = "scaled",
        format = "pwmat/config",
        direct = True,
        sort = True)
```

7.3.6 pwdata API Example: Converting MPtraj Files to LMDB Format

The open-source MPtraj dataset is provided in JSON format. After converting it to LMDB, use the `cvt_configs` command with the `-t` and `-q` options to quickly locate selected structures and convert them to extxyz or pwmlff/npy format for training.

```
import json
from pwdata.fairchem.datasets.ase_datasets import LMDBDatabase
from ase import Atoms
from ase.db.row import AtomsRow
from pwdata.utils.constant import get_atomic_number_from_name
from tqdm import tqdm
import numpy as np

def MPjson2lmdb():
    mp_file =
 ↪  "/data/home/wuxingxing/codespace/pwdata/examples/mp_data/mptest.json"
    save_file =
 ↪  "/data/home/wuxingxing/codespace/pwdata/examples/mp_data/sub.aselmdb"
    Mpjson = json.load(open(mp_file))
    db = LMDBDatabase(filename=save_file, readonly=False)
    for key_1, val_1 in tqdm(Mpjson.items(),
     ↪  total=len(Mpjson.keys())):
```

```
        for key_2, val_2 in val_1.items():
            _atomrow, data = cvt_dict_2_atomrow(val_2)
            db._write(_atomrow, key_value_pairs={}, data=data)
    db.close()

def cvt_dict_2_atomrow(config:dict):
    cell = read_from_dict('matrix', config['structure']['lattice'],
↪   require=True)
    atom_type_list = get_atomic_number_from_name([_['label'] for _ in
↪   config['structure']['sites']])
    position = [_['xyz'] for _ in config['structure']['sites']]
    magmom = read_from_dict('magmom', config, require=True)
    atom = Atoms(positions=position,
                 numbers=atom_type_list,
                 magmoms=magmom,
                 cell=cell)

    atom_rows = AtomsRow(atom)
    atom_rows.pbc = np.ones(3, bool)
    # read stress -> xx, yy, zz, yz, xz, xy
    stress = read_from_dict('stress', config, require=True)
    atom_rows.stress =
↪   [stress[0][0],stress[1][1],stress[2][2],stress[1][2],stress[0][2],stress[0][[
    force = read_from_dict('force', config, require=True)
    energy = read_from_dict('corrected_total_energy', config,
↪   require=True)
    atom_rows.__setattr__('force',  force)
    atom_rows.__setattr__('energy', energy)
    data = {}
    data['uncorrected_total_energy'] =
↪   read_from_dict('uncorrected_total_energy', config, default=None)
    data['corrected_total_energy'] =
↪   read_from_dict('uncorrected_total_energy', config, default=None)
    data['energy_per_atom'] = read_from_dict('energy_per_atom',
↪   config, default=None)
    data['ef_per_atom'] = read_from_dict('ef_per_atom', config,
↪   default=None)
    data['e_per_atom_relaxed'] = read_from_dict('e_per_atom_relaxed',
↪   config, default=None)
    data['ef_per_atom_relaxed'] =
↪   read_from_dict('ef_per_atom_relaxed', config, default=None)
    data['bandgap'] = read_from_dict('bandgap', config, default=None)
    data['mp_id'] = read_from_dict('mp_id', config, default=None)
```

```
    return atom_rows, data

def read_from_dict(key:str, config:dict, default=None,
 ↪  require=False):
    if key in config:
        return config[key]
    else:
        if require:
            raise ValueError("key {} not found in config".format(key))
        else:
            return default
if __name__=="__main__":
    MPjson2lmdb()
```

8 MatPL Literature Examples

This chapter collects selected MatPL benchmarks and published studies that used MatPL as references for users.

8.1 Selected Literature

The publications associated with these examples are summarized below. See each corresponding section for details.

1. Comparison of Features

This example compares the ability of the `feature types implemented in MatPL` to represent physical systems. These include `cosine features`, `Gaussian features`, `Moment Tensor Potential (MTP) features`, `Spectral Neighbor Analysis Potential features`, simplified smooth Deep Potential descriptors with `Chebyshev-polynomial` and `Gaussian-polynomial features`, and Atomic Cluster Expansion features. Using a `linear regression model`, the study evaluates training root-mean-square errors (RMSEs) for `atomic-group energies`, `total energies`, and `forces` against density-functional-theory results for `amorphous sulfur` and `carbon` systems. For details about the features, see the Feature Wiki.

For more benchmark details, see the Lonxun WeChat article and the paper [Accuracy evaluation of different machine learning force field features].

2. Simulating Liquid-to-Crystal Silicon Growth with a Machine-Learning Force Field

[Paper: Liquid-to-crystal Si growth simulation using machine learning force field]

This example uses PMLFF to simulate silicon-melt growth far from equilibrium. It shows that an MLFF trained on atomic energies decomposed from first-principles calculations (a `PWmat feature`) can accurately reproduce the growth process observed in first-principles simulations. The work proposes a method for correcting systematic bias during ML-FF training, which is important for accurately predicting key quantities such as phase-transition temperatures. The results demonstrate that an ML-FF can accurately simulate silicon-melt growth and support its use for far-from-equilibrium simulations.

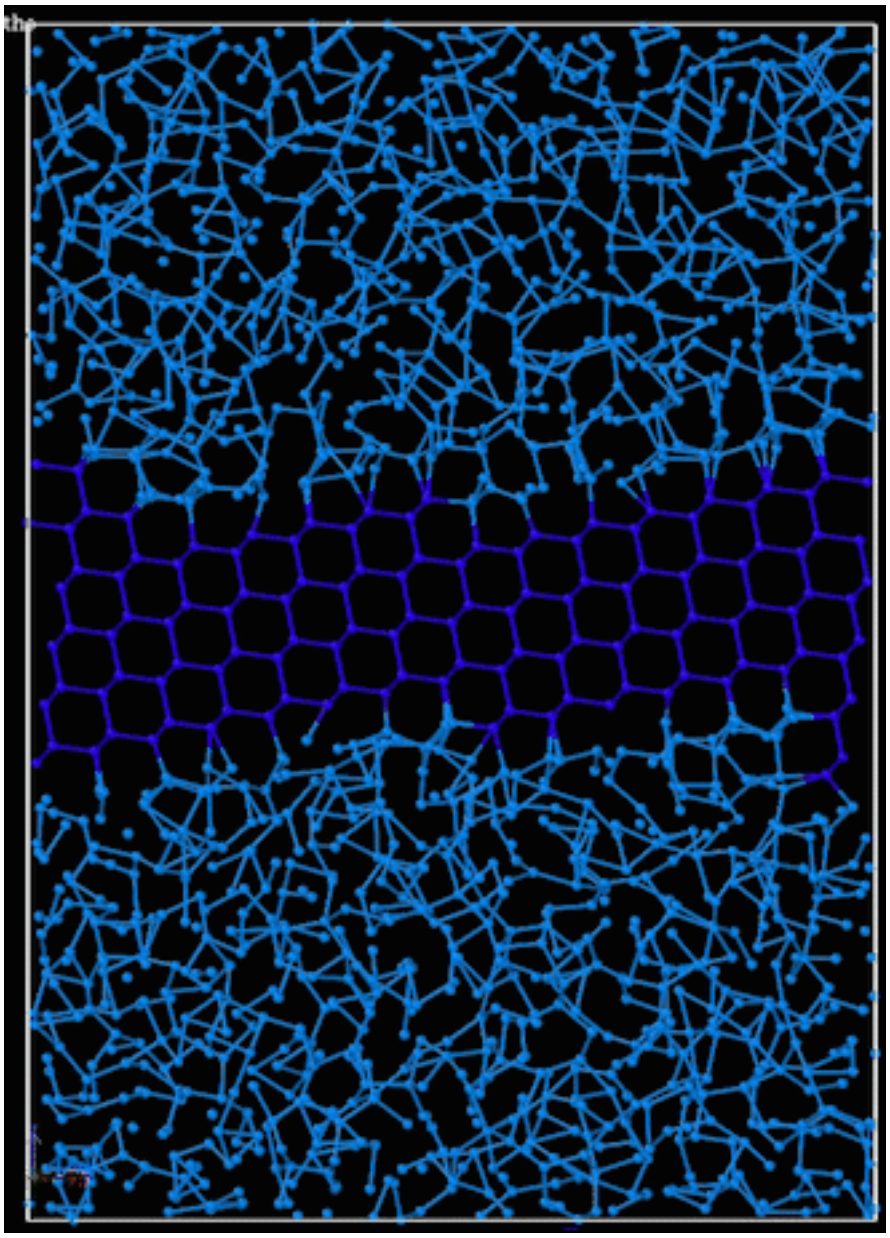


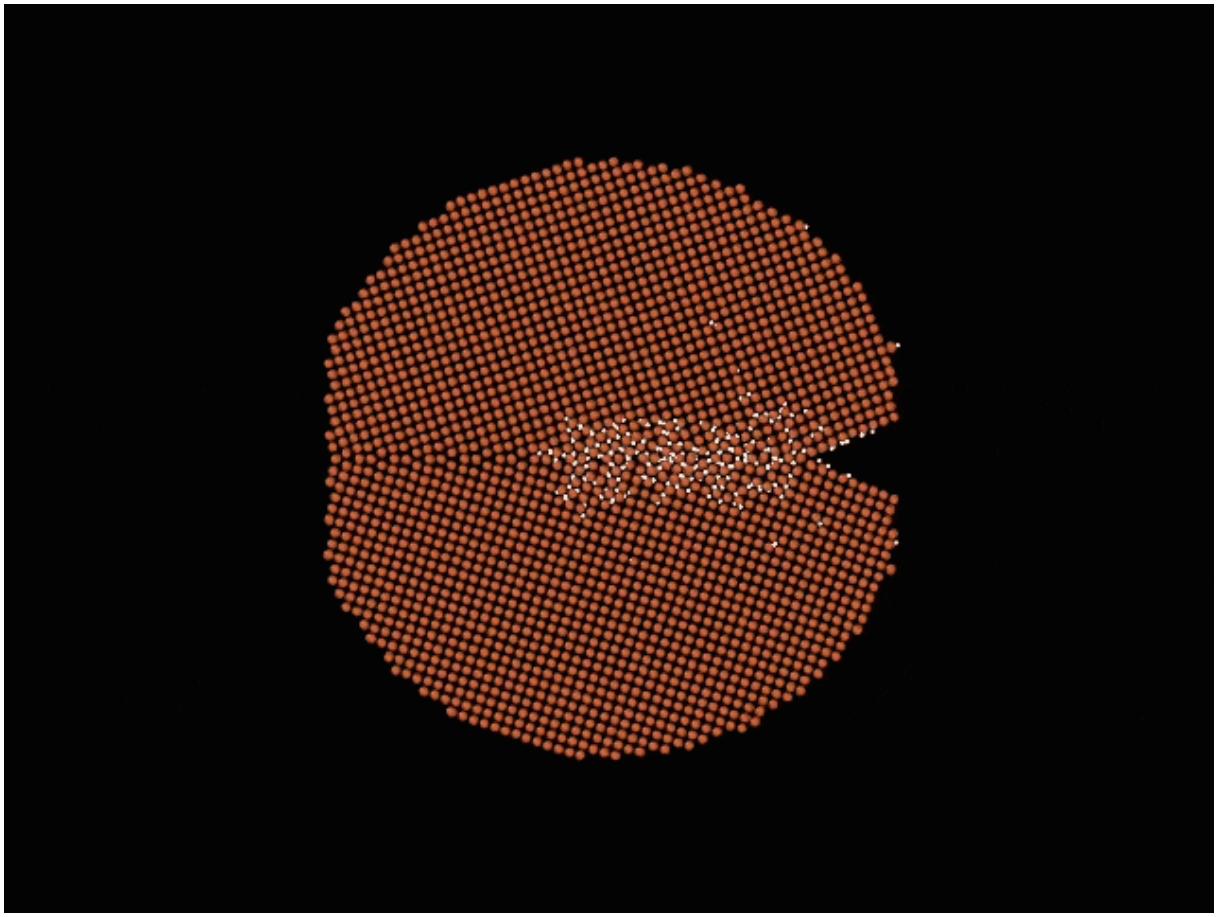
Figure 17: Silicon growth process

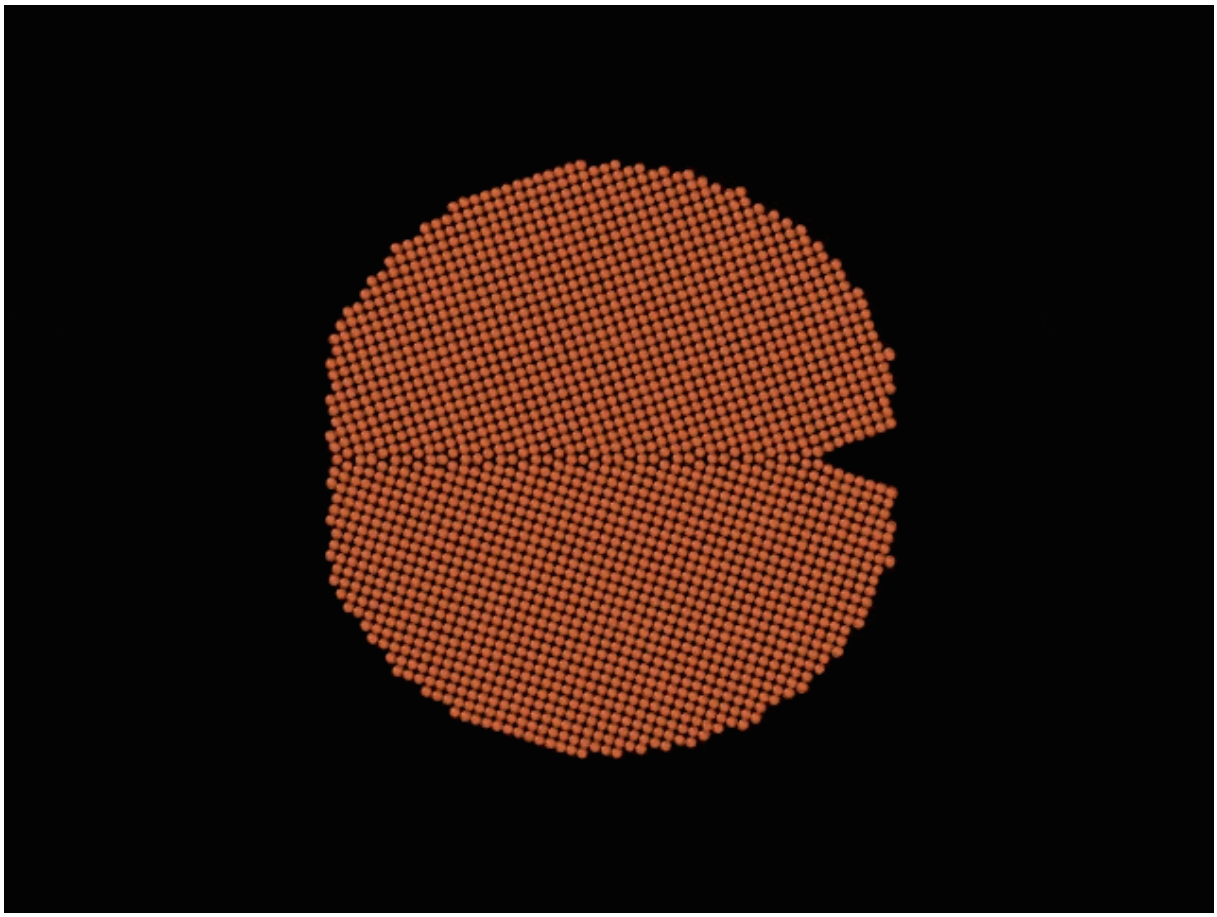
3. Machine-Learning Force Field for Fe–H and the Role of Hydrogen in Crack Propagation in α-Fe

[Paper: Machine learning force field for Fe-H system and investigation on role of hydrogen on the crack propagation in α-Fe]

This example studies how hydrogen affects crack propagation in α-iron using a machine-learning force field. Its main findings are: 1. A neural-network force field for the Fe–H system was trained on atomic energies from density-functional-theory calculations and exhibits good statistical and dynamical properties. 2. Molecular-dynamics simulations show that increasing the hydrogen concentration at a crack tip accelerates crack propagation, indicating that hydrogen promotes cracking. 3. In samples containing grain boundaries, microvoids

form near the crack tip, relieving tensile stress and facilitating crack growth; their formation, however, appears largely unrelated to hydrogen. 4. Crack propagation is faster in structures with a shorter periodic length along the x direction, possibly because of cooperative effects along x. 5. Compared with embedded-atom-potential results, the machine-learning force field reveals a pronounced influence of hydrogen, highlighting the importance of accurately describing hydrogen–metal interactions. 6. Hydrogen accumulation at the crack tip plays a key role in hydrogen-embrittlement crack propagation, motivating further investigation under different conditions.





4. Morphological Evolution of Lithium Dendrites Revealed by Machine-Learning Force-Field MD

For details, see the [Lonxun WeChat article] and the paper [Revealing Morphology Evolution of Lithium Dendrites by Large-Scale Simulation Based on Machine Learning Force Field].

This example combines a machine-learning force field with a self-consistent continuum-solvation model to simulate the morphological evolution of lithium dendrites in an operating electrolyte. The evolution occurs in two stages. In the first, a decrease in surface-atom energy drives local orientational rearrangement of an initially single-crystal dendrite, producing multiple crystalline domains. In the second, a decrease in internal atomic energy drives those domains to slide along grain boundaries and lowers the grain-boundary energy. The study also examines how different exposed-surface orientations affect dendrite morphology. Overall, reductions in surface and grain-boundary energies drive the morphological evolution.

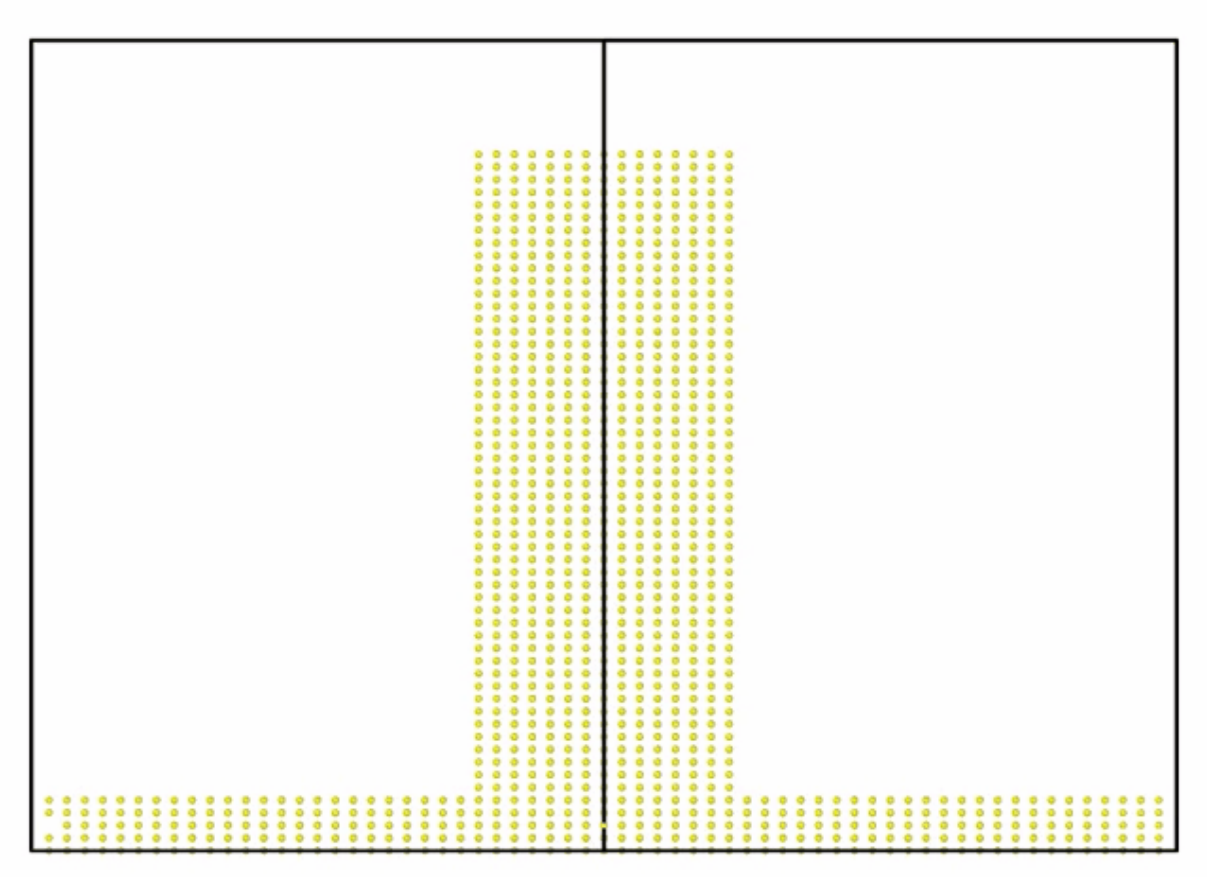


Figure 18: Morphological evolution of lithium dendrites in an operating electrolyte

5. Mg–Cu Alloy Force Field

6. Thermal Conductivity of Amorphous Silicon with a Machine-Learning Force Field

8.2 Comparison of NN Descriptors

[Article: Accuracy evaluation of different machine learning force field features]

This work compares the ability of the `descriptor types implemented in MatPL` to represent physical systems. The descriptors include `cosine descriptors`, `Gaussian descriptors`, `moment tensor potential (MTP) descriptors`, `spectral neighbor analysis potential descriptors`, simplified smooth deep potentials with `Chebyshev-polynomial descriptors` and `Gaussian-polynomial descriptors`, and atomic cluster expansion descriptors. See the Feature Wiki for details.

For sulfur, NVT AIMD simulations were performed at 300 K and 1500 K. A 2 ps simulation at 300 K produced 2,000 structures. At 1500 K, a 3 ps molecular-dynamics simulation was followed by 2 ps of AIMD to generate the training dataset. Sulfur rings broke during the simulations, providing configurations containing broken bonds.

For carbon, four distinct phases were selected for NVT AIMD simulations from 300 K to 3500 K. High-temperature configurations at 3500 K were added to broaden the configuration space. Each phase was simulated for 1,000 steps, producing a total of 4,000 training structures.

System	Description	Temperature (K)	Steps (fs)
Sulfur-300 K	α-S 128 atoms	300	2000
Sulfur-1500 K	128 atoms	1500	2000
Diamond	64 atoms	300–3500	1000
Graphene	64 atoms	300–3500	1000
Graphenylene	64 atoms	300–3500	1000
M-carbon	64 atoms	300–3500	1000

Details and AIMD steps for the sulfur and carbon systems

Selected Results

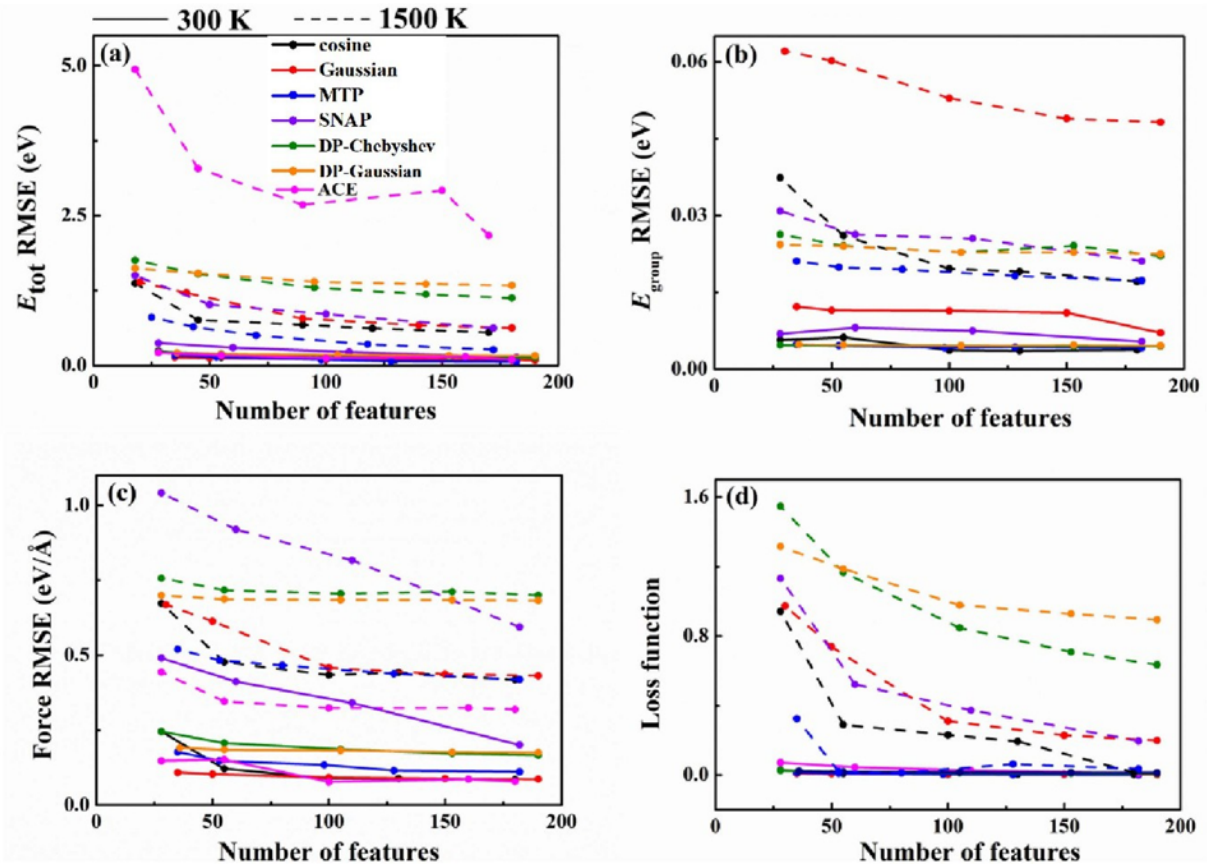


Figure 19: proportion_time

Training errors for different descriptor types on the sulfur-300 K dataset (solid lines) and

sulfur-1500 K dataset (dashed lines): (a) total energy, (b) atomic energy, (c) force, and (d) loss function.

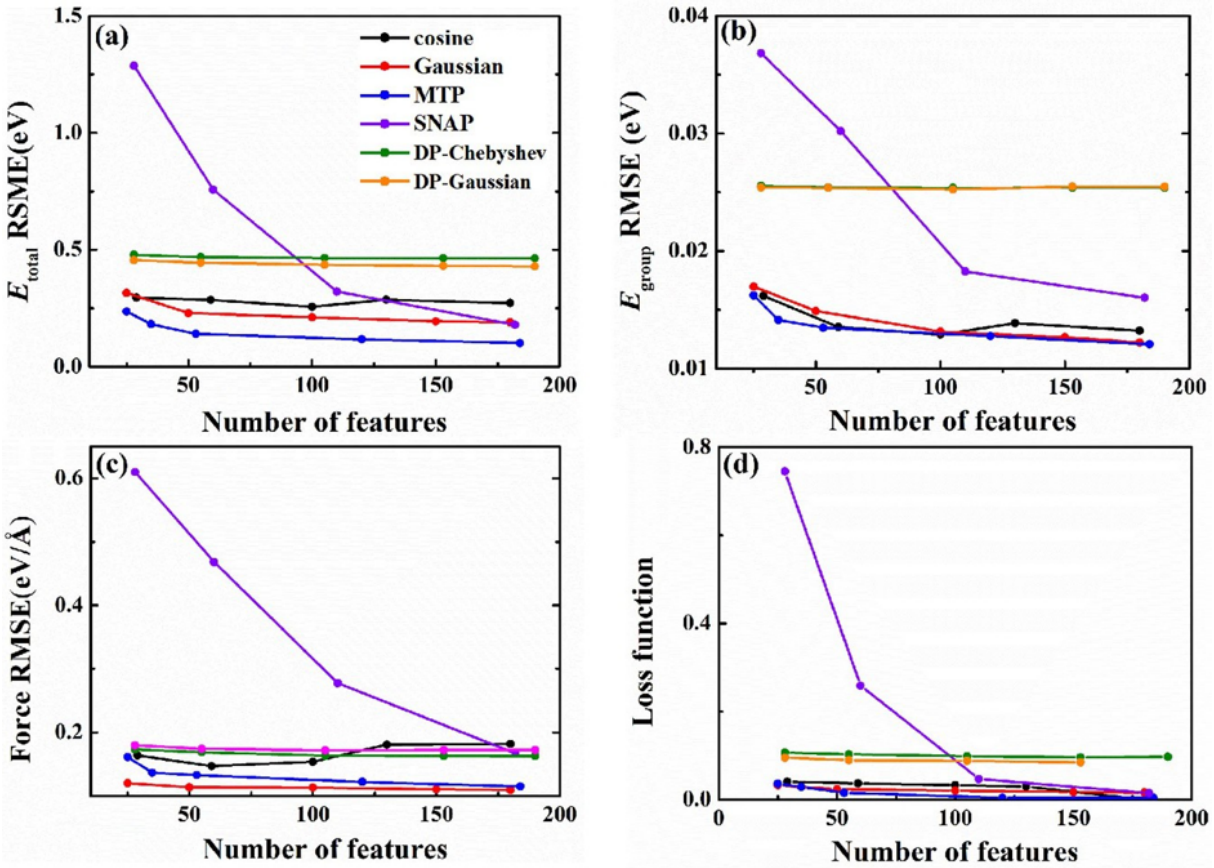


Figure 20: proportion_time

Training errors for different descriptor types on the combined carbon-system dataset: (a) total energy, (b) atomic energy, (c) force, and (d) loss function.

8.3 Silicon Melt-Growth Process

[Reference: Si growth]

This example uses PMLFF to simulate silicon melt growth far from equilibrium. It shows that an MLFF trained with atom-resolved energies derived from first-principles calculations (a `PWmat feature`) can accurately reproduce the growth process observed in first-principles simulations. The results demonstrate the suitability of MLFFs for far-from-equilibrium simulations.

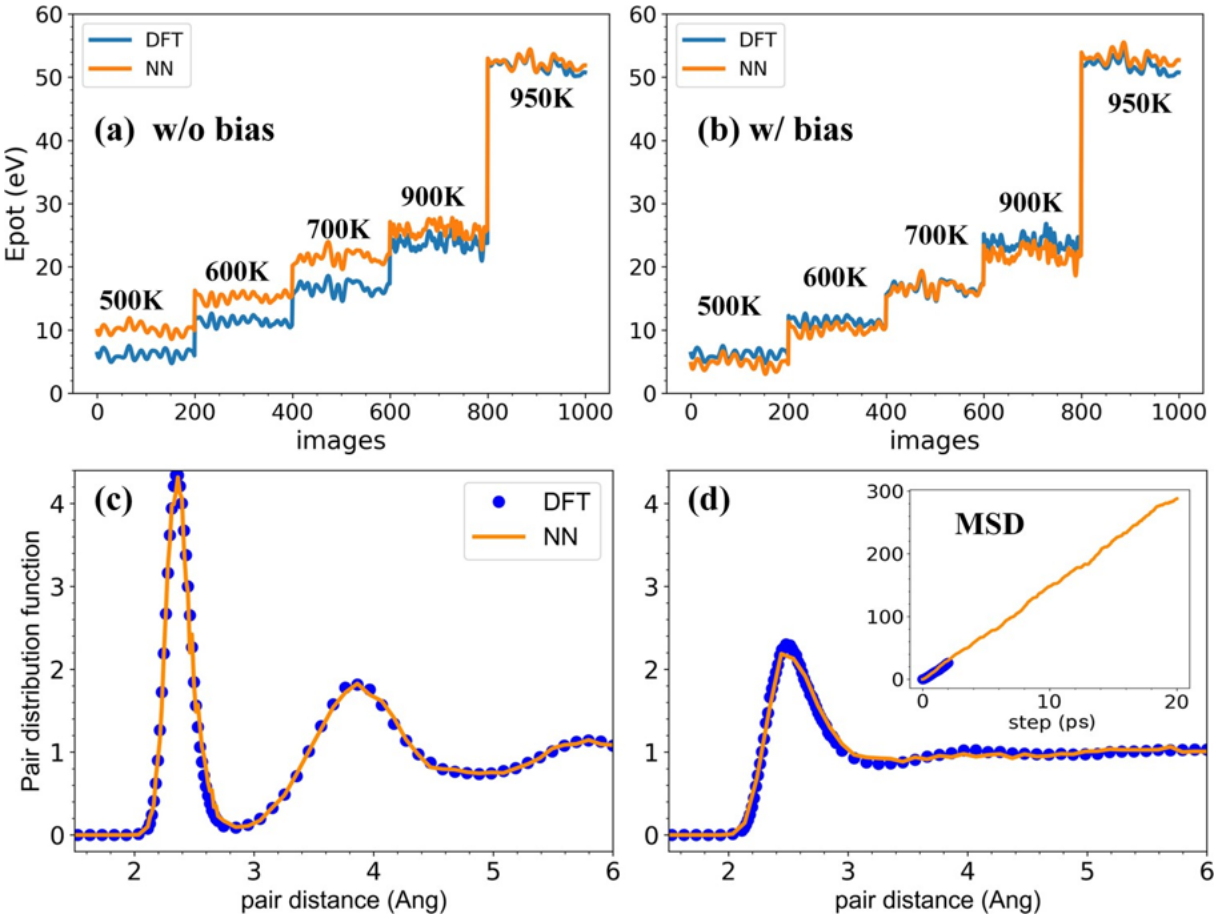


Figure 21: proportion_time

Comparison of total energies from NN and DFT: (a) without an offset and (b) with an offset. The x-axis represents snapshots from crystal-growth molecular-dynamics (MD) simulations at different temperatures. The inset in (b) shows the mean-square displacement of the liquid phase at 1500 K. Panels (c) and (d) show the pair-distribution functions of the crystalline phase at 950 K and the liquid phase at 1500 K, respectively.

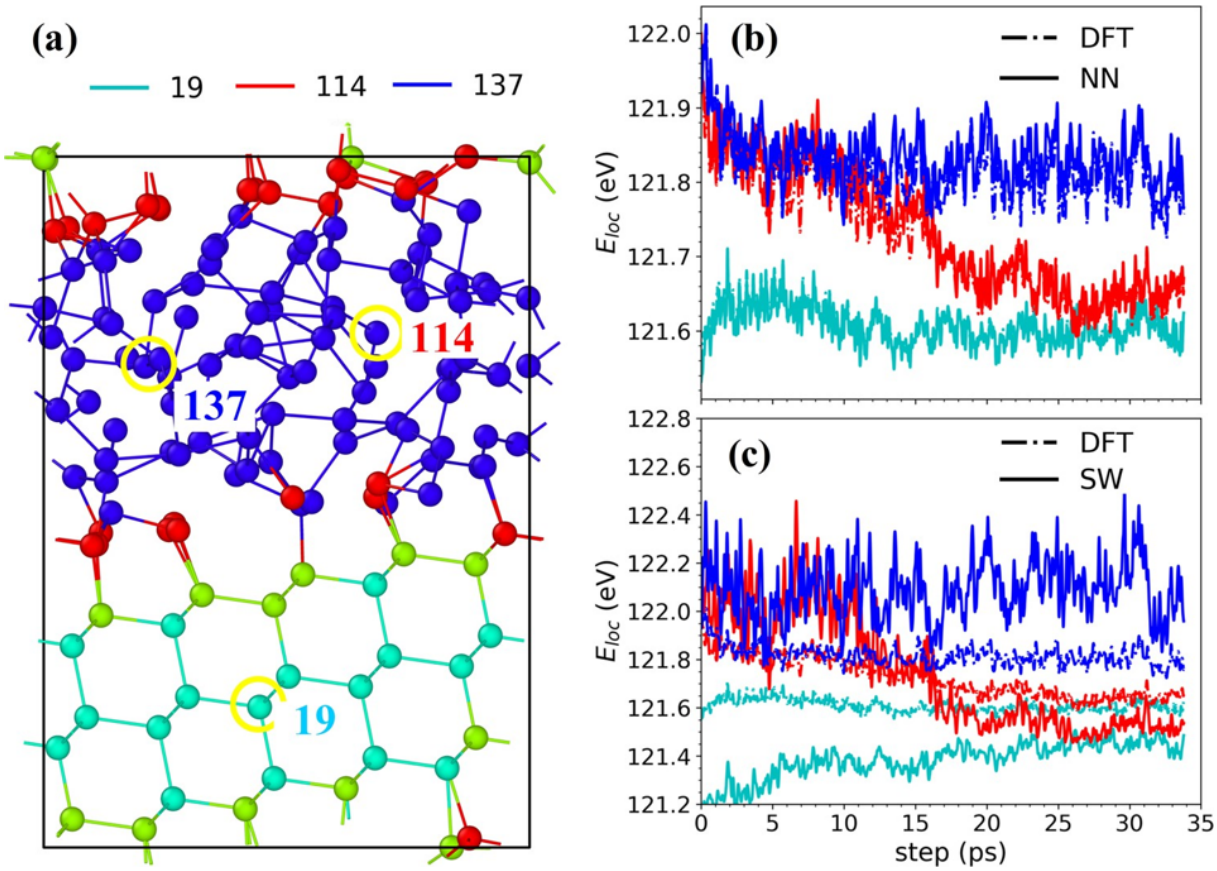


Figure 22: proportion_time

(a) Supercell structure of a crystalline silicon slab with an FCC (111) surface. (b) Comparison of the local energy, Eloc(t), calculated using DFT, the neural network, and the SW potential (the classical Stillinger-Weber force field).

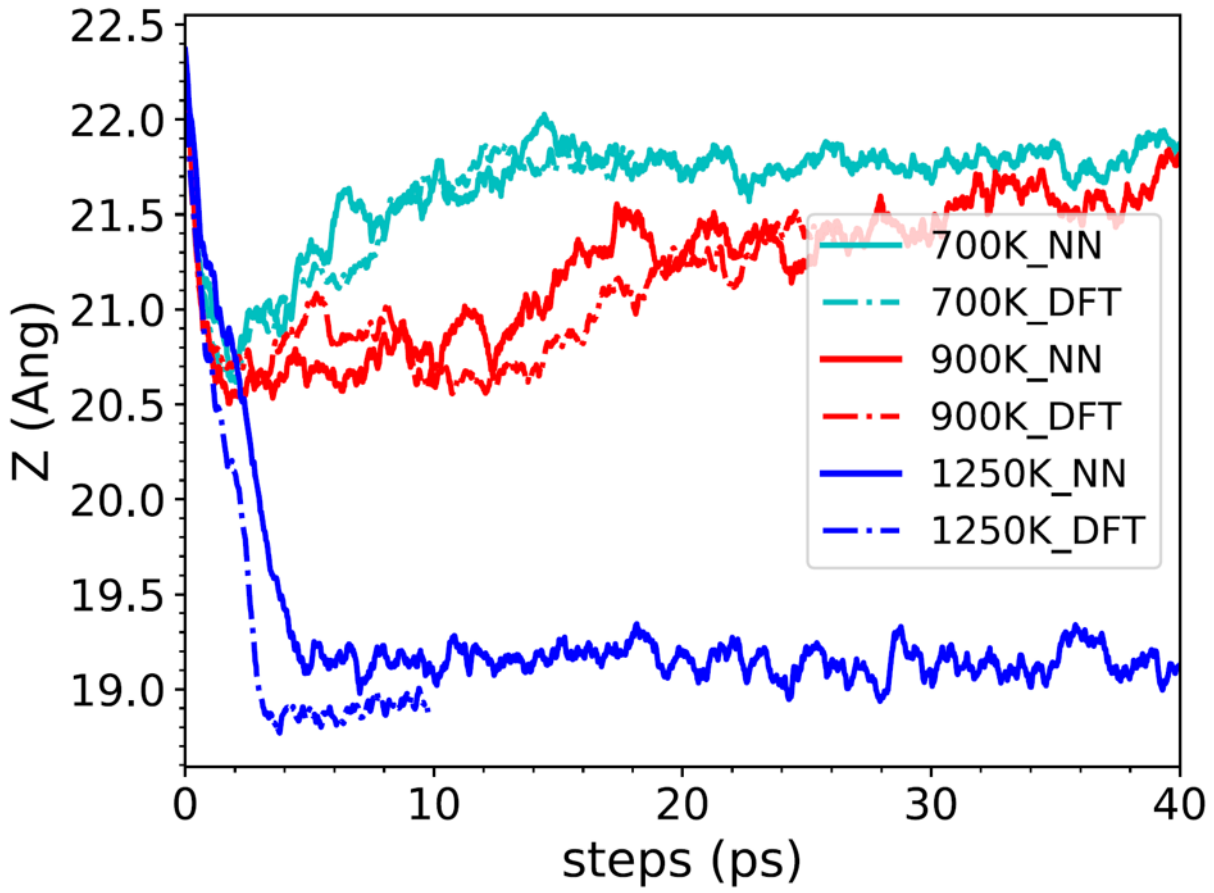


Figure 23: proportion_time

Growth of the Z dimension over time calculated with DFT and NN models (features 1 and 2).

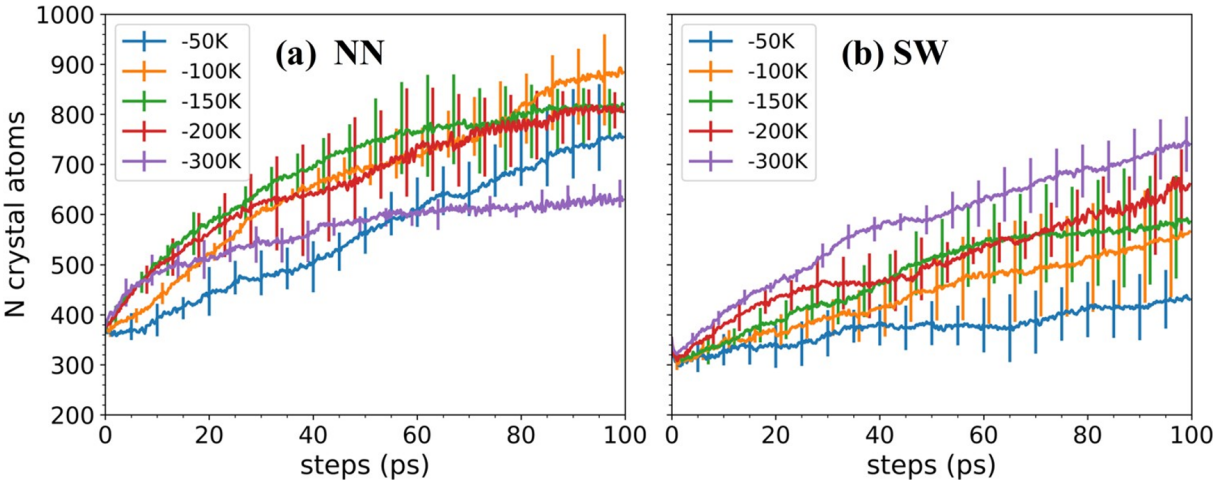


Figure 24: proportion_time

Growth curves (with error bars) showing the number of crystalline atoms at different ΔT values: (a) neural network and (b) SW potential.

8.4 Fe-H System Force Field

[Reference: Machine learning force field for Fe-H system and investigation on role of hydrogen on the crack propagation in α-Fe]

This example examines the effect of hydrogen on crack propagation in α-Fe using a machine-learning force field. The study: (1) constructs a neural-network force field for the Fe-H system from density-functional-theory data and demonstrates sound statistical and dynamical behavior; (2) uses molecular dynamics to show that higher hydrogen concentrations at a crack tip accelerate crack propagation; (3) observes microvoid formation near crack tips in specimens containing grain boundaries, which relieves tensile stress and promotes propagation, although microvoid formation appears only weakly related to hydrogen; (4) finds faster propagation in structures with shorter periodicity along x, possibly because of cooperative effects; (5) shows a stronger hydrogen effect than embedded-atom models, highlighting the importance of accurately describing hydrogen-metal interactions; and (6) identifies hydrogen accumulation at crack tips as a key factor in hydrogen-embrittlement cracking.

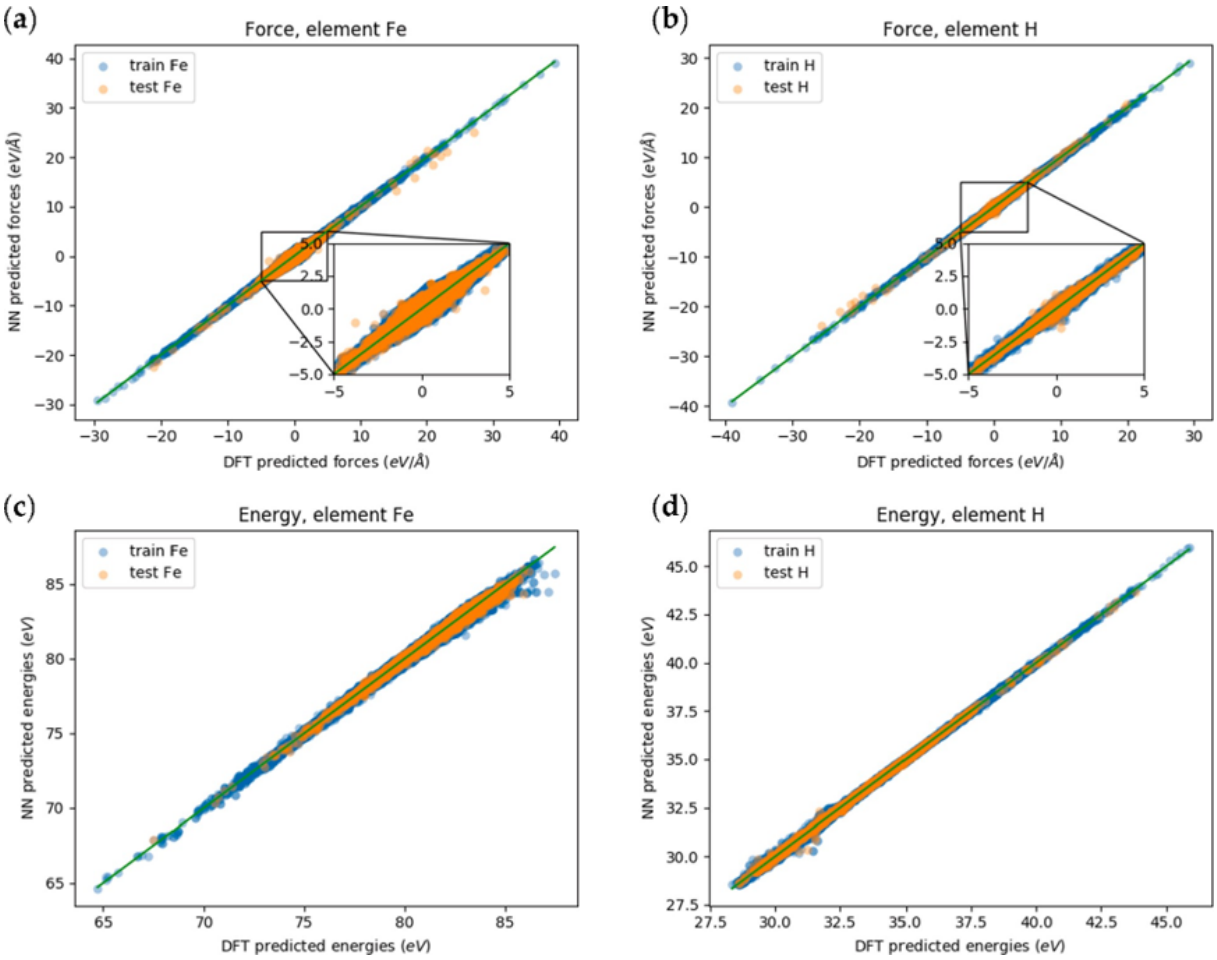


Figure 25: proportion_time

Model-fitting accuracy.

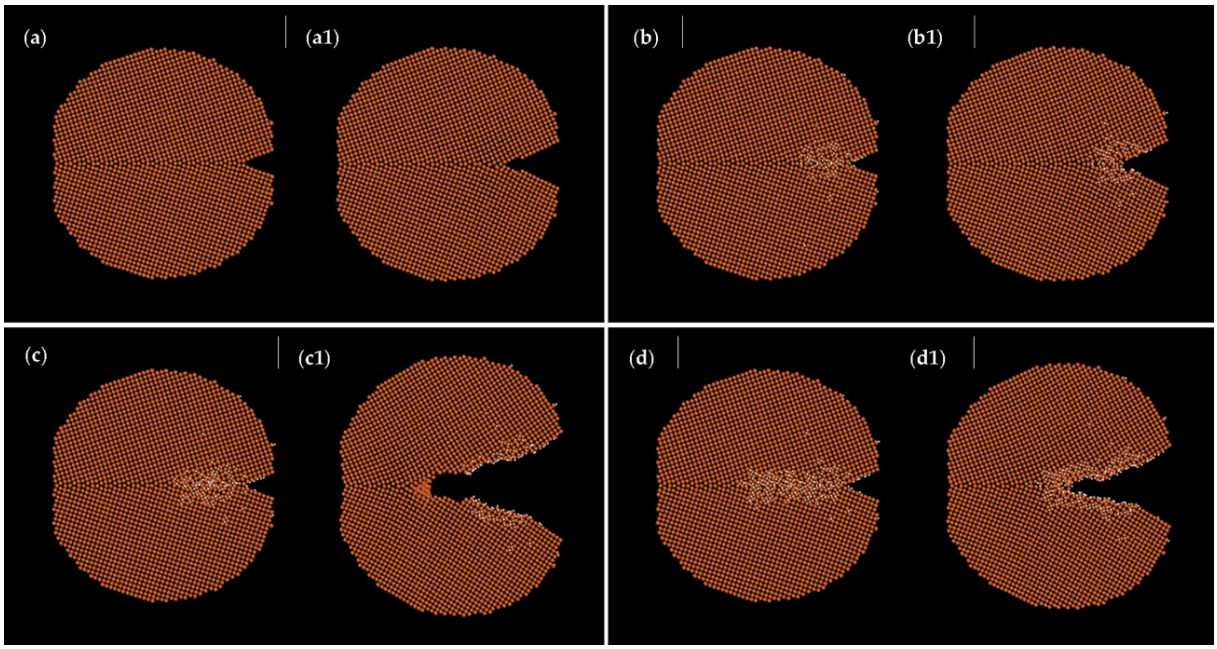


Figure 26: proportion_time

Molecular-dynamics simulations of a system with a tilted grain boundary. Panels (a, a1) show the initial and final frames without hydrogen; (b, b1), (c, c1), and (d, d1) show the initial and final frames at total hydrogen concentrations of 0.709%, 1.097%, and 1.856%, respectively.

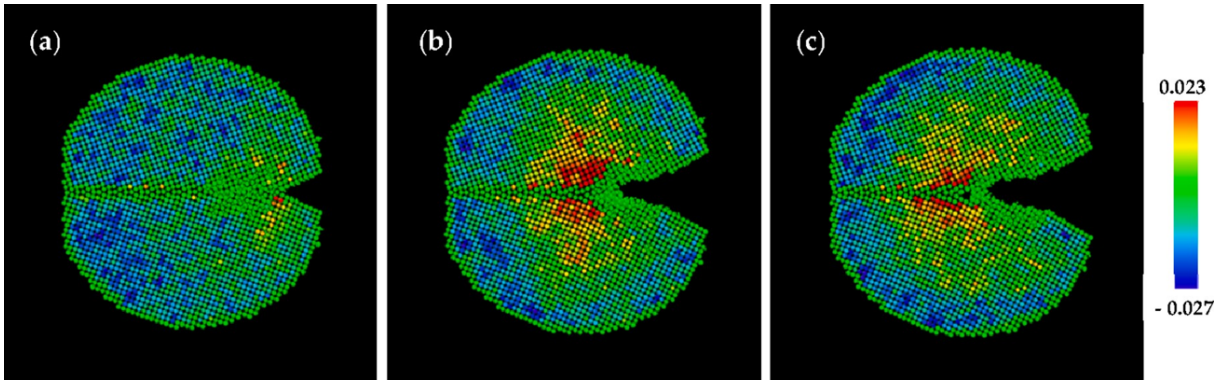


Figure 27: proportion_time

Volumetric-strain distribution for the system with a total hydrogen concentration of 1.097%: (a) initial frame and (b) frame at approximately 6000 fs.

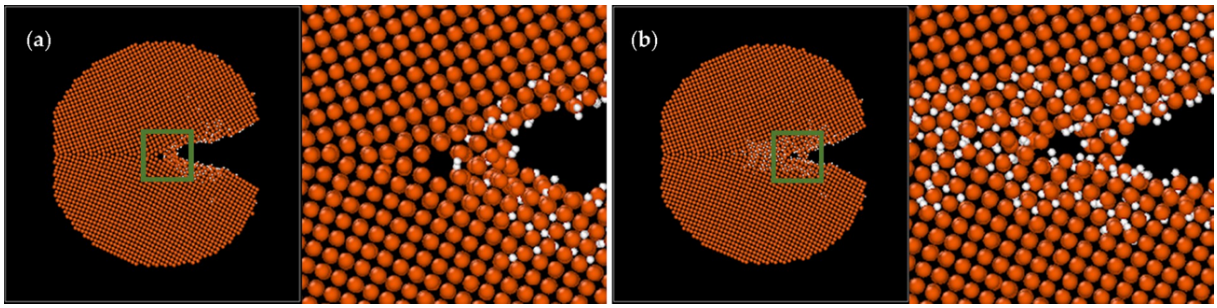


Figure 28: proportion_time

Volumetric-strain distribution for the system with a total hydrogen concentration of 1.097%: (a) initial frame and (b) frame at approximately 6000 fs.

For additional results, see Machine learning force field for Fe-H system and investigation on role of hydrogen on the crack propagation in α-Fe.

8.5 Morphological Evolution of Lithium Dendrites

For details, see the [Lonxun WeChat article] and [Revealing Morphology Evolution of Lithium Dendrites by Large-Scale Simulation Based on Machine Learning Force Field].

Lithium-ion batteries offer high energy density and are widely used in electric vehicles and large-scale energy-storage systems. Lithium-metal anodes combine the lowest chemical potential with high capacity and are therefore promising for next-generation batteries with higher specific energy. During repeated charge-discharge cycles, however, lithium-metal anodes undergo dendrite growth and severe volume changes. Dendrites reduce Coulombic efficiency, energy density, and stability; once sufficiently long, they may pierce the separator, contact the cathode, and cause a short circuit or fire. Controlling dendrite growth is therefore essential to the development of lithium-metal anodes.

Advances in transmission electron microscopy (TEM) allow direct observation of lithium-dendrite morphology and phase structure. Nevertheless, limited spatial and temporal resolution leaves the dynamics of morphological evolution poorly understood. Existing simulations typically use phase-field methods or empirical force fields, whose limited accuracy makes it difficult to predict realistic dendrite morphologies. Large-scale simulations with atomistic accuracy are therefore needed.

Machine-learning methods trained on quantum-chemical calculations provide both accuracy and speed. A force field trained on accurate small-system data can extend simulations of alkali metals and other materials toward mesoscopic or even macroscopic scales. Potential-energy-surface data may also come from density-functional-theory calculations that represent realistic environments, such as lithium atoms in an implicit electrolyte, making realistic dendrite-evolution simulations possible.

This example uses molecular dynamics driven by a `MatPL machine-learning force field` to simulate the morphological evolution of lithium dendrites in an electrolyte environment. It reveals the dendrite-growth mechanism and supports the development of lithium-metal anode materials.

Accelerating model development with DFT-accuracy atomic-energy labels

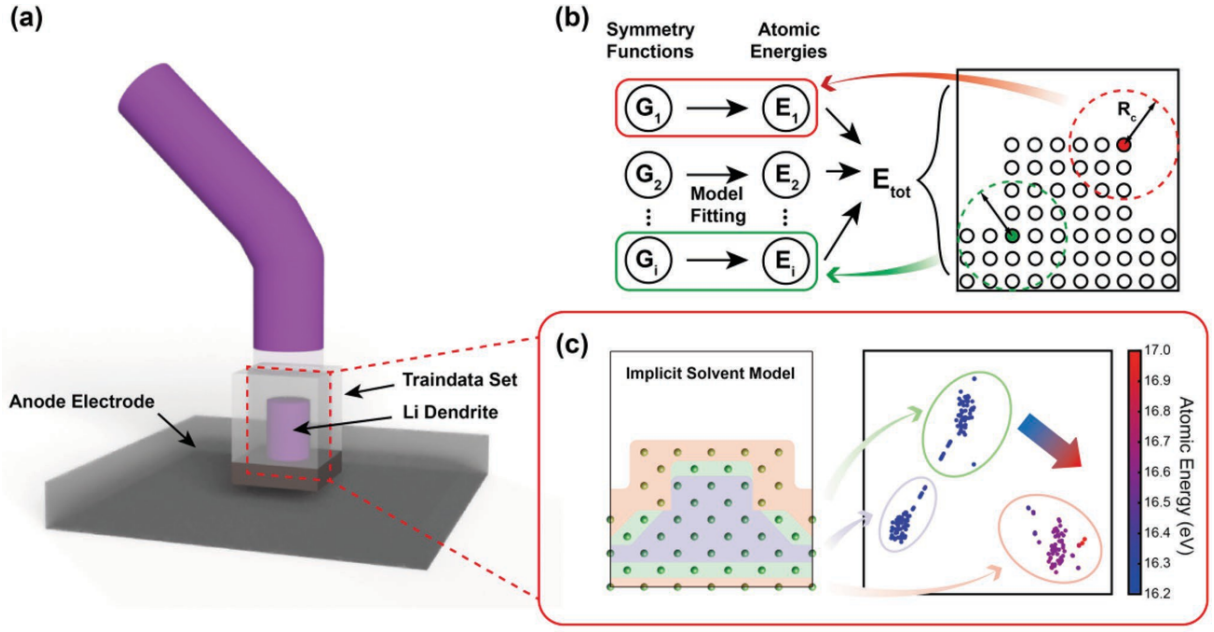


Figure 29: proportion_time

(a) Composition of the small-scale dendrite dataset; (b) MLFF architecture; and (c) cross-sectional view of the body-centered-cubic structure and visualization of descriptor-atomic-energy relationships.

Active-learning and validation strategy for cross-scale force-field applications

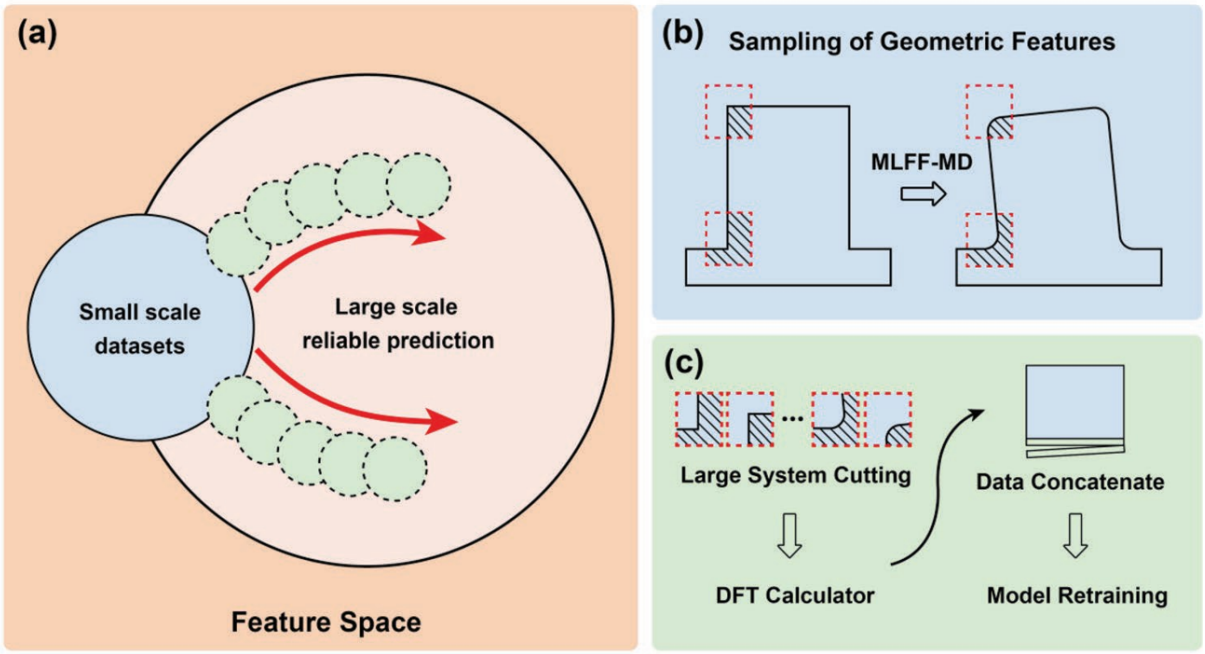


Figure 30: proportion_time

Schematic of active learning for cross-scale simulations: (a) data sampling during active-learning model expansion; (b) sampling key changing regions from MLFF molecular-dynamics trajectories; and (c) DFT labeling of the selected regions followed by model retraining.

Morphological evolution and driving-force analysis for dendrites with different initial configurations

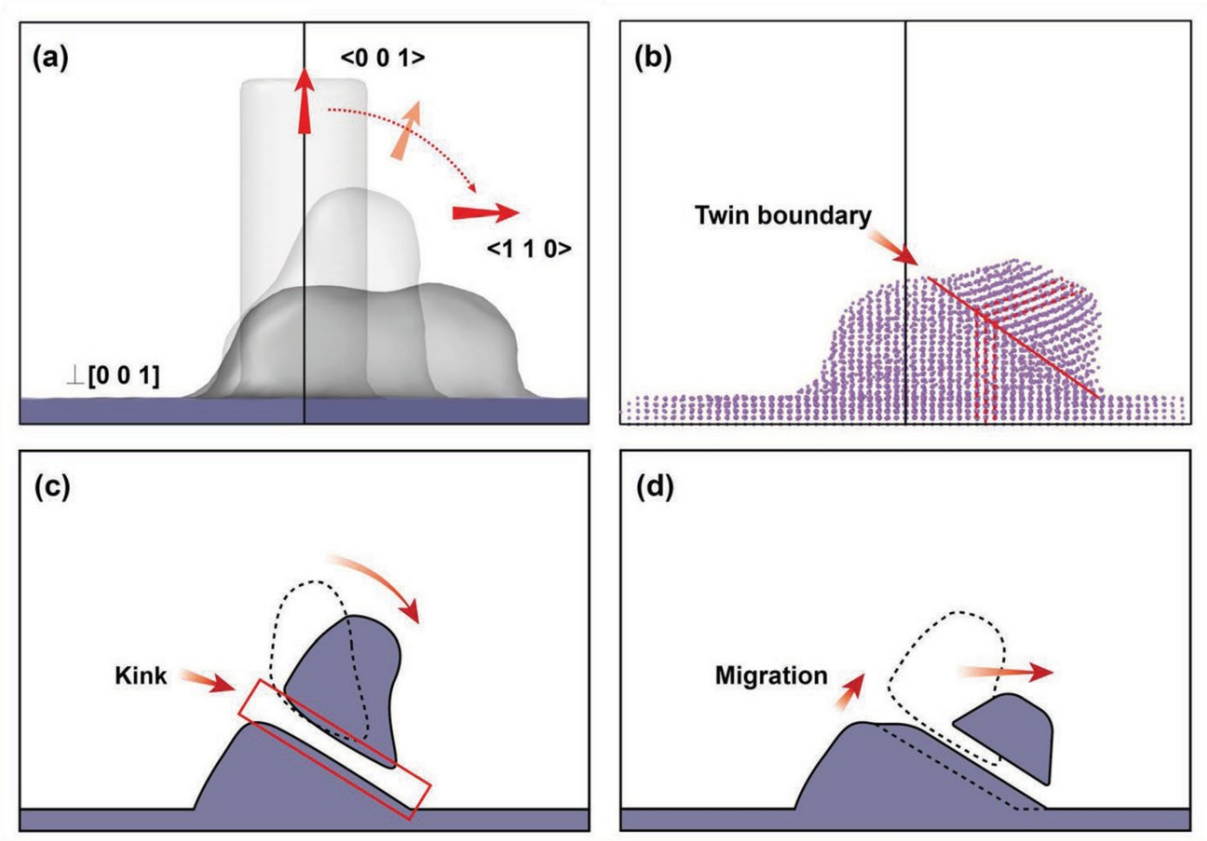


Figure 31: proportion_time

Morphological evolution of a cylindrical structure

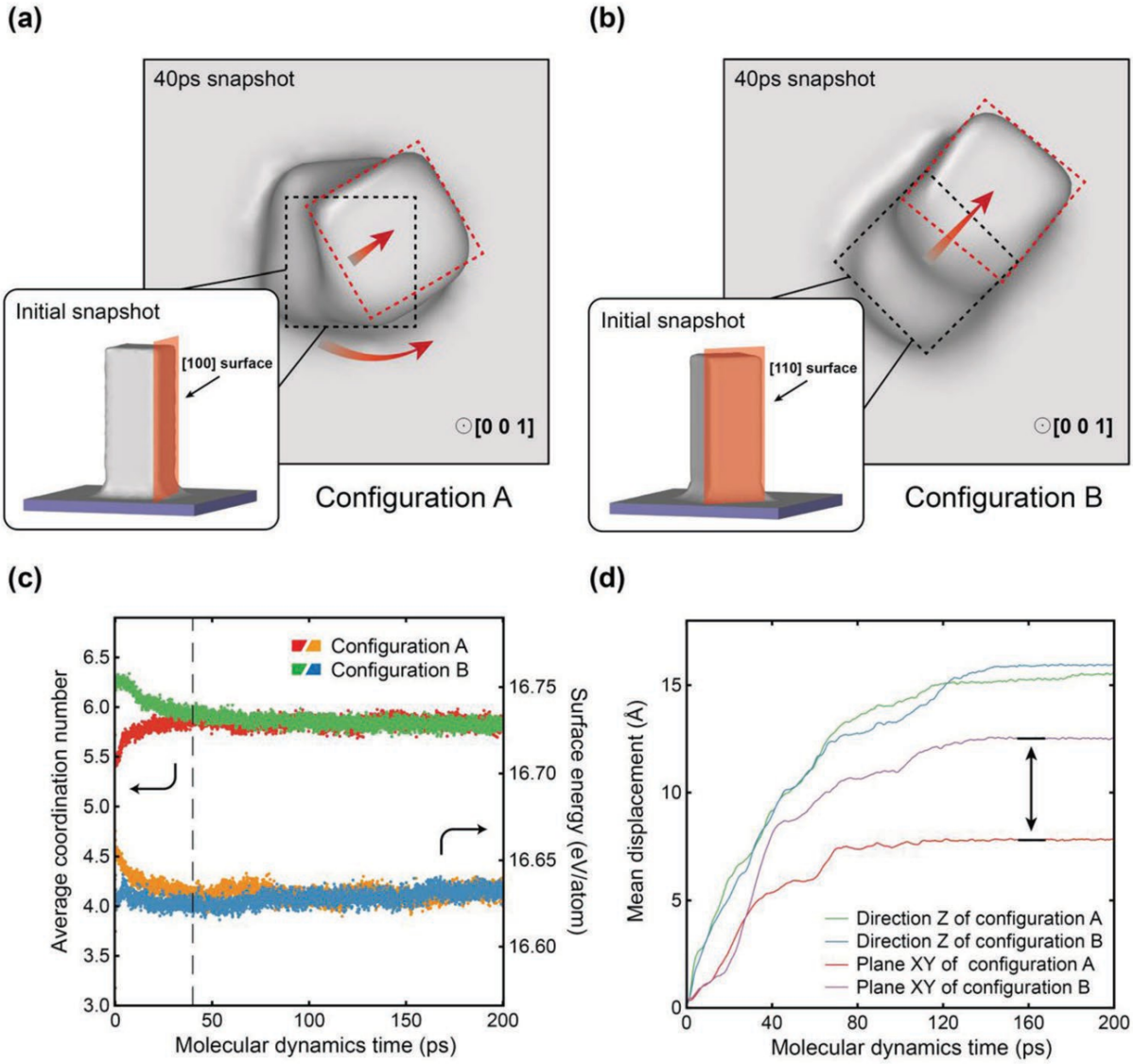


Figure 32: proportion_time

Morphological evolution of rectangular structures with different exposed surfaces

8.6 Mg-Cu Alloy Force Field

Xingze Gen, Lin-Wang Wang, and Xiangying Meng. Comput. Mat. Sci. 246 (2025) 113486

Fitting Accuracy

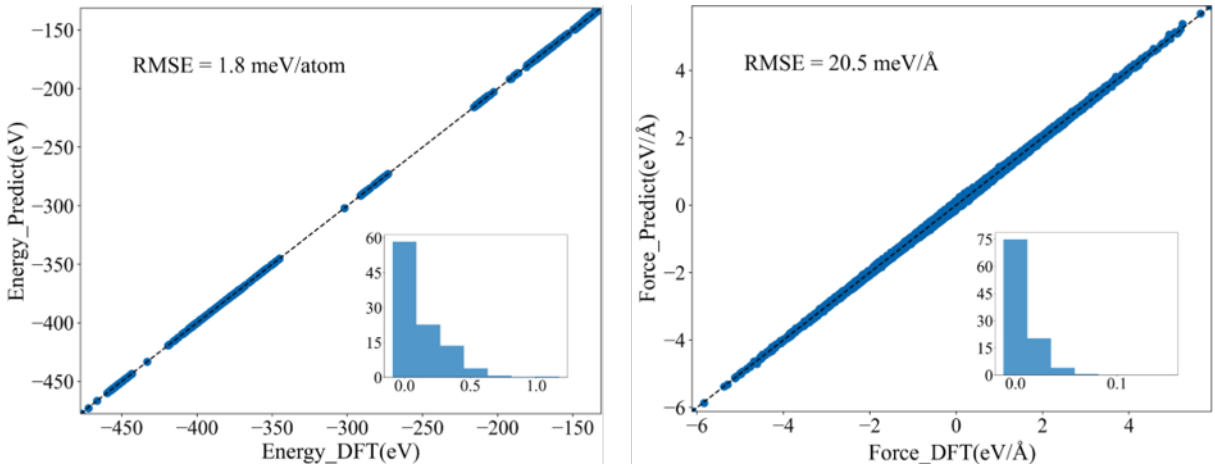


Figure 33: proportion_time

Mg and Cu Phonon-Spectrum Tests

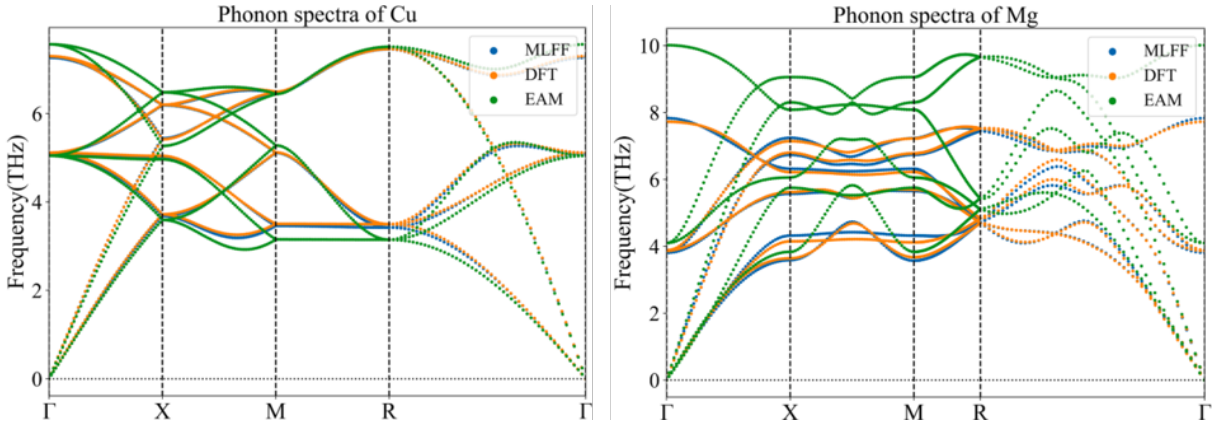


Figure 34: proportion_time

Yield Strength

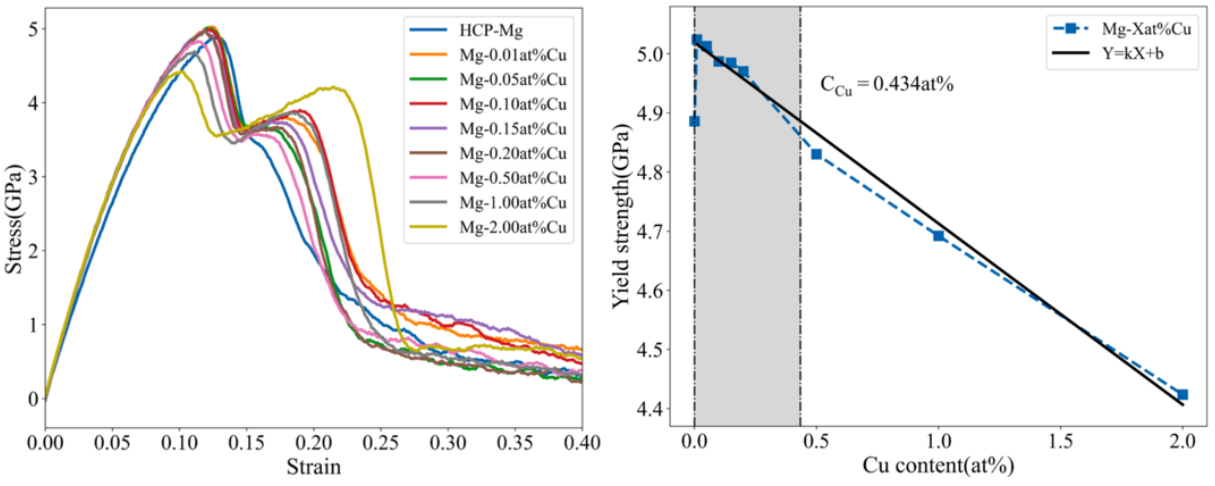


Figure 35: proportion_time

A small amount of Cu increases the alloy′ s yield strength, whereas higher Cu concentrations reduce it.

Effect of Grain Count on the Strength of Mg-Cu Alloys

Modeling Structures with Different Grain Counts

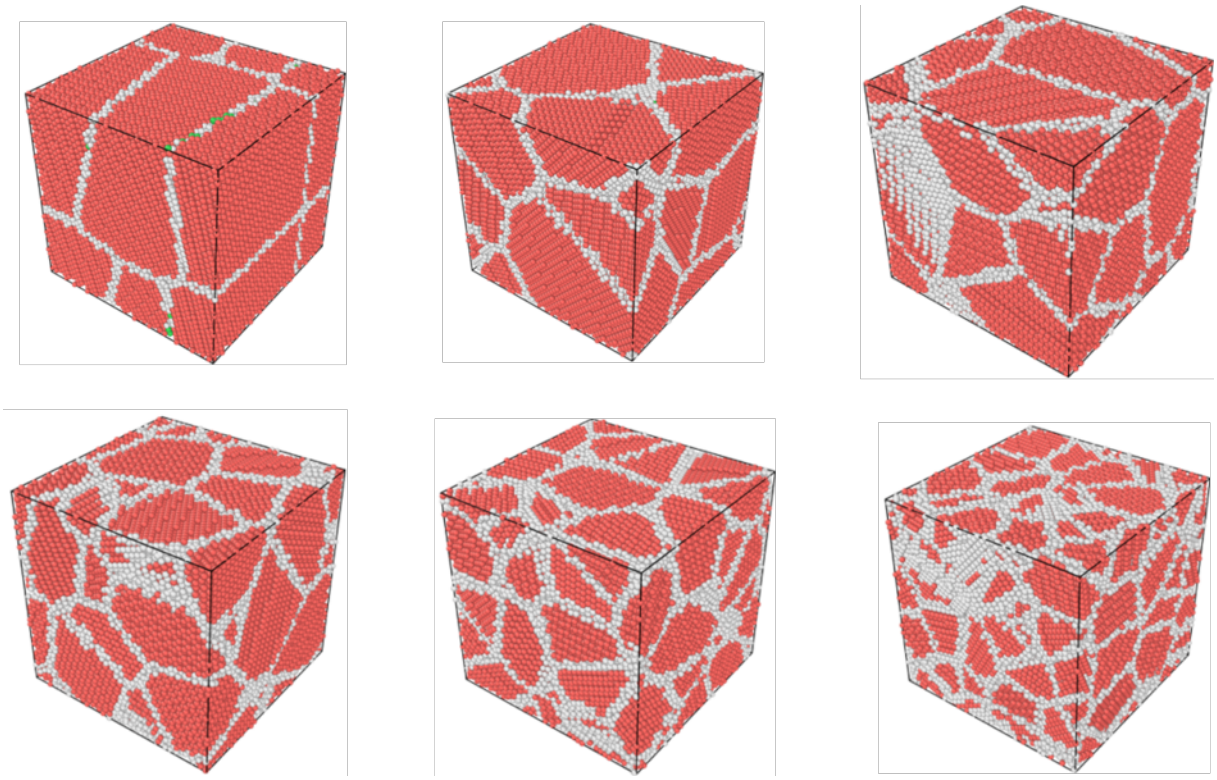


Figure 36: proportion_time

Grain refinement has only a minor effect on alloy strength.

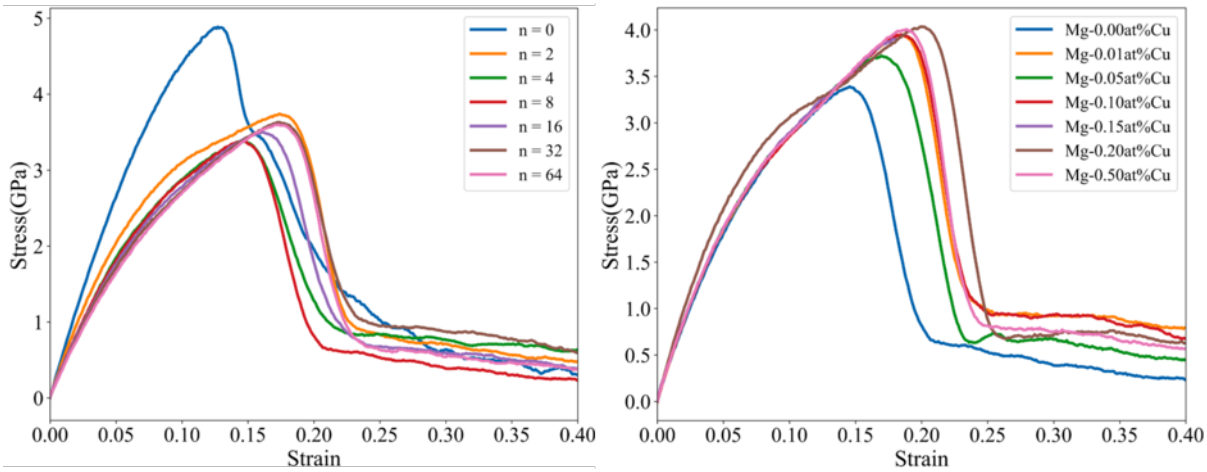


Figure 37: proportion_time

8.7 Calculating the Thermal Conductivity of Amorphous Silicon with a Machine-Learning Force Field

Yufeng Huang et al. Phys. Rev. B 99, 064103 (2019)

Training-Set Accuracy

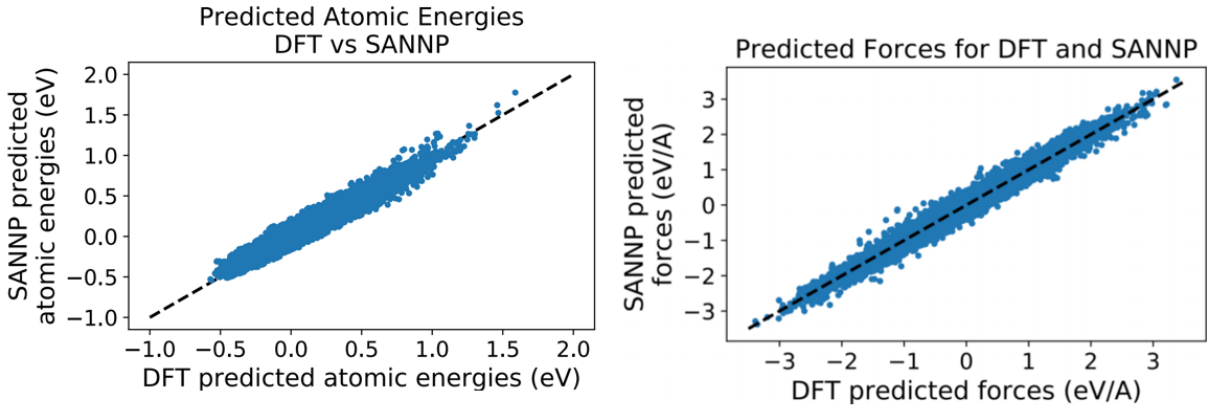


Figure 38: proportion_time

Test-Set Accuracy

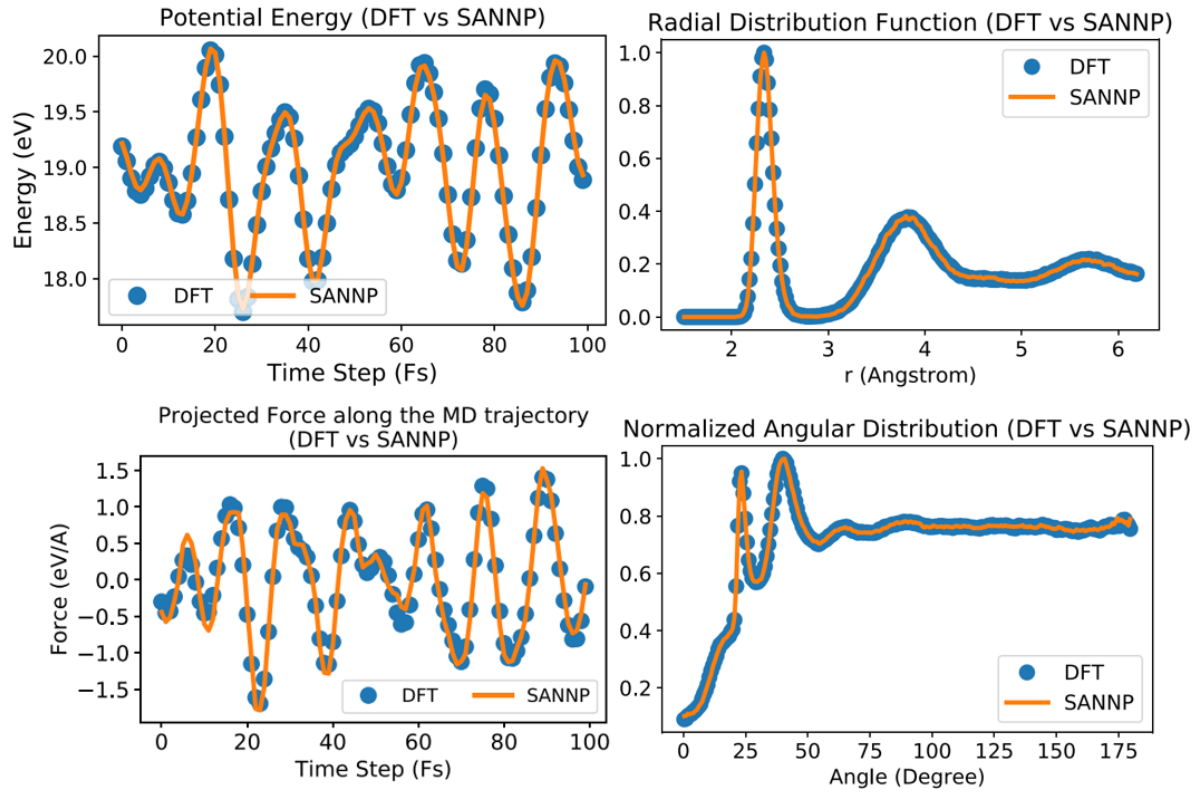


Figure 39: proportion_time

Green-Kubo Formula

$$\kappa_{\mu\nu}(t) = \frac{1}{k_B T^2 V} \int_0^t dt' \langle J_\mu(0) J_\nu(t') \rangle$$

$$\mathbf{J} = \sum_i \mathbf{v}_i E_i + \sum_i \mathbf{r}_i \frac{d}{dt} E_i$$

$$= \sum_i \mathbf{v}_i (K_i + U_i) + \sum_i \mathbf{W}_i \cdot \mathbf{v}_i$$

Thermal Conductivity

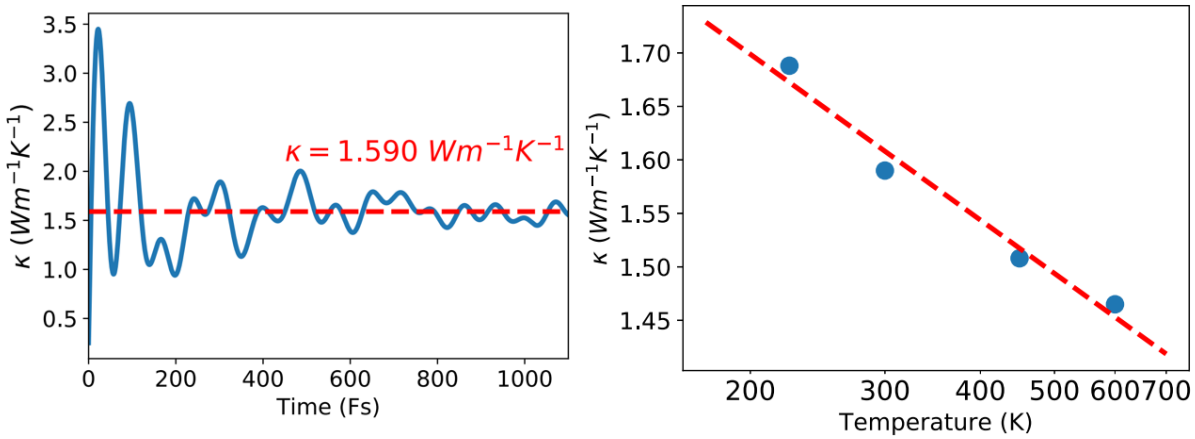


Figure 40: proportion_time